NMAM INSTITUTE OF TECHNOLOGY, NITTE

Department of Computer Science and Engineering

Operating Systems Laboratory

Line-by-Line Study Guide

Experiments 1–10 explained for the lab MSE

Companion to the lab manual
Academic Year 2026–27

Prepared for
Prajwal Jogi

Every program in this guide was compiled and run (gcc 13.3, Ubuntu 24.04). Outputs, signal behaviour and scheduling results were checked against the Linux man pages, textbook examples and an independent simulator. The line numbers always match your lab manual.

Read this first

You asked for a guide that explains every line of the ten programs, with the best methods, checked against the internet. This is that guide. Before you start, here are the seven most important things I found while checking your manual:

1. **All ten programs reproduce your manual's output.** I rebuilt each program, compiled it and ran it (Experiment 10 was typed from your handwritten notebook pages). The three scheduling programs (FCFS, SJF, Round Robin) also matched an independent simulator on 400 random inputs each.
2. **Experiment 9 does not really use semaphores.** The program has no `sem_t`, no `sem_wait()`, no `sem_post()` and no threads. It is a menu-driven simulation of the bounded buffer. The aim says "using semaphores", so Chapter 9 explains your program *and* gives two upgraded versions that use real POSIX semaphores. Learn all three.
3. **Experiment 1:** the script starts with `#!/bin/bash`, but your C program runs it with `/bin/sh`, which ignores that line. `read -p` is not part of the POSIX standard. It works on Ubuntu 24.04 (I tested), but a stricter shell may reject it, so a portable version is given.
4. **Experiment 3:** the order of the "Parent Created..." and "Child..." lines is decided by the scheduler. The tidy order in the manual is only one possibility. Your own run may look different, and that is fine.
5. **Experiment 5:** `printf()` and `exit()` are not safe to call inside a signal handler (the `signal-safety(7)` man page says so). It is fine for the lab, but the best method uses `sigaction()` and `write()`.
6. **Experiments 6–8** assume every process arrives at time 0. If your examiner adds arrival times, use the "twist" programs at the end of each chapter (tested against textbook examples).
7. **Experiment 10** (Banker's algorithm): your notebook program loops for ever on some unsafe inputs, because the flag `found` is never reset. One extra line fixes it. The program also tests safety only; the resource-request step ("can this request be granted?") is explained and tested at the end of Chapter 10.

How each chapter is organised

Big picture	What the program does, in plain English.
Concepts	The theory you must know before the code makes sense.
Program map	A short table: which lines do which job.
Line by line	Every line of your program, in order, with the **same line numbers as your manual**.
Dry run	The program executed by hand on paper, step by step.
Output explained	Why each line of your recorded output appears.
Gotchas / best method	Real pitfalls and a better version of the code (tested).
Write from memory	A short recipe you can reproduce in the exam.
Viva questions	Likely questions with short answers.

The coloured boxes

Key idea

Blue boxes hold the most important idea of a section.

Watch out

Orange boxes warn about mistakes and surprises.

Best method

Green boxes show the better or safer way to do something.

Write it from memory

Teal boxes are short recipes for the exam.

Viva questions

Purple boxes hold viva questions with answers.

In code listings, **system calls** are shown in **teal bold**, C keywords in **blue**, strings in orange and comments in *green*. Indentation was added to the code for readability; it does not change the program.

A suggested plan for the week

Day	What to study
1	“Start here” chapter, then Experiment 1
2	Experiments 2 and 3
3	Experiments 4 and 5
4	Experiments 6 and 7 (do the dry runs on paper yourself)
5	Experiment 8, then Experiment 9 (your menu version)
6	Experiment 9 (the two semaphore versions), then Experiment 10 and Appendix A
7	Cover the code and use the “Write it from memory” boxes; answer the viva questions aloud; try the exam-style variations in Appendix E

Contents

Read this first **1**

Start here: C and Linux basics **8**

0.1 From a `.c` file to a running program . . . 8
0.2 The anatomy of a C program . . . 8
0.3 The header files used in your manual . . . 9
0.4 Variables and types . . . 9
0.5 Strings and arrays . . . 10
0.6 Printing and reading: `printf` and `scanf` . . . 10
0.7 Making decisions and repeating . . . 10
0.8 Pointers, just enough . . . 11
0.9 Structures . . . 11
0.10 Errors and exit status . . . 11
0.11 System calls and library functions . . . 12
0.12 Processes in one page . . . 12

1 exec() system call **13**

1.1 Big picture . . . 13
1.2 Concepts you need first . . . 13
1.3 The program at a glance . . . 15
1.4 Line-by-line explanation . . . 15
1.5 Dry run . . . 17
1.6 The output, explained . . . 18
1.7 Gotchas and the best method . . . 18

2 File read using system calls **21**

2.1 Big picture . . . 21
2.2 Concepts you need first . . . 21
2.3 The program at a glance . . . 22
2.4 Line-by-line explanation . . . 22
2.5 Dry run . . . 24
2.6 The output, explained . . . 24
2.7 Gotchas and the best method . . . 25

3 Child processes using fork() 27
3.1 Big picture . . . 27
3.2 Concepts you need first . . . 27
3.3 The program at a glance . . . 28
3.4 Line-by-line explanation . . . 28
3.5 Dry run . . . 30
3.6 The output, explained . . . 30
3.7 Gotchas and the best method . . . 31

4 File write using system calls 34
4.1 Big picture . . . 34
4.2 Concepts you need first . . . 34
4.3 The program at a glance . . . 35
4.4 Line-by-line explanation . . . 35
4.5 Dry run . . . 37
4.6 The output, explained . . . 38
4.7 Gotchas and the best method . . . 38

5 Signal handling 41
5.1 Big picture . . . 41
5.2 Concepts you need first . . . 41
5.3 The program at a glance . . . 42
5.4 Line-by-line explanation . . . 43
5.5 Dry run . . . 45
5.6 The output, explained . . . 45
5.7 Gotchas and the best method . . . 46

6 FCFS scheduling 49
6.1 Big picture . . . 49
6.2 Concepts you need first . . . 49
6.3 The program at a glance . . . 50
6.4 Line-by-line explanation . . . 50
6.5 Dry run . . . 52
6.6 The output, explained . . . 53
6.7 Gotchas and the best method . . . 53

7 SJF scheduling 56
7.1 Big picture . . . 56

7.2 Concepts you need first . . . 56
7.3 The program at a glance . . . 57
7.4 Line-by-line explanation . . . 57
7.5 Dry run . . . 59
7.6 The output, explained . . . 60
7.7 Gotchas and the best method . . . 60

8 Round Robin scheduling 64
8.1 Big picture . . . 64
8.2 Concepts you need first . . . 64
8.3 The program at a glance . . . 65
8.4 Line-by-line explanation . . . 65
8.5 Dry run . . . 68
8.6 The output, explained . . . 68
8.7 Gotchas and the best method . . . 69

9 Producer–Consumer using semaphores 72
9.1 Big picture . . . 72
9.2 Concepts you need first . . . 72
9.3 Program A: your manual's program . . . 74
9.4 Program B: the menu program with real semaphores . . . 77
9.5 Program C: two threads and three semaphores . . . 79

10 Banker's algorithm (deadlock avoidance) 85
10.1 Big picture . . . 85
10.2 Concepts you need first . . . 85
10.3 The program at a glance . . . 87
10.4 Line-by-line explanation . . . 87
10.5 Dry run . . . 90
10.6 The output, explained . . . 91
10.7 Gotchas and the best method . . . 92

A Quick reference: functions and system calls 97

B Scheduling cheat sheet 99
B.1 Formulas . . . 99
B.2 Comparison . . . 99
B.3 Worked examples (all produced by the programs in this guide) . . . 100

C Commands and common errors **101**
C.1 Commands you will use . . . 101
C.2 Common errors . . . 101

D How this guide was checked, and sources **102**
D.1 What was tested . . . 102
D.2 Limits . . . 102
D.3 Sources checked on the internet . . . 103

E Likely exam variations (tested) **104**

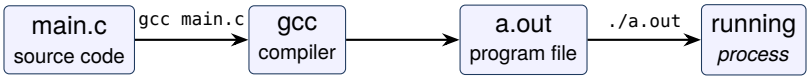
F Two mock papers (with answers) **111**
F.1 Mock paper A (Experiments 3, 6–10) . . . 111
F.2 Mock paper B (Experiments 1, 2, 4, 5) . . . 112

Start here: C and Linux basics

You said you do not know what each line means. This chapter explains every building block that the ten programs use. Read it once, slowly. Later chapters will point back to it, so you do not need to memorise it now.

0.1 From a `.c` file to a running program

You write *source code* in a text file such as `main.c`. The computer cannot run text, so the compiler `gcc` translates it into a program. Then you run that program.



Command	What it does
`gcc main.c`	Compile `main.c`; the program is saved as `a.out`.
`gcc main.c -o main`	Same, but the program is named `main`.
`./a.out`	Run the program. The `./` means "in this folder". Without it the shell only looks in the system program folders and says "command not found".
`gcc -Wall -Wextra main.c`	Show helpful warnings. Use this while learning.
`gcc main.c -pthread`	Needed when the program uses threads or semaphores (Experiment 9).
`echo $?`	Print the *exit status* of the last command (0 means success).

0.2 The anatomy of a C program

Every program in your manual has the same skeleton:

```
1 #include <stdio.h>      /* 1: bring in the printf() function  */
2
3 int main()              /* 2: the program starts here         */
4 {                       /* 3: opening brace: block starts     */
5     printf("Hello\n");  /* 4: a statement; it ends with ;     */
6     return 0;           /* 5: give the exit status 0 (= OK)   */
7 }                       /* 6: closing brace: block ends       */
```

Line 1 `#include` tells the compiler to paste a *header file* here. A header file only *announces* functions ("`printf` exists and takes these arguments"). Without it, the compiler does not know what `printf` is. Angle brackets `<...>` mean "look in the system folders".

Line 2 `main` is the function where every C program begins. `int` means it gives back a whole number to the operating system when it finishes. That number is the *exit status*.

Lines 3, 6 Braces `{ }` group statements into one block.

Line 4 A *statement* is one instruction and always ends with a semicolon. `\n` is the "new line" character.

Line 5 `return 0;` ends `main`. By convention **0 means success and any other number means failure**. Your manual uses `return 1;` when something went wrong.

Comments Text between `/*` and `*/` is ignored by the compiler. It is only for humans.

0.3 The header files used in your manual

Header	What it gives you	Where it is really needed
`stdio.h`	`printf`, `scanf`, `perror`, `fflush`, `fprintf`	every experiment
`stdlib.h`	`exit`, `malloc`	`exit()` in Exp. 3 and 5. Other programs include it but do not need it.
`unistd.h`	`read`, `write`, `close`, `fork`, `execl`, `getpid`, `getppid`, `access`, `alarm`, `sleep`, `STDOUT_FILENO`	Exp. 1–5. (Exp. 6, 7, 9 include it without using it.)
`fcntl.h`	`open` and the flags `O_RDONLY`, `O_WRONLY`, `O_CREAT`, `O_TRUNC`	Exp. 2 and 4
`sys/wait.h`	`wait`, `waitpid`	Exp. 3
`signal.h`	`signal`, `sigaction`, `SIGINT`, `SIGTERM`...	Exp. 5
`pthread.h`	`pthread_create`, `pthread_join`	Exp. 9 (included, but the manual program uses no thread)
`semaphore.h`	`sem_init`, `sem_wait`, `sem_post`	Exp. 9 upgraded versions
`errno.h`, `string.h`	`errno`, `EINTR`; `memset`	improved versions only

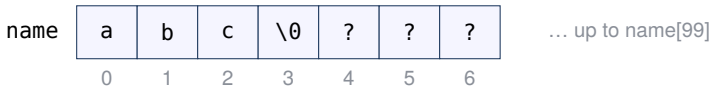
0.4 Variables and types

A *variable* is a named box in memory. Its *type* says what kind of value it holds.

Type	Meaning
`int`	a whole number such as 3 or -1
`float`, `double`	a number with a decimal point (`double` is more precise)
`char`	one character; a `char` array holds text
`ssize_t`	a *signed* size. `read()` returns it: the number of bytes, 0 at end of file, -1 on error. It must be signed so that -1 is possible.
`pid_t`	a process ID number
`struct`	your own bundle of variables (see Section 0.9)
`sem_t`, `pthread_t`	a semaphore; a thread handle (Experiment 9)

0.5 Strings and arrays

C has no "string" type. Text is an *array of characters* that ends with a special character `'\0'` (the *null terminator*). So `char name[100];` holds at most **99** visible characters, because one cell is kept for the `'\0'`.



- Index numbers start at **0**. The first cell is `name[0]`.
- `sizeof(buffer)` gives the size of a real array in bytes. For `char buffer[100]` it is 100.
- A *variable-length array* such as `struct Process P[n];` has a size that is only known when the program runs (C99 feature). The size must be positive; that is why the manual checks `n <= 0` first.

0.6 Printing and reading: `printf` and `scanf`

Code	Meaning
`%d`	print/read an `int`
`%s`	print/read a string (text)
`%c`	one character
`%f`, `%.2f`	a decimal number; `%.2f` prints exactly 2 digits after the point
`\n`, `\t`	new line; tab
`%%`	a literal percent sign

- `printf("Age %d\n", age);` replaces `%d` by the value of `age`.
- `scanf("%d", &n);` reads a number from the keyboard and stores it *inside* the variable `n`. The `&` means "address of": `scanf` must know *where* in memory to put the number. For a text array the name already is an address, so no `&` is used: `scanf("%99s", sname);`.
- **`%99s`** reads one word (it stops at a space or Enter) and stores at most 99 characters *plus* the `'\0'`. The documentation says the array must have room for width +1 characters, so `char sname[100]` with `%99s` is exactly right. Plain `%s` with no width is as unsafe as the old `gets()`: a long input overflows the array.
- `scanf` returns how many items it read successfully (or `EOF`). The manual does not check it; the improved versions do.

0.7 Making decisions and repeating

Form	Meaning
`if (c) {..} else if (c2) {..} else {..}`	run the first block whose condition is true
`for (i=0; i<n; i++) {..}`	start with `i=0`; repeat while `i<n`; add 1 after each round

Form	Meaning
`while (c) {..}`	repeat while `c` is true (check first)
`do {..} while (c);`	repeat while `c` is true (check *after*, so it runs at least once; used in Exp. 9)
`switch (x) { case 1: ..; break; default: .. }`	jump to the matching `case`. `break` leaves the switch; without it the code "falls through" into the next case.
`== != <= >=`	comparisons. Do not confuse `==` (compare) with `=` (store).
`&& || !`	AND, OR, NOT
`i++`, `item--`, `x += 2`	add 1, subtract 1, add 2 to the variable

0.8 Pointers, just enough

A *pointer* holds the address of something. You only need three facts for this manual:

- `&x` is the address of `x`. That is what `scanf` needs.
- An array name such as `buffer` already acts as the address of its first cell. `buffer + done` means "the cell `done` places further on".
- `NULL` is the "points to nothing" address. `(char*)NULL` is `NULL` with an explicit type; it ends the argument list of `execl()` (Experiment 1).

0.9 Structures

A `struct` groups related variables. In the scheduling programs each process has an id, burst time, waiting time and turnaround time:

```
struct Process { int pid, bt, wt, tat; };   /* the new type          */
struct Process P[4];                        /* an array of 4 of them */
P[2].bt = 5;                                /* dot picks one field   */
temp = P[0]; P[0] = P[1]; P[1] = temp;      /* '=' copies ALL fields */
```

The last line is a swap of two whole processes. It is used by the sorting step of SJF (Experiment 7).

0.10 Errors and exit status

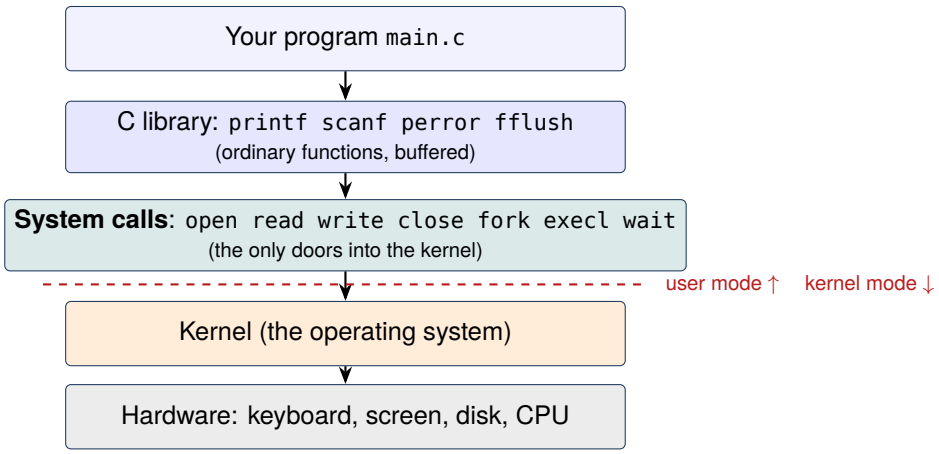
Most system calls return -1 when they fail, and set a global number called `errno` that says why. `perror("text")` prints `text: reason` on the error output. For example, when a file does not exist, `perror("Error")` prints `Error: No such file or directory`.

Reason shown	Meaning
No such file or directory	the name is wrong or the file is in another folder (`ENOENT`)
Permission denied	you are not allowed to read/write it (`EACCES`)
Interrupted system call	a signal arrived while the call was waiting (`EINTR`)

Exit status in one line

`return 0;` (or `exit(0)`) means "everything went fine". A non-zero value means "an error happened". The shell shows it with `echo $?`. Several manual programs print an error but still `return 0`; the improved versions return 1 instead.

0.11 System calls and library functions



- A **system call** asks the *kernel* to do something that only the kernel may do (touch a file, create a process). `open`, `read`, `write`, `close`, `fork`, `execl`, `wait`, `alarm` are system calls (strictly, a few such as `execl` are thin library wrappers over the real call `execve`).
- `printf`, `scanf` and `perror` are **library functions**. They keep a buffer in your program and call `write` or `read` underneath.
- **Manual pages:** `man 2 open` shows a system call (section 2); `man 3 printf` shows a library function (section 3). Use `man 2 fork`, `man 7 signal` and so on whenever you are unsure.
- **File descriptors.** Every open file is a small number. Three are always open: **0** = standard input (keyboard), **1** = standard output (screen), **2** = standard error (screen, used by `perror`). `open` returns the lowest free number, normally 3.
- **Buffering.** On a terminal, `printf` output is flushed at each newline. When output goes to a file or pipe it is held until the buffer is full or the program ends. `fflush(stdout)` forces it out now. This matters in Experiments 2 and 3.

0.12 Processes in one page

A *program* is a file on disk. A *process* is a program that is running. Every process has a **PID** (its own number) and a **PPID** (the PID of its parent). Commands to look at them: `ps -f` (list your processes with PID and PPID), `kill PID` (send a signal to a process), `echo $$` (PID of the current shell). Experiments 1, 3 and 5 are all about processes: replacing one (`exec`), copying one (`fork`) and signalling one (`kill`).

EXPERIMENT 1

exec() system call

Aim from the lab manual

Write a C program using the `exec()` system call: (a) read the name of a shell script from the keyboard; (b) execute that script using `/bin/sh`; (c) display an appropriate error message if the script cannot be found or executed; (d) the shell script should prompt for three numbers, then find and display the greatest of the three.

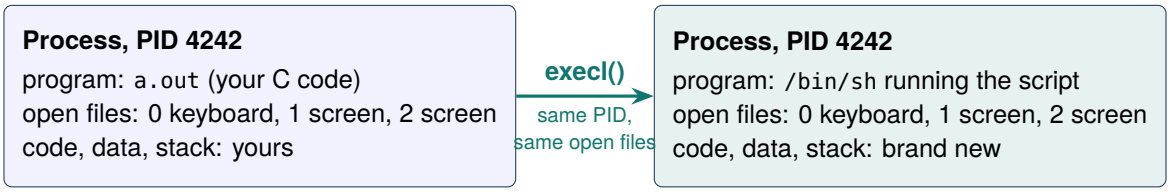
1.1 Big picture

There are two small files. `largest.sh` is a shell script that finds the biggest of three numbers. `main.c` is a *launcher*: it asks for the script's name, checks that the file exists and can be read, and then **replaces itself** with the shell `/bin/sh`, which runs the script.

The key word is *replaces*. After a successful `execl()` your C program no longer exists, so nothing after that line ever runs. This is the one idea of Experiment 1.

1.2 Concepts you need first

1.2.1 fork() copies, exec() replaces



`fork()` (Experiment 3) makes a *new* process. `exec` does not make any process: it loads a different program into the *same* process. The process number (PID) does not change.

What survives an `exec` and what does not:

- **Stays the same:** the PID and parent PID, the current folder, the environment variables, and every open file descriptor (so keyboard and screen are still connected). That is why the script can read from your keyboard.
- **Replaced:** the code, the variables, the heap and the stack. Signal handlers go back to their default action.

1.2.2 The exec family

There are six library functions. The letters in their names tell you how they work: **l** = arguments as a *list*, **v** = arguments in a *vector* (array), **p** = search the `PATH` for the program, **e** = you also pass the *environment*.

Function	Letters	Meaning
`execl`	list	full path of the program, then each argument, ending with a null pointer
`execlp`	list + PATH	same, but the program is searched in `PATH` (`execlp("sh","sh",f,(char*)NULL)`)
`execle`	list + env	like `execl` plus an array of environment strings
`execv`	vector	arguments are already in an array `argv[]` that ends with `NULL`
`execvp`	vector + PATH	array of arguments and a `PATH` search
`execve`	vector + env	the real *system call*; the other five are library wrappers around it

All of them return **only if an error occurs** (return value -1, reason in `errno`). When the call succeeds there is no one left to return to. By convention the first argument after the path is the program's own name; the program sees it as `argv[0]`.

1.2.3 How a script gets executed

You type	What really happens
`sh largest.sh`	The shell program is started and *reads the script as a text file*. The file only needs **read** permission.
`./largest.sh`	The kernel opens the file, reads the first line (`#!/bin/bash`, the *shebang*) and starts that interpreter. Now the file needs **execute** permission too.

Your program uses the first way, so it checks for `R_OK` (readable), not `X_OK`. The four modes of `access()` are `F_OK` (exists), `R_OK` (readable), `W_OK` (writable), `X_OK` (executable).

1.2.4 Shell-script basics for largest.sh

- A variable is set as `a=5` and used as `$a`. `read a b c` reads one line and splits it at spaces into three variables.
- `[ ... ]` is itself a command (the same as `test`). The spaces just inside the brackets are **required**.
- Number tests: `-eq` equal, `-ne` not equal, `-gt` greater, `-ge` greater or equal, `-lt` less, `-le` less or equal. (Strings use `=` instead.)
- `A && B` runs `B` only if `A` succeeded: a logical AND.
- `if .. then .. elif .. then .. else .. fi` is the decision structure. `fi` is `if` spelled backwards.

1.3 The program at a glance

Lines	Job
1–3	include the headers
5–6	start of `main`; an array to hold the script name
7–8	ask for the script name and read it
9–13	check the file exists and is readable; if not, print an error and stop
14–15	replace this process by `/bin/sh` running the script; print an error only if that fails
17–18	reached only when `execl` failed

1.4 Line-by-line explanation

1.4.1 The C program (main.c)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
```

Line 1 `stdio.h` provides `printf`, `scanf` and `perror`.

Line 2 `stdlib.h` is not needed here (nothing from it is used). It does no harm.

Line 3 `unistd.h` provides `access()`, `execl()` and the constant `R_OK`.

Line 4 A blank line, only for readability.

```
5 int main() {
6     char sname[100];
7     printf("Enter shell script name: ");
8     scanf("%99s", sname);
```

Line 5 `int main() {` is where the program starts. The brace on the same line is only a style choice.

Line 6 `char sname[100];` reserves 100 characters for the script name: 99 letters plus the end marker `'\0'`. It is not initialised, so it holds garbage until `scanf` fills it.

Line 7 Prints the prompt. There is no `\n`, so the cursor stays on the same line. On a terminal the C library flushes this prompt before it waits for your typing, so you see it.

Line 8 `scanf("%99s", sname);` reads **one word** (it stops at a space or Enter) and stores at most 99 characters. There is no `&` because an array name is already an address. A script name that contains a space cannot be entered this way.

```
9     /* the script must exist and be readable */
10    if(access(sname, R_OK) == -1) {
11        perror("Error: cannot open script");
12        return 1;
13    }
```

Line 9 A comment: the script must exist and be readable.

Line 10 `access(sname, R_OK)` asks the kernel: "may I read this file?" It returns **0 if yes and** -1 **if no** (and sets `errno`). A missing file also gives -1. So `== -1` means "the check failed".

Line 11 `perror("Error: cannot open script");` prints your text, then a colon, then the system's reason, for example `No such file or directory`. This produces the second line of your recorded output.

Line 12 `return 1;` leaves `main` at once with exit status 1 (failure). `execl` is never reached.

Line 13 Closing brace of the `if` block.

```
14      if(execl("/bin/sh", "sh", sname, (char*)NULL) == -1)
15          perror("Error: cannot execute script");
16
17      return 0;
18  }
```

Line 14 The most important line: `if(execl("/bin/sh", "sh", sname, (char*)NULL) == -1)`

- `"/bin/sh"` — the full path of the program to start: the shell.
- `"sh"` — becomes `argv[0]`, the name the new program sees for itself.
- `sname` — becomes `argv[1]`. The shell treats it as "run the commands in this file". The whole call is the same as typing `sh largest.sh` in a terminal.
- `(char*)NULL` — marks the end of the list. `execl` takes any number of arguments and cannot count them, so it looks for a null pointer. The cast makes it a pointer of the correct size.
- **Return value:** if the call works, it never returns. So `== -1` can only ever be tested when exec *failed*.

Line 15 `perror("Error: cannot execute script");` belongs to the `if` above (a single statement needs no braces). It runs only when `execl` failed, for example if `/bin/sh` could not be started.

Line 16 A blank line.

Line 17 `return 0;` ends the program. It is reached only after a failed exec. (It would be more honest to return 1 here.)

Line 18 Closing brace of `main`.

1.4.2 The shell script (largest.sh)

```
1  #!/bin/bash
2  read -p "Enter 3 numbers: " a b c
3  if [ $a -ge $b ] && [ $a -ge $c ]
4  then
5      echo "$a is the greatest"
6  elif [ $b -ge $a ] && [ $b -ge $c ]
7  then
8      echo "$b is the greatest"
9  else
10     echo "$c is the greatest"
11 fi
```

Line 1 `#!/bin/bash` is the *shebang*. It matters only when the file is executed directly (`./largest.sh`). Your C program runs `sh largest.sh`, so here this line is just a comment

and `/bin/sh` (not bash) runs the script.

Line 2 `read -p "Enter 3 numbers: " a b c` prints the prompt, reads one line and splits it at spaces: the first word goes into `a`, the second into `b`, the rest into `c`. The option `-p` (prompt) is a bash feature; see the gotchas below.

Line 3 `if [ $a -ge $b ] && [ $a -ge $c ]` means "$a \geq b$ *and* $a \geq c$". The second test runs only if the first is true.

Line 4 `then` starts the commands to run when the condition is true.

Line 5 `echo "$a is the greatest"` prints the text; `$a` is replaced by its value.

Line 6 `elif [ $b -ge $a ] && [ $b -ge $c ]` is the next test: "$b \geq a$ and $b \geq c$".

Line 7 `then` again.

Line 8 Prints `b` as the greatest.

Line 9 `else` means every remaining case. If `a` and `b` both lost, `c` must be the greatest, so no test is needed.

Line 10 Prints `c` as the greatest.

Line 11 `fi` ends the `if` statement.

Why -ge and not -gt?

Try the input `5 5 3`. With `-gt` the first test (`5 -gt 5`) fails, the second fails too, and the `else` wrongly prints 3. With `-ge` the first test succeeds and the answer 5 is correct. Using "greater *or equal*" makes ties work.

1.5 Dry run

Case A: the user types `largest.sh`, then `3 4 2`.

#	Where	What happens
1	C line 7	the prompt `Enter shell script name:` appears
2	C line 8	`sname` becomes `largest.sh`
3	C line 10	`access("largest.sh", R_OK)` returns 0, so the condition is false and lines 11–12 are skipped
4	C line 14	the kernel loads `/bin/sh` into this very process. The C program is gone; the PID is the same
5	script line 2	the shell prints `Enter 3 numbers:` and reads `3 4 2`, so $a = 3$, $b = 4$, $c = 2$
6	script line 3	$3 \geq 4$ is false, so the `&&` second test is not even run. Go to `elif`
7	script line 6	$4 \geq 3$ true and $4 \geq 2$ true, so line 8 runs
8	script line 8	prints `4 is the greatest`
9	end	the script ends, so the process ends. C lines 15–18 never ran

Case B: the user types `large.sh` (no such file). `access` returns -1 with `errno` = `ENOENT`, so line 11 prints `Error: cannot open script: No such file or directory` and line 12 stops the program with status 1.

Case C: the file exists but `execl` fails (very rare, e.g. `/bin/sh` missing). Line 14 evaluates to -1, line 15 prints `Error: cannot execute script: …`, and line 17 ends the program.

1.6 The output, explained

```
gcc main.c
./a.out
Enter shell script name: largest.sh     <- printed by C line 7, "largest.sh" typed by you
Enter 3 numbers: 3 4 2                  <- printed by the SCRIPT (line 2), not by C
4 is the greatest                       <- script line 8

./a.out
Enter shell script name: large.sh
Error: cannot open script: No such file or directory   <- C line 11 + errno text
```

Notice that the prompt `Enter 3 numbers:` comes from the script. By then your C program has already been replaced. I ran both cases through a pseudo-terminal and the output is identical to your manual.

1.7 Gotchas and the best method

1. The shebang is ignored, and read -p is not portable

Because the program calls `sh largest.sh`, the `#!/bin/bash` line is not used: the script is run by `/bin/sh`, not by bash. `read -p` (print a prompt) is a bash feature that the POSIX standard does not define; the standard `read` accepts only `-r`. On Ubuntu 24.04, where `/bin/sh` is dash 0.5.12, it happens to work — I tested it — so your recorded output is genuine. But a stricter `/bin/sh` is free to reject it, so portable scripts print the prompt with `printf` and then call `read`. Check which shell you have with `ls -l /bin/sh`. The portable version is below.

2. Other small problems

- Line 17 returns 0 (success) even after a failed `execl`. Return 1.
- `scanf` is not checked. If the input ends (Ctrl+D), `sname` stays uninitialised garbage. In my test with empty input the program did print the "cannot open script" message, but only by luck: this is undefined behaviour.
- The script uses unquoted `$a`. If the user types fewer than three numbers, `[` complains. Quote variables as `"$a"` and check that all three exist.
- Between `access()` and `execl()` the file could change. Real programs just try `execl` and handle the failure. For a lab this is not a concern.

Best method: portable launcher and script (tested)

```c
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    char sname[100];
    printf("Enter shell script name: ");
    if (scanf("%99s", sname) != 1) {        /* did scanf really read a word? */
        fprintf(stderr, "Error: no script name given\n");
```

```
        return 1;
    }
    if (access(sname, R_OK) == -1) {        /* exists and readable? */
        perror("Error: cannot open script");
        return 1;
    }
    execl("/bin/sh", "sh", sname, (char *)NULL);
    /* we reach this line ONLY if execl() failed */
    perror("Error: cannot execute script");
    return 1;
}
```

```
#!/bin/sh
printf "Enter 3 numbers: "
read a b c
if [ -z "$c" ]; then
    echo "Please enter three numbers"
    exit 1
fi
if [ "$a" -ge "$b" ] && [ "$a" -ge "$c" ]
then
    echo "$a is the greatest"
elif [ "$b" -ge "$a" ] && [ "$b" -ge "$c" ]
then
    echo "$b is the greatest"
else
    echo "$c is the greatest"
fi
```

What changed: `scanf` is checked; errors go to the error stream; the program returns 1 on every failure; the script is POSIX-clean (`printf` plus `read` instead of `read -p`, quoted variables, and a check that the third number exists). It works with dash, bash and every other `/bin/sh`.

Method 2: fork + exec + wait — how a real shell runs a command

Replacing your own process is rare. Normally a program does `fork()`, lets the *child* call `exec`, and the *parent* waits and then continues. (`fork` is explained in Chapter 3.)

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    char sname[100];
    int status;
    printf("Enter shell script name: ");
    if (scanf("%99s", sname) != 1) return 1;
    if (access(sname, R_OK) == -1) {
        perror("Error: cannot open script");
        return 1;
    }
    pid_t pid = fork();
    if (pid == 0) {                          /* child: becomes the shell */
        execl("/bin/sh", "sh", sname, (char *)NULL);
        perror("Error: cannot execute script");  /* only if exec failed */
        _exit(127);
    }
    waitpid(pid, &status, 0);                /* parent: wait for child */
    if (WIFEXITED(status))
```

```
        printf("Parent: script finished, exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

The child becomes the shell; the parent survives, calls `waitpid` and reads the script's exit status with `WIFEXITED` and `WEXITSTATUS`. `_exit(127)` is the usual status for "could not execute". I tested this version too.

Write it from memory

1. Include `stdio.h` and `unistd.h`. Declare `char sname[100];`.
2. Prompt, then `scanf("%99s", sname);`
3. `if (access(sname, R_OK) == -1) { perror("..."); return 1; }`
4. `execl("/bin/sh", "sh", sname, (char*)NULL);`
5. On the next line `perror("...");` and `return 1;` — only a failed exec gets here.

Order to remember: **ask** → **check** → **exec** → **complain**. Script: `read` three numbers, then `if / elif / else` with `-ge` and `&&`.

Viva questions

Q. What does exec do?
A. It replaces the program of the current process with a new program. The PID stays the same.

Q. Does exec create a new process?
A. No. `fork` creates a process; `exec` only changes what it runs.

Q. When does execl return?
A. Only on failure, with the value -1 and `errno` set. On success the old program is gone.

Q. Why is the last argument (char*)NULL?
A. `execl` is variadic and cannot know how many arguments follow, so the list must end with a null pointer. The cast gives it the correct pointer type.

Q. What is argv[0] in this program?
A. The string `sh`: the new program's own name, by convention.

Q. Why does the script get your keyboard input?
A. Open file descriptors survive `exec`. Descriptor 0 is still the keyboard.

Q. Why check R_OK and not X_OK?
A. The shell is started and it reads the script as data. Only read permission is needed.

Q. Name the exec variants and what the letters mean.
A. `execl, execlp, execle, execv, execvp, execve`. l = list, v = vector, p = search PATH, e = pass environment.

Q. How can the parent keep running after the script?
A. Use `fork()`; the child calls `exec`; the parent calls `wait()`.

Q. What does perror print?
A. Your text, a colon, and the system message for the current `errno`, on the error stream.

EXPERIMENT 2

File read using system calls

Aim from the lab manual

Write a C program that uses appropriate unique system calls to read the contents of a file and display it on the terminal. The program should prompt the user to enter the filename from the keyboard.

2.1 Big picture

The program asks for a filename, **opens** the file, **reads** it piece by piece, **writes** each piece to the screen, and **closes** the file. "Unique system calls" means you must use the raw calls `open`, `read`, `write` and `close`, not library helpers such as `fopen`, `fgets` or `cat`.

2.2 Concepts you need first

2.2.1 File descriptors

When you `open` a file the kernel gives you a small number, the **file descriptor** (fd). Every later call uses that number instead of the name. Each process has its own table of descriptors, and three are already filled when it starts:

fd	points to
`0`	stdin: the keyboard
`1`	stdout: the screen
`2`	stderr: the screen (perror writes here)
`3`	contents.txt, opened by open()

`open` always returns the *lowest unused* number, so the first file you open gets **3**.

2.2.2 The four system calls

Call	Meaning and return value
`int open(path, flags)`	open the file; returns a descriptor, or -1 on error. `O_RDONLY` = read only.
`ssize_t read(fd, buf, count)`	read *up to* `count` bytes into `buf`. Returns the number of bytes actually read, **0 at end of file**, -1 on error.
`ssize_t write(fd, buf, count)`	write `count` bytes from `buf` to `fd`. Returns the number of bytes written, -1 on error.
`int close(fd)`	release the descriptor.

Three facts about read()

1. It may return *fewer* bytes than you asked for, so you must use its return value, never the buffer size.
2. The kernel remembers the *file offset*: each call continues where the last one stopped.
3. The data it delivers is raw bytes. It is **not** a C string (no `'\0'` at the end), so you cannot print it with `printf("%s")`. You must say how many bytes to write.

2.3 The program at a glance

Lines	Job
1–4	headers
8–10	variables: file name, data buffer, descriptor, byte count
11–12	ask for and read the filename
13–18	open the file; stop with an error message if that fails
19–21	print a heading and flush it
22–30	the loop: read a chunk, write the same chunk, repeat until end of file
31–35	report a read error if there was one, close the file, finish

2.4 Line-by-line explanation

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5
6  int main()
7  {
8      char fname[100], buffer[100];
9      int fd;
10     ssize_t n;
```

Line 1 `stdio.h`: `printf`, `scanf`, `perror`, `fflush`, `stdout`.

Line 2 `stdlib.h`: not used here, harmless.

Line 3 `unistd.h`: `read`, `write`, `close` and the constant `STDOUT_FILENO` (which is 1).

Line 4 `fcntl.h`: `open` and the flag `O_RDONLY`.

Lines 6–7 `int main()` and the opening brace.

Line 8 Two arrays. `fname` holds the file name (up to 99 characters). `buffer` is the *chunk buffer*: the file is copied through it, at most 100 bytes at a time.

Line 9 `int fd;` will hold the file descriptor returned by `open`.

Line 10 `ssize_t n;` holds the number of bytes returned by `read`. It is *signed* so that it can hold -1.

```
11     printf("Enter filename: ");
12     scanf("%99s", fname);
13     fd = open(fname, O_RDONLY);
```

```
14      if (fd == -1)
15      {
16          perror("Error");
17          return 1;
18      }
```

Line 11 Prompt (no new line, so you type on the same line).

Line 12 `scanf("%99s", fname);` reads one word into `fname`, at most 99 characters.

Line 13 `fd = open(fname, O_RDONLY);` asks the kernel to open the file for reading only. On success `fd` is the new descriptor (normally 3); on failure it is -1.

Lines 14–15 `if (fd == -1)` and the opening brace: the open failed.

Line 16 `perror("Error");` prints `Error: No such file or directory` (or another reason such as `Permission denied`).

Line 17 `return 1;` stops the program with a failure status.

Line 18 Closing brace.

```
19      printf("file contents:\n");
20      /* flush stdio before using write() directly */
21      fflush(stdout);
22      /* keep reading until end of file */
```

Line 19 Prints the heading with `printf`, a *library* function with its own buffer.

Line 20 A comment explaining the next line.

Line 21 `fflush(stdout);` pushes the heading out of that buffer *now*. The file data will be sent by `write`, which bypasses the buffer. Without the flush, the heading could appear *after* the file content when output is redirected (see the gotchas).

Line 22 A comment: keep reading until the end of the file.

```
23      while ((n = read(fd, buffer, sizeof(buffer))) > 0)
24      {
25          if (write(STDOUT_FILENO, buffer, n) != n)
26          {
27              perror("Error");
28              break;
29          }
30      }
31      if (n == -1)
32          perror("Error");
33      close(fd);
34      return 0;
35  }
```

Line 23 The heart of the program: `while ((n = read(fd, buffer, sizeof(buffer))) > 0)`

- `read(fd, buffer, sizeof(buffer))` reads up to 100 bytes from the file into `buffer` . `sizeof(buffer)` is 100 because `buffer` is a real array.
- `n = ...` stores the return value in `n`.
- The **extra parentheses** around the assignment are essential. Without them `n = read(...) > 0` would first compare and then store 0 or 1 in `n`.
- `> 0` keeps looping while at least one byte arrived. The loop stops when `read` returns

0 (end of file) or −1 (error).

Lines 24, 30 Braces of the loop body.

Line 25 `write(STDOUT_FILENO, buffer, n)` writes exactly `n` bytes (not 100: the last chunk is usually shorter) to descriptor 1, the screen. If the number written differs from `n`, something went wrong.

Lines 26–29 The error branch: `perror("Error");` and `break;` (leave the loop).

Line 31 `if (n == -1)` is true only if the loop ended because `read` failed.

Line 32 `perror("Error");` reports that read error.

Line 33 `close(fd);` releases the descriptor. (The kernel would also do it at exit, but closing is good habit.)

Line 34 `return 0;` success.

Line 35 Closing brace of `main`.

2.5 Dry run

Suppose the file has **250 bytes**. Each pass of the loop works like this:

read call	n returned	write call	file offset afterwards
1	100	`write(1, buffer, 100)`	100
2	100	`write(1, buffer, 100)`	200
3	50	`write(1, buffer, 50)`	250
4	0 (end of file)	none — loop ends	250

Your own file `contents.txt` holds `Hello World` and a new line, 12 bytes: the first `read` returns 12, the `write` prints 12 bytes, the second `read` returns 0 and the loop ends.

2.6 The output, explained

```
Enter filename: contents.txt
file contents:
Hello World

Enter filename: cont.txt
Error: No such file or directory
```

The first run goes through lines 13, 19, the loop and 33. In the second run `open` returns −1 (`errno` = `ENOENT`), so line 16 prints the reason and line 17 stops the program before the heading is printed.

2.7 Gotchas and the best method

1. Why fflush(stdout) matters — tested

When output goes to a pipe or file, `printf` text waits in its buffer, but `write` goes straight out. I removed line 21 and ran `printf 'contents.txt\n' | ./a.out | cat`:

```
Hello World                        <- data from write(), sent at once
Enter filename: file contents:     <- printf text, flushed only when the program ended
```

With line 21 the order is correct. On a plain terminal you will not notice, because a newline flushes the buffer.

2. Other points

- **Small buffer, many system calls.** Each `read` and `write` is a trip into the kernel. A 1 MB file needs about 10 486 reads with a 100-byte buffer but only 256 with a 4096-byte buffer.
- `write` may write fewer bytes than asked, and a signal can interrupt it (`EINTR`). Line 25 treats any shortfall as an error; robust code loops (see below).
- Do not print `buffer` with `printf("%s", buffer)`: it has no terminating `'\0'`.

Best method: robust copy loop (tested on a 10 000-byte file)

```c
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>

int main(void)
{
    char fname[256], buf[4096];
    printf("Enter filename: ");
    if (scanf("%255s", fname) != 1) return 1;
    int fd = open(fname, O_RDONLY);
    if (fd == -1) { perror(fname); return 1; }
    printf("file contents:\n");
    fflush(stdout);
    ssize_t n;
    while ((n = read(fd, buf, sizeof buf)) > 0) {
        ssize_t done = 0;
        while (done < n) {                      /* write() may write less than asked */
            ssize_t w = write(STDOUT_FILENO, buf + done, n - done);
            if (w == -1) {
                if (errno == EINTR) continue;    /* interrupted by a signal: try again */
                perror("write"); close(fd); return 1;
            }
            done += w;
        }
    }
    if (n == -1) { perror("read"); close(fd); return 1; }
    close(fd);
    return 0;
}
```

It uses a 4096-byte buffer, handles partial writes with the inner loop, retries after `EINTR`, and reports the file name in the error message with `perror(fname)`. I checked that it prints all 10 000 bytes of a test file.

Write it from memory

1. Headers: `stdio.h`, `unistd.h`, `fcntl.h`. Declare `char fname[100], buffer[100]; int fd; ssize_t n;`
2. Prompt and `scanf("%99s", fname);`
3. `fd = open(fname, O_RDONLY);` and `if (fd == -1) { perror("Error"); return 1; }`
4. `while ((n = read(fd, buffer, sizeof(buffer))) > 0) write(STDOUT_FILENO, buffer, n);`
5. `close(fd);`

Remember: **open, read-write loop, close**.

Viva questions

Q. What is a file descriptor?
A. A small non-negative integer that identifies an open file in the process's descriptor table.

Q. What are descriptors 0, 1 and 2?
A. Standard input (keyboard), standard output (screen) and standard error (screen).

Q. What does read return at the end of a file?
A. Zero. It returns -1 on error.

Q. Why is n of type ssize_t and not size_t?
A. `ssize_t` is signed, so it can store the error value -1.

Q. Why write n bytes and not sizeof(buffer)?
A. The last chunk is usually shorter than the buffer. Writing the whole buffer would print garbage.

Q. Difference between read() and fread()?
A. `read` is a system call with no buffering in your program. `fread` is a library function that keeps its own buffer and calls `read` underneath.

Q. Why is there a loop?
A. `read` returns at most the buffer size, so a bigger file needs several calls.

Q. What does close do?
A. It frees the descriptor so the number can be reused and the kernel can release its resources.

Q. What happens if the file does not exist?
A. `open` returns -1 and sets `errno` to `ENOENT`; `perror` prints "No such file or directory".

EXPERIMENT 3

Child processes using fork()

Aim from the lab manual

Write a C program to create five child processes using the system call `fork()` and display their ids.

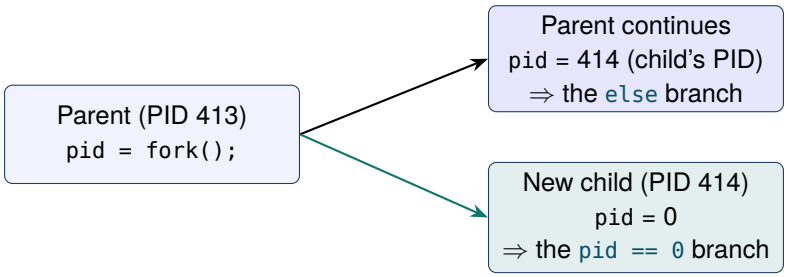
3.1 Big picture

The parent process runs a loop five times. In each round it calls `fork()` once, which creates one child. The child prints its own PID and its parent's PID and then **exits immediately**. The parent prints the child's PID and goes on to the next round. After the loop the parent waits for all five children and prints a final message.

3.2 Concepts you need first

3.2.1 fork() returns twice

`fork()` makes an almost exact copy of the calling process. From the instruction after `fork()`, *two* processes run the same code, and the only easy way to tell them apart is the return value:



Return value	Who gets it and what it means
greater than 0	the *parent*. The number is the PID of the new child
0	the *child*
−1	the parent only. The call failed, *no child was created*, and `errno` says why

3.2.2 What the child inherits and what is different

- The child gets its **own copy** of the variables. After `fork` a change made by one process is invisible to the other. (Linux uses *copy-on-write*: pages are really copied only when one process writes to them, so `fork` is cheap.)
- The child has a **new PID**, and its parent PID (PPID) is the parent's PID.
- Open files are shared copies of the same descriptors. Timers set by `alarm()` are *not* inherited.
- `getpid()` returns the caller's own PID. `getppid()` returns its parent's PID.

3.2.3 wait(), zombies and orphans

- `wait(NULL)` suspends the parent until *one* child has ended. It returns that child's PID, or -1 if there is no child left to wait for. `NULL` means "I do not need the exit status".
- A child that has ended but has not yet been waited for is a **zombie**: the kernel keeps a small record (PID, exit status) so the parent can still collect it. Calling `wait` removes the record.
- If the parent ends first, its children become **orphans** and are adopted by the init process (or the nearest "subreaper"), which waits for them.

3.2.4 Buffered output and fork

If `stdout` is a file or pipe, `printf` output sits in a buffer inside the process. `fork` copies that buffer too, so the *child would print the parent's unflushed text again*. Flushing before each `fork` avoids it.

3.3 The program at a glance

Lines	Job
1–4	headers (`sys/wait.h` is for `wait`)
8–9	loop counter `i` and the variable `pid` for `fork`'s result
10–13	loop start; flush; the `fork()` call itself
14–17	fork failed
18–21	child branch: print ids, then `exit(0)`
22–25	parent branch: print the child's PID
27–30	parent waits for all five children
31–33	final message, then end

3.4 Line-by-line explanation

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main()
7  {
8      int i;
9      pid_t pid;
```

Lines 1–3 `stdio.h` (`printf`, `fflush`), `stdlib.h` (`exit`), `unistd.h` (`fork`, `getpid`, `getppid`).

Line 4 `sys/wait.h` provides `wait()`.

Lines 6–7 `int main()` and the opening brace.

Line 8 `int i;` counts the rounds.

Line 9 `pid_t pid;` holds the value returned by `fork`. `pid_t` is the type for process ids (an integer).

```
10     for(i=1; i<=5; i++){
```

```
11          /* flush so the child does not copy unflushed output */
12          fflush(stdout);
13          pid = fork();
```

Line 10 `for(i=1; i<=5; i++){` runs the body for `i` = 1, 2, 3, 4, 5. Starting at 1 makes the messages read "Child 1 ...Child 5".

Lines 11–12 `fflush(stdout);` empties the output buffer *before* the process splits, so a child cannot inherit and repeat old text.

Line 13 `pid = fork();` is the moment the process splits in two. Both processes continue at the next line, each with its own `pid`.

```
14          if(pid < 0)
15          {
16              printf("Error ");
17              return 1;
18          } else if(pid == 0) {
19              printf("Child %d with id %d and Parent Process id %d\n",
20                     i, getpid(), getppid());
21              exit(0);
22          } else {
23              printf("Parent Created Child %d with pid %d\n",
24                     i, pid);
25          }
```

Line 14 `if(pid < 0)` — `fork` failed (for example too many processes).

Lines 16–17 Prints `Error` and returns 1. (Better: `perror("fork");`, which also prints the reason.)

Line 18 `else if(pid == 0) {` — this block runs **only in the child**, because only the child sees 0.

Lines 19–20 One `printf` continued over two lines. Three `%d` are filled by `i` (child number), `getpid()` (the child's own PID) and `getppid()` (the parent's PID).

Line 21 `exit(0);` ends the child with status 0. **This line is vital**: it stops the child from going back to the top of the loop and forking its own children (see the dry run).

Line 22 `} else {` — runs **only in the parent**, where `pid` is the child's PID.

Lines 23–24 Prints "Parent Created Child i with pid ...". Here `pid` is the number `fork` returned.

Lines 25–26 Close the `else` block and the loop body. The parent now goes to the next round.

```
27      for(i=1; i<=5; i++)
28      {
29          wait(NULL);
30      }
31      printf("Parent pid %d after finishing all Child execution\n",
32             getpid());
33      return 0;
34  }
```

Lines 27–30 A second loop that calls `wait(NULL)` five times. Each call blocks until one child ends and then removes its zombie record. Five children need five waits.

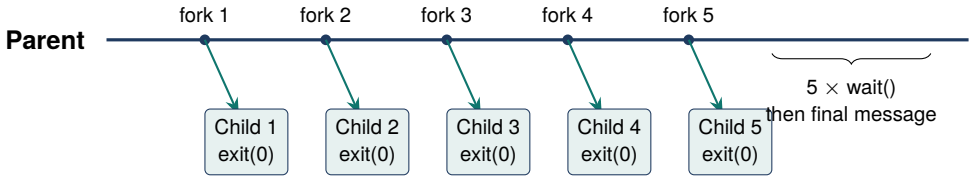
Lines 31–32 After all children are gone the parent prints its own PID with `getpid()`.

Lines 33–34 `return 0;` and the closing brace.

3.5 Dry run

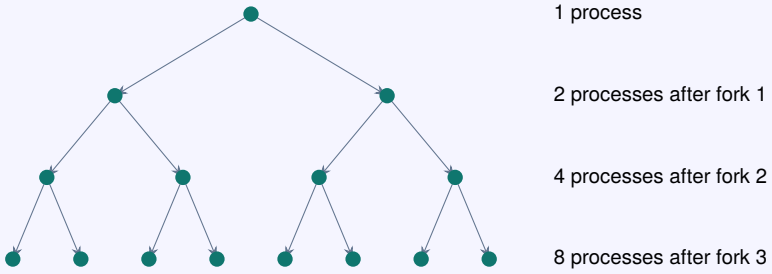
Suppose the parent's PID is 413 and the kernel hands out PIDs in order.

i	fork() gives parent	fork() gives child	What each prints
1	414	0	parent: "Parent Created Child 1 ...414"; child 414: "Child 1 ...id 414 and Parent Process id 413"
2	415	0	the same pattern with 415
3	416	0	...
4	417	0	...
5	418	0	...



Why the child must call exit(0)

Without line 21 the child would continue with the loop and fork *its own* children, and so would every grandchild. The number of processes doubles in every round, so five rounds give $2^5 = \mathbf{32}$ processes in total (the original plus 31 children), not 6. I tested this: a loop of five `fork()` calls with no `exit` printed 32 lines.



Why the fflush before fork — tested

I removed line 12 and sent the output to a file (`./a.out > out.txt`). The file had **21 lines instead of 11**: each child inherited the parent's unflushed "Parent Created..." lines and printed them again when it exited. On a terminal the newline flushes the buffer, so you would not see the problem there.

3.6 The output, explained

Your recorded output:

```
Parent Created Child 1 with pid 414
Child 1 with id 414 and Parent Process id 413
Parent Created Child 2 with pid 415
Child 2 with id 415 and Parent Process id 413
```

```
  ... (same pattern up to Child 5, pid 418)
Parent pid 413 after finishing all Child execution
```

Every child line shows the *same* parent id (413): all five children have the same parent. The id printed by the parent (`with pid 414`) equals the id printed by the child (`id 414`): that is the value `fork` returned.

The order is not guaranteed. After `fork`, the scheduler decides which process runs first. Here are two real runs on my machine (output sent to a pipe), copied exactly:

Run 1 (two of the `Child` lines come before their `Parent Created` lines):

```
Child 1 with id 189 and Parent Process id 187
Parent Created Child 1 with pid 189
Parent Created Child 2 with pid 190
Parent Created Child 3 with pid 191
Parent Created Child 4 with pid 192
Child 2 with id 190 and Parent Process id 187
Child 3 with id 191 and Parent Process id 187
Child 4 with id 192 and Parent Process id 187
Child 5 with id 193 and Parent Process id 187
Parent Created Child 5 with pid 193
Parent pid 187 after finishing all Child execution
```

Run 2 (a different order, same program):

```
Child 1 with id 203 and Parent Process id 201
Parent Created Child 1 with pid 203
Parent Created Child 2 with pid 204
Child 2 with id 204 and Parent Process id 201
Parent Created Child 3 with pid 205
Child 3 with id 205 and Parent Process id 201
Parent Created Child 4 with pid 206
Child 4 with id 206 and Parent Process id 201
Child 5 with id 207 and Parent Process id 201
Parent Created Child 5 with pid 207
Parent pid 201 after finishing all Child execution
```

Only two things are certain: each child's line says its parent is the original process, and the final line comes last (because of `wait`). If the examiner's output differs from yours, that is normal.

3.7 Gotchas and the best method

Common mistakes

- **Forgetting `exit(0)` in the child** gives 31 children, as shown above.
- **Calling `wait` too few times** leaves zombies. Use a loop that ends when `wait` returns -1.
- **Using `printf("Error ")` with no new line and no reason.** Prefer `perror("fork");`.
- **Using `_exit` carelessly.** `_exit` does not flush `stdio` buffers; `exit` does. The man page example for `fork` calls `fflush(stdout)` before `_exit`.
- **Assuming variables are shared.** A change made by the child is not visible in the parent, because each has its own memory.

Best method: collect each child's exit status (tested)

A child can return information to its parent through its exit status. The parent reads it with `wait(&status)` and the macros `WIFEXITED` and `WEXITSTATUS`. This version also uses `perror` and

stops the wait loop exactly when no child is left.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int i, status;
    for (i = 1; i <= 5; i++) {
        fflush(stdout);
        pid_t pid = fork();
        if (pid < 0) { perror("fork"); return 1; }
        if (pid == 0) {                     /* child */
            printf("Child %d: pid=%d parent=%d\n", i, getpid(), getppid());
            exit(i);                        /* exit status = child number */
        }
        printf("Parent created child %d with pid %d\n", i, pid);
    }
    while (1) {                             /* reap children until none are left */
        pid_t done = wait(&status);
        if (done == -1) break;              /* ECHILD: no more children */
        if (WIFEXITED(status))
            printf("Parent: child %d ended with status %d\n", done, WEXITSTATUS(status));
    }
    printf("Parent %d: all children finished\n", getpid());
    return 0;
}
```

```
Parent: child 210 ended with status 1     (the last six lines of one real run; the
Parent: child 211 ended with status 2      "Parent created" and "Child" lines above
Parent: child 212 ended with status 3      them can appear in any order)
Parent: child 213 ended with status 4
Parent: child 214 ended with status 5
Parent 208: all children finished
```

Write it from memory

```c
for (i = 1; i <= 5; i++) {
    fflush(stdout);
    pid = fork();
    if (pid < 0)       { perror("fork"); return 1; }
    else if (pid == 0) { printf(... getpid(), getppid()); exit(0); }  /* child */
    else               { printf(... pid); }                           /* parent */
}
for (i = 0; i < 5; i++) wait(NULL);
```

Memory aid: **the child exits, the parent loops**; `fork` returns 0 to the child.

Viva questions

Q. What does fork() return?
A. In the parent: the child's PID. In the child: 0. On failure: -1 and no child is created.

Q. How many processes exist if the child does not exit?
A. $2^5 = 32$ including the original, because every process forks in every remaining round.

Q. Do parent and child share variables?
A. No. Each has its own copy of memory (copy-on-write). Open files are shared descriptors.

Q. What is a zombie process?
A. A child that has ended but whose parent has not yet called `wait`; only its small kernel record remains.

Q. What is an orphan process?
A. A child whose parent ended first. It is adopted by init (or a subreaper).

Q. What does wait(NULL) do?
A. It blocks until one child ends, returns its PID, and ignores the exit status.

Q. Difference between wait and waitpid?
A. `wait` waits for *any* child. `waitpid` can wait for a specific child and has options such as `WNOHANG` (do not block).

Q. What does getppid() return in the child?
A. The PID of the parent process (413 in the example).

Q. Why flush stdout before fork?
A. A pending buffer is copied into the child and would be printed twice.

Q. Difference between exit() and _exit()?
A. `exit` flushes stdio buffers and runs exit handlers; `_exit` leaves immediately without flushing.

Q. Why can fork() fail?
A. The system or the user has reached the limit on processes, or memory is short. The call returns -1.

EXPERIMENT 4

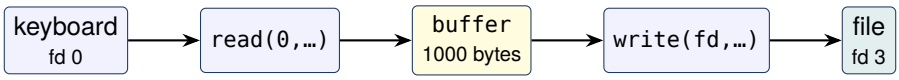
File write using system calls

Aim from the lab manual

Write a C program using unique system calls to open a file for writing (create it if it does not exist). The program should read text from the keyboard and write it into the file until Ctrl+D.

4.1 Big picture

The program asks for a filename, opens that file for writing (creating it, or emptying it if it already exists), and then copies everything you type into it, one line after another. When you press **Ctrl+D** the copying stops, the file is closed, and a confirmation message is printed. This is Experiment 2 turned around: before, the file was the *source*; now it is the *destination*.



4.2 Concepts you need first

4.2.1 open() with flags and a mode

`open(path, flags, mode)` takes the flags as **bits that are combined with the OR operator** `|`. Each flag is a number with one bit set, so they can be added together without interfering. The *mode* says which permissions a *new* file gets.

Flag	Meaning	Value on my Linux
`O_WRONLY`	open for writing only	1
`O_CREAT`	create the file if it does not exist	64 (octal 100)
`O_TRUNC`	if it exists, cut it to length 0 (old content is lost)	512 (octal 1000)
`O_CREAT|O_WRONLY|O_TRUNC`	all three together	577
`O_APPEND`	(not used here) every write goes to the end	1024

The numbers differ between systems, so always use the names. The manual page also tells us that this exact combination is what the older call `creat(path, mode)` does.

4.2.2 The mode 0644 and permission bits

Octal digit	Binary	r w x	Who	
6	110	r w –	owner (first digit)	read and write
4	100	r – –	group (second digit)	read only
4	100	r – –	others (third digit)	read only

So `0644` means `-rw-r--r--` in `ls -l`. The leading `0` tells C that the number is *octal*. Read = 4, write = 2, execute = 1. Two more facts: the mode is used **only when the file is created**, and the final permission is `mode & ~umask`. With the usual umask 022 the result is still 0644 (I checked: the file shows `-rw-r--r--`). If you use `O_CREAT` you *must* pass the mode; without it, random bytes from the stack become the permissions.

4.2.3 Reading from the keyboard and Ctrl+D

- Descriptor **0** is the keyboard, so `read(0, buffer, 1000)` reads what you type.
- A terminal works in *line mode*: your typing is collected by the terminal driver, and `read` gets it only after you press Enter. One `read` normally returns one line, including the final `\n`.
- **Ctrl+D is not a character and not a signal.** It is the terminal's "end-of-file" key. According to the terminal documentation it hands over whatever is waiting in the line buffer without waiting for Enter. If nothing is waiting (you pressed it at the start of an empty line), `read` returns **0**, which means end of file.
- Ctrl+D is never stored in the file.

4.3 The program at a glance

Lines	Job
1–4	headers
8–9	file descriptor, 1000-byte buffer, filename array
10–11	ask for and read the filename
12–17	open (or create) the file for writing; stop if it fails
18	ask the user to type
19–33	loop: read a line from the keyboard, write it to the file; stop at end of file
34–37	close the file, confirm, finish

4.4 Line-by-line explanation

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd;
    char buffer[1000], fname[100];
```

```
10      printf("Enter the filename to be written \n");
11      scanf("%99s", fname);
```

Line 1 `stdio.h`: `printf` and `scanf`.

Line 2 `stdlib.h`: not used, harmless.

Line 3 `fcntl.h`: `open` and the flags `O_CREAT`, `O_WRONLY`, `O_TRUNC`.

Line 4 `unistd.h`: `read`, `write`, `close`.

Line 8 `int fd;` will hold the descriptor of the output file.

Line 9 `char buffer[1000], fname[100];` the buffer receives up to 1000 bytes per `read`; `fname` holds the file name.

Line 10 Prompt. The string ends with a space and `\n`, so the cursor moves to the next line.

Line 11 `scanf("%99s", fname);` reads the file name (one word, at most 99 characters).

```
12      fd = open(fname, O_CREAT | O_WRONLY | O_TRUNC, 0644);
13      if (fd == -1)
14      {
15          printf("Open error");
16          return 0;
17      }
18      printf("Enter the contents to be written \n");
```

Line 12 `fd = open(fname, O_CREAT | O_WRONLY | O_TRUNC, 0644);`

- `fname` — the file to open.
- `O_CREAT` — create it if missing.
- `O_WRONLY` — we only need to write.
- `O_TRUNC` — if it exists, erase its old contents first.
- `0644` — permissions for a newly created file (owner read+write, others read).
- Result: the new descriptor, or -1 on failure.

Lines 13–14 `if (fd == -1)` and the opening brace: the open failed (no permission, folder missing, ...).

Line 15 `printf("Open error");` — a message (without new line, and without the reason; `perror` would be better).

Line 16 `return 0;` ends the program. It returns 0 (success) even though an error happened; 1 would be more correct.

Line 17 Closing brace.

Line 18 Tells the user to start typing.

```
19      while(1)
20      {
21          int n = read(0, buffer, sizeof(buffer));
22          if(n > 0) {
23              write(fd, buffer, n);
24          }
25          else if(n == 0) {
26              break;
27          }
28          else {
29              printf("Read error");
```

```
30              close(fd);
31              return 0;
32          }
33      }
```

Line 19 `while(1)` is an infinite loop. It is left only by `break` (line 26) or `return` (line 31).

Line 21 `int n = read(0, buffer, sizeof(buffer));` waits for the keyboard (descriptor 0) and reads up to 1000 bytes. `n` is how many bytes arrived. Here it can be > 0 (text), 0 (Ctrl+D at the start of a line) or -1 (error).

Lines 22–24 `if(n > 0) { write(fd, buffer, n); }` writes exactly the `n` bytes that were just read into the *file* (descriptor `fd`). The return value of `write` is ignored.

Lines 25–27 `else if(n == 0) { break; }` — end of file (Ctrl+D). Leave the loop.

Lines 28–32 `else` — `n` is negative, so `read` failed: print a message, close the file and stop.

Line 33 Closing brace of the `while` loop.

```
34      close(fd);
35      printf("Contents written to %s\n", fname);
36      return 0;
37  }
```

Line 34 `close(fd);` releases the file. The data has already been handed to the kernel by `write`.

Line 35 Prints "Contents written to" and the file name (`%s` is replaced by `fname`).

Lines 36–37 `return 0;` and the closing brace.

4.5 Dry run

Normal case: you type `hello world`, press Enter, then press Ctrl+D on the empty line.

#	What you do	read() returns	What the program does
1	type `hello world`, press Enter	12 (11 letters + `\n`)	`write(fd, buffer, 12)`
2	press Ctrl+D on the empty line	0	`break`, leave the loop
3	—	—	`close`, print "Contents written to …"

The file now contains `hello world` plus a newline (12 bytes). This is the output in your manual.

Ctrl+D in the middle of a line (tested): type `abc` but do *not* press Enter; press Ctrl+D; then type `def`, Enter, and Ctrl+D again.

#	What you do	read() returns	Result
1	type `abc`, press Ctrl+D	3	the three letters are delivered and written; the program *keeps waiting*
2	type `def`, Enter	4	`def` and a newline are written
3	Ctrl+D again	0	end of file

The file now contains `abcdef` and a newline. **So if you press Ctrl+D after typing text on a line, press it twice.**

4.6 The output, explained

```
gcc main.c
./a.out
Enter the filename to be written
contents.txt
Enter the contents to be written
hello world
Contents written to contents.txt      <- appears after you press Ctrl+D (nothing is shown for ^D)
cat contents.txt
hello world
```

The second run of the program with the same file name would *replace* the old text, because of `O_TRUNC`. The last two lines use the shell command `cat` to prove that the file was really written.

4.7 Gotchas and the best method

Mixing scanf and read — tested

The program reads the file name with `scanf` (a library function with its own buffer) and the text with `read` (a raw system call). From the **keyboard** this works, because the terminal gives the program one line at a time. If the input comes from a **pipe or file**, `scanf` grabs a whole block into its private buffer and the later `read` sees nothing. I tested `printf 'pipe.txt\nhello\n' | ./a.out`: the program ran, but the file it created was **empty (0 bytes)**. For the lab, type at the keyboard and you will not meet this problem.

Other points

- `O_TRUNC` silently destroys an existing file. Use `O_APPEND` instead of `O_TRUNC` to add to the end.
- The return value of `write` (line 23) is not checked; the open-error and read-error branches return 0 (success).
- `int n` works, but `ssize_t n` is the correct type for `read`'s result.
- Forgetting the third argument of `open` while using `O_CREAT` gives the new file random permissions.

Best method: checked results and proper error messages (tested)

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>
4
```

```c
int main(void)
{
    char buf[1024], fname[256];
    printf("Enter the filename to be written: ");
    if (scanf("%255s", fname) != 1) return 1;
    int fd = open(fname, O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd == -1) { perror(fname); return 1; }
    printf("Type your text (Ctrl+D to finish):\n");
    ssize_t n;
    while ((n = read(STDIN_FILENO, buf, sizeof buf)) > 0) {
        if (write(fd, buf, n) != n) { perror("write"); close(fd); return 1; }
    }
    if (n == -1) { perror("read"); close(fd); return 1; }
    close(fd);
    printf("Contents written to %s\n", fname);
    return 0;
}
```

`write` is checked, every error path returns 1 and prints the system's reason, `STDIN_FILENO` replaces the magic number 0, and `perror(fname)` names the file that caused the problem. I tested it by typing two lines and pressing Ctrl+D: the file held exactly those two lines with permissions `-rw-r--r--`.

Write it from memory

1. Headers: `stdio.h`, `fcntl.h`, `unistd.h`.
2. `scanf("%99s", fname);`
3. `fd = open(fname, O_CREAT | O_WRONLY | O_TRUNC, 0644);` and check for -1.
4. `while ((n = read(0, buffer, sizeof(buffer))) > 0) write(fd, buffer, n);`
5. `close(fd);` and print the confirmation.

Remember the **three flags** (create, write-only, truncate) and the mode **0644**. Read returns 0 on Ctrl+D.

Viva questions

Q. Why three flags in open()?
A. `O_WRONLY` to write, `O_CREAT` to create the file if it is missing, `O_TRUNC` to empty it if it exists.

Q. What does 0644 mean?
A. Octal permission bits: owner read+write (6), group read (4), others read (4). It applies only when the file is created, and is reduced by the umask.

Q. Why are the flags combined with the | operator?
A. Each flag is a separate bit, so OR sets several bits in one integer.

Q. How does the program know that the user pressed Ctrl+D?
A. `read` returns 0 (end of file) when Ctrl+D is pressed on an empty line.

Q. Is Ctrl+D a signal?
A. No. Ctrl+C sends a signal (SIGINT). Ctrl+D is the terminal's end-of-file marker.

Q. What is descriptor 0?
A. Standard input, normally the keyboard.

Q. What happens if the file already exists?
A. Its content is erased because of `O_TRUNC`; with `O_APPEND` instead, new text is added at the end.

Q. What does write return?
A. The number of bytes written, or -1 on error.

Q. What is the difference between creat() and open()?
A. `creat(path, mode)` is the same as `open(path, O_WRONLY|O_CREAT|O_TRUNC, mode)`.

EXPERIMENT 5

Signal handling

Aim from the lab manual

Write a C program that registers a signal handler for `SIGINT`, `SIGTERM`, `SIGHUP`, `SIGALRM` and `SIGUSR1`. The handler should receive the signal and perform some operation.

5.1 Big picture

The program installs one function, `signalhandler`, for five different signals. It sets an alarm for 20 seconds, prints its own PID, and then loops forever printing "Waiting for signal" and sleeping 5 seconds. You open a *second terminal* and send signals to the program with the `kill` command. Each signal makes the handler print a message; the `SIGTERM` handler also ends the program.

5.2 Concepts you need first

5.2.1 What is a signal?

A **signal** is a small message that the kernel delivers to a process *asynchronously*, meaning at any moment, between two instructions of the program. Signals come from the keyboard (Ctrl+C), from other processes (`kill`), or from the kernel itself (a timer expired, bad memory access). For each signal a process has three choices:

1. do the **default action** (for all five signals of this experiment the default is *terminate the process*);
2. **ignore** it (`SIG_IGN`);
3. run **your own handler function**. This experiment does the third.

Signal	No.	Default action	Typical source
`SIGHUP`	1	terminate	the terminal was closed ("hang up"), or `kill -HUP`
`SIGINT`	2	terminate	**Ctrl+C**
`SIGQUIT`	3	terminate + core dump	Ctrl+\
`SIGKILL`	9	terminate; **cannot be caught**	`kill -9`
`SIGUSR1`	10	terminate	free for your own use (`kill -USR1`)
`SIGSEGV`	11	terminate + core dump	illegal memory access
`SIGALRM`	14	terminate	the timer set by `alarm()` expired
`SIGTERM`	15	terminate	`kill PID` (the default signal of `kill`), a polite request to end
`SIGCHLD`	17	ignore	a child process ended
`SIGSTOP`	19	stop; **cannot be caught**	`kill -STOP`

The numbers are those of Linux on x86-64 (I checked them on my machine) and can differ on other processors, so always use the names. `SIGKILL` and `SIGSTOP` can never be caught, blocked or ignored. That is why `kill -9` always works.

5.2.2 The functions used

Function	What it does
`signal(sig, handler)`	registers `handler` for `sig`. Returns the old handler or `SIG_ERR`.
`void handler(int sig)`	the shape of every handler: it gets the signal number as its only argument, so one function can serve many signals.
`alarm(n)`	asks the kernel to send `SIGALRM` after `n` seconds. It fires **once**. A new call replaces the old alarm, and `alarm(0)` cancels it. It returns the seconds that were left on the previous alarm.
`sleep(n)`	suspends the process for `n` seconds, but **stops early if a signal handler runs**, and then returns the seconds that were left.
`getpid()`	returns the process's own PID (needed so you can aim `kill`).

In the second terminal you type `kill -SIGINT 430` (or `kill -INT 430` or `kill -2 430`). The command is called "kill" for historical reasons: it just *sends a signal*, and what happens next depends on the receiving program. `kill 430` sends `SIGTERM`.

Two facts examiners like

1. Ordinary ("standard") signals are *not queued*: if the same signal arrives twice before the handler runs, it is delivered only once. **2.** The man page for `signal` says: "avoid its use, use `sigaction` instead", because the behaviour of `signal()` differs between Unix systems. On Linux with glibc, `signal()` behaves like BSD: the handler stays installed after a signal, the same signal is blocked while the handler runs, and certain slow system calls are restarted.

5.3 The program at a glance

Lines	Job
1–4	headers
6–28	the handler function: print a message depending on which signal arrived (and exit on `SIGTERM`)
32–36	register the handler for the five signals
37	set a one-shot alarm for 20 seconds
38	print the PID so you can use `kill`
39–43	loop forever: print, sleep 5 seconds

5.4 Line-by-line explanation

5.4.1 The handler

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <signal.h>
5
6  void signalhandler(int sig)
7  {
8      if(sig == SIGINT)
9      {
10         printf("Caught an Interrupt Signal\n");
11     }
```

Line 1 `stdio.h`: `printf`.

Line 2 `stdlib.h`: `exit`.

Line 3 `unistd.h`: `getpid`, `alarm`, `sleep`.

Line 4 `signal.h`: `signal` and the constants `SIGINT`, `SIGTERM`, …

Line 6 `void signalhandler(int sig)` declares a function that returns nothing and receives the signal number in `sig`. When a signal arrives, the kernel interrupts the program, calls this function, and later continues the program where it was interrupted. One function serves all five signals; it tells them apart by `sig`.

Line 7 Opening brace of the function.

Line 8 `if(sig == SIGINT)` — is this the interrupt signal (Ctrl+C)?

Lines 9–11 Prints "Caught an Interrupt Signal". There is *no* `exit`, so the program continues. This is why Ctrl+C no longer stops the program once the handler is installed.

```
12     if(sig == SIGTERM)
13     {
14         printf("Caught a Terminate Signal\n");
15         exit(0);
16     }
17     if(sig == SIGHUP){
18         printf("Caught a Hangup Signal\n");
19     }
20     if(sig == SIGUSR1)
21     {
22         printf("Caught a User Signal\n");
23     }
24     if(sig == SIGALRM)
25     {
26         printf("Caught an Alarm Signal\n");
27     }
28 }
```

Lines 12–16 `if(sig == SIGTERM)`: prints "Caught a Terminate Signal" and then calls `exit(0)`. Because a handler replaces the default action, the default "terminate" no longer happens by itself; the program has to end itself here. This is the only way (besides `kill -9`) to stop the program.

Lines 17–19 `SIGHUP`: prints "Caught a Hangup Signal".

Lines 20–23 `SIGUSR1`: prints "Caught a User Signal".

Lines 24–27 `SIGALRM`: prints "Caught an Alarm Signal".

Line 28 Closing brace of the function. Control returns to the interrupted code.

Style note Five separate `if` statements work because `sig` has only one value at a time. A `switch (sig)` would be tidier.

5.4.2 The main function

```
29
30 int main()
31 {
32     signal(SIGINT, signalhandler);
33     signal(SIGTERM, signalhandler);
34     signal(SIGHUP, signalhandler);
35     signal(SIGALRM, signalhandler);
36     signal(SIGUSR1, signalhandler);
37     alarm(20);
38     printf("Process Id of program running = %d\n", getpid());
```

Lines 30–31 `int main()` and its opening brace.

Lines 32–36 Five registrations. `signal(SIGINT, signalhandler);` means "when `SIGINT` arrives, call `signalhandler`". Note that the function name is written *without brackets*: you hand over the function's address; you are not calling it.

Line 37 `alarm(20);` asks the kernel to send `SIGALRM` in 20 seconds. It fires once (I ran the program for 22 seconds without sending anything: "Caught an Alarm Signal" appeared once at about 20 seconds).

Line 38 Prints the PID. You need this number for `kill`.

```
39     while(1)
40     {
41         printf("Waiting for signal\n");
42         sleep(5);
43     }
44     return 0;
45 }
```

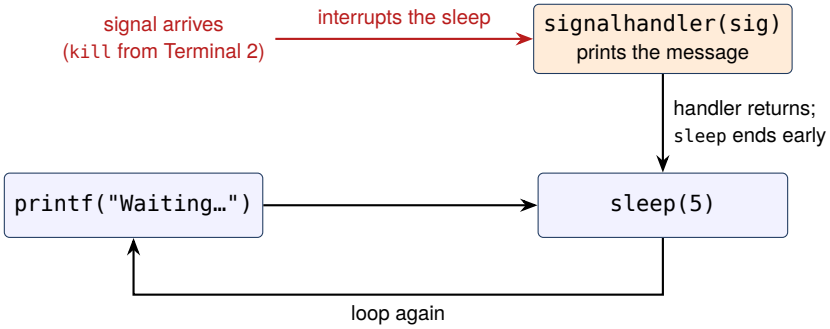
Lines 39–40 `while(1) {` is an endless loop.

Line 41 Prints "Waiting for signal".

Line 42 `sleep(5);` waits 5 seconds. If a signal handler runs meanwhile, `sleep` returns at once, and the loop prints "Waiting for signal" again immediately. That is exactly the pattern in your output.

Lines 43–45 Closing brace of the loop, `return 0;` (never reached, because the loop never ends) and the closing brace of `main`.

5.5 Dry run



Here is your two-terminal experiment, step by step (the PID was 430):

Terminal 2 command	What happens in Terminal 1
`kill -SIGINT 430`	`signalhandler(2)` runs: "Caught an Interrupt Signal". `sleep` ends early, the loop prints "Waiting for signal".
`kill -SIGHUP 430`	"Caught a Hangup Signal", then "Waiting for signal".
`kill -SIGUSR1 430`	"Caught a User Signal", then "Waiting for signal".
`kill -SIGALRM 430`	"Caught an Alarm Signal": you sent `SIGALRM` by hand. (The real `alarm(20)` would fire by itself 20 seconds after start.)
`kill -SIGTERM 430`	"Caught a Terminate Signal", then `exit(0)`: the program ends and the shell prompt returns.

5.6 The output, explained

```
Terminal 1                          what produced the line
Process Id of program running = 430 line 38
Waiting for signal                  line 41 (first pass)
Caught an Interrupt Signal          handler, sig == SIGINT
Waiting for signal                  sleep() returned early, loop runs again
Caught a Hangup Signal              handler, SIGHUP
...
Caught a Terminate Signal           handler, SIGTERM -> exit(0) ends the program
```

I ran this exact sequence of five `kill` commands against your program and got the same lines in the same order, and the program ended after `SIGTERM`. I also ran it for 22 seconds without sending anything:

```
Process Id of program running = 95
Waiting for signal          (printed at 0 s, 5 s, 10 s, 15 s)
Waiting for signal
Waiting for signal
Waiting for signal
Caught an Alarm Signal      <- the alarm(20) set on line 37 fired by itself
Waiting for signal
Caught a Terminate Signal   <- sent by the timeout command at 22 s
```

5.7 Gotchas and the best method

1. printf() and exit() are not safe inside a handler

A handler can interrupt your program *anywhere*, even in the middle of `printf`. If the handler calls `printf` again while the first call is still using the `stdio` buffer, the output can be corrupted or the program can hang. The Linux manual page `signal-safety(7)` lists the functions that are *async-signal-safe*: `write`, `_exit`, `getpid`, `kill`, `alarm`, `signal`, `sigaction`, `wait`, `waitpid`, `sem_post` and more. `printf` and `exit` are *not* on that list. For a lab program it works, but it is not the best method.

2. Other points

- **Ctrl+C does not stop this program**, because the `SIGINT` handler only prints. Stop it with `kill -TERM PID`, or `kill -9 PID`, or Ctrl+\ (`SIGQUIT`).
- **alarm() fires only once.** For a repeating timer call `alarm()` again inside the handler.
- **sleep() is cut short** by a signal and returns the seconds left. To really wait 5 seconds use `left = 5; while ((left = sleep(left)) > 0);`.
- **Standard signals are not queued**: ten `kill -USR1` sent quickly may run the handler fewer than ten times.
- The `signal()` function is not portable across Unix systems (see the key-idea box). Use `sigaction()` in serious code.

Best method: sigaction + write() + a flag (tested)

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>

static volatile sig_atomic_t stop = 0;       /* handler sets it, main() reads it */

/* write() is async-signal-safe, printf() is not */
#define SAY(msg) do { ssize_t r_ = write(STDOUT_FILENO, msg, sizeof(msg) - 1); (void)r_; } while (0)

static void handler(int sig)
{
    switch (sig) {
    case SIGINT:  SAY("Caught an Interrupt Signal\n"); break;
    case SIGHUP:  SAY("Caught a Hangup Signal\n");     break;
    case SIGUSR1: SAY("Caught a User Signal\n");       break;
    case SIGALRM: SAY("Caught an Alarm Signal\n");     break;
    case SIGTERM: SAY("Caught a Terminate Signal\n");  stop = 1; break;
    }
}

int main(void)
{
    int sigs[] = { SIGINT, SIGTERM, SIGHUP, SIGALRM, SIGUSR1 };
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);                /* block no extra signals in handler */
    for (int i = 0; i < 5; i++)
        sigaction(sigs[i], &sa, NULL);

```

```c
    alarm(20);
    printf("Process Id of program running = %d\n", getpid());
    while (!stop) {
        printf("Waiting for signal\n");
        sleep(5);
    }
    printf("Exiting cleanly\n");
    return 0;
}
```

- `sigaction` gives explicit, portable control. `sa.sa_handler` is the function, `sigemptyset(&sa.sa_mask)` says no extra signals are blocked while it runs. The loop over an array registers all five signals with the same code.
- The handler uses only `write()`, which is async-signal-safe. The macro `SAY` computes the length at compile time with `sizeof`, so it does not even call `strlen`.
- `SIGTERM` does *not* call `exit` inside the handler. It only sets the flag `stop`. The main loop sees the flag and ends cleanly, printing "Exiting cleanly". The flag is declared `volatile sig_atomic_t`, the only kind of variable a handler may safely change.
- A tiny race remains: if `SIGTERM` arrives just between `while (!stop)` and `sleep(5)`, the program notices up to 5 seconds late. Production code closes this gap with `sigprocmask` and `sigsuspend`.

I tested this version with the same five `kill` commands: the output lines are identical to your manual's, followed by "Exiting cleanly", and the process ends.

Write it from memory

1. Headers: `stdio.h`, `stdlib.h`, `unistd.h`, `signal.h`.
2. `void signalhandler(int sig) { if (sig == SIGINT) printf(...); ... if (sig == SIGTERM) { printf(...); exit(0); } }`
3. In `main`: five lines `signal(SIGxxx, signalhandler);` then `alarm(20);`
4. Print `getpid()`, then `while(1) { printf("Waiting for signal\n"); sleep(5); }`

Remember: **handler takes an int; signal(SIG, handler) five times; alarm; endless loop with sleep**. Test from a second terminal with `kill -SIGINT PID`.

Viva questions

Q. What is a signal?
A. A software notification sent by the kernel to a process, asynchronously. It can come from the keyboard, another process or the kernel.

Q. What are the three things a process can do with a signal?
A. Take the default action, ignore it, or run a handler function.

Q. Which signals cannot be caught?
A. `SIGKILL` and `SIGSTOP`. They also cannot be blocked or ignored.

Q. Why does the SIGTERM handler call exit(0)?
A. Installing a handler replaces the default action (terminate). Without `exit` the program would ignore `SIGTERM` and keep running.

Q. What does alarm(20) do?
A. It requests one `SIGALRM` after 20 real seconds. A later `alarm` call replaces it; `alarm(0)` cancels it.

Q. Why does the loop print "Waiting for signal" right after each handler message?
A. A caught signal interrupts `sleep(5)`, which returns early; the loop then goes round again at once.

Q. Difference between signal() and sigaction()?
A. `signal()` is the old interface and behaves differently on different Unix systems. `sigaction()` is POSIX and lets you choose the exact behaviour (flags, mask).

Q. What does the kill command really do?
A. It sends a signal (default `SIGTERM`) to the process with the given PID.

Q. What is an async-signal-safe function?
A. One that may be called safely from a signal handler, for example `write` and `_exit`. `printf` is not.

Q. Are signals queued?
A. Standard signals are not: repeated identical signals can merge into one.

Q. What is the signal sent by Ctrl+C?
A. `SIGINT`. (Ctrl+\ sends `SIGQUIT`, Ctrl+Z sends `SIGTSTP`.)

EXPERIMENT 6

FCFS scheduling

Aim from the lab manual

Write a C program to implement the First Come First Served (FCFS) scheduling algorithm to determine the total waiting time and average waiting time, and the total turn-around time and average turn-around time.

6.1 Big picture

Many processes want the CPU but there is only one CPU, so the operating system must *schedule*: it decides who runs next. With **FCFS** the rule is the same as in a queue at a ticket counter: the process that came first is served first, and it keeps the CPU until it has finished. The program reads the CPU time that each process needs, works out how long each process waits, and prints a table, the totals and the averages.

Assumption of your program: all processes arrive at time 0, in the order you type them (P1, P2, ...).

6.2 Concepts you need first

6.2.1 The five terms

Term	Short form	Meaning
Arrival time	AT	the time the process enters the ready queue (always 0 in your program)
Burst time	BT	the CPU time the process needs
Completion time	CT	the time the process finishes
Turn-around time	TAT	CT − AT: total time from arrival to completion
Waiting time	WT	TAT − BT: time spent waiting in the ready queue, not running

When every AT = 0 these simplify: TAT = CT, and WT = the time at which the process *starts*.

6.2.2 FCFS in four sentences

- The ready queue is a plain FIFO queue.
- It is **non-preemptive**: once a process has the CPU, nobody takes it away until the process finishes.
- There is **no starvation**: every process eventually gets its turn.
- Its weakness is the **convoy effect**: short processes stuck behind one long process make the average wait long.

6.2.3 The formulas your program uses

$$\mathrm{WT}_1 = 0 \qquad \mathrm{WT}_i = \mathrm{WT}_{i-1} + \mathrm{BT}_{i-1} \qquad \mathrm{TAT}_i = \mathrm{WT}_i + \mathrm{BT}_i$$

Process i has to wait until process $i-1$ has *started* (that is WT_{i-1}) and then *run for its whole burst* (BT_{i-1}). A Gantt chart is simply a picture of these numbers.

6.3 The program at a glance

Lines	Job
1–4	headers (only `stdio.h` is really needed)
6–9	define `struct Process` (id, burst, waiting, turn-around)
13–14	variables: count `n`, loop counter `i`, two running totals
15–21	read `n` and reject a non-positive value
22–28	create the array of processes and read each burst time
29–35	compute waiting and turn-around times (the algorithm)
36–41	print the table
42–46	add up the totals
47–50	print totals and averages

6.4 Line-by-line explanation

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

struct Process
{
    int pid, bt, wt, tat;
};

int main()
{
    int n, i;
    double awt = 0, atat = 0;
```

Line 1 `stdio.h`: `printf` and `scanf`, the only functions this program needs.

Lines 2–4 `stdlib.h`, `unistd.h` and `sys/wait.h` are never used here. They are harmless leftovers.

Lines 6–9 `struct Process { int pid, bt, wt, tat; };` defines a *new type* that groups four integers: process number, burst time, waiting time and turn-around time. It is only a blueprint; no variable exists yet. It is written outside `main` so that the whole file can use it.

Line 11–12 `int main()` and the opening brace.

Line 13 `int n, i;` — `n` is the number of processes, `i` is the loop counter.

Line 14 `double awt = 0, atat = 0;` — two accumulators. Despite their names they first hold

the **totals** of waiting time and turn-around time (the averages are computed later by dividing). They are `double` so that the divisions on lines 48 and 50 give decimals.

```
15      printf("Enter number of processes: ");
16      scanf("%d", &n);
17      if(n <= 0)
18      {
19          printf("Number of processes must be positive\n");
20          return 1;
21      }
22      struct Process P[n];
23      for(i=0; i<n; i++)
24      {
25          P[i].pid = i+1;
26          printf("Enter burst time for P%d: ", i+1);
27          scanf("%d", &P[i].bt);
28      }
```

Lines 15–16 Prompt, then `scanf("%d", &n);` reads the number of processes. The `&` gives `scanf` the address of `n`.

Lines 17–21 If `n` is zero or negative, print a message and stop with status 1. This check is needed because line 22 makes an array of size `n`, and a size must be positive.

Line 22 `struct Process P[n];` creates an array of `n` `Process` records, `P[0]` to `P[n-1]`. The size is chosen at run time (a variable-length array).

Lines 23–24 `for(i=0; i<n; i++)` visits each process in order.

Line 25 `P[i].pid = i+1;` stores the process number. Arrays start at 0 but people count from 1, hence the `+1`. The dot picks one field of a record.

Line 26 Prompts "Enter burst time for P...".

Line 27 `scanf("%d", &P[i].bt);` reads the burst time straight into the `bt` field of the ith process. `&P[i].bt` is the address of that field.

Line 28 Closing brace of the loop.

```
29      P[0].wt = 0;
30      P[0].tat = P[0].bt + P[0].wt;
31      for(i=1; i<n; i++)
32      {
33          P[i].wt = P[i-1].wt + P[i-1].bt;
34          P[i].tat = P[i].bt + P[i].wt;
35      }
```

Line 29 `P[0].wt = 0;` — the first process starts at once, so it waits 0.

Line 30 `P[0].tat = P[0].bt + P[0].wt;` — turn-around = burst + waiting.

Lines 31–32 `for(i=1; i<n; i++)` handles the rest, starting at 1 because process 0 is already done.

Line 33 `P[i].wt = P[i-1].wt + P[i-1].bt;` — waiting time of i = (when the previous one started) + (how long the previous one ran) = when the previous one *finished*.

Line 34 `P[i].tat = P[i].bt + P[i].wt;` — turn-around time of i.

Line 35 Closing brace.

```
36      printf("\n Process BT\t WT\t TAT\t");
```

```
37      for(i=0; i<n; i++)
38      {
39          printf("\n %d\t %d\t %d\t %d ", P[i].pid, P[i].bt, P[i].wt,
40                 P[i].tat);
41      }
42      for(i=0; i<n; i++)
43      {
44          awt += P[i].wt;
45          atat += P[i].tat;
46      }
```

Line 36 Prints the table heading. `\t` is a tab, which lines up the columns. The leading `\n` starts a new line.

Lines 37–41 The loop prints one row per process: id, burst, waiting, turn-around. Each row starts with `\n`, so rows appear on separate lines. The call is split over lines 39–40 only for length.

Lines 42–46 A second loop adds all waiting times to `awt` and all turn-around times to `atat`. `awt += P[i].wt;` is short for `awt = awt + P[i].wt;`.

```
47      printf("\n\nTotal Waiting Time = %.2f\n", awt);
48      printf("Average Waiting Time = %.2f\n", awt/n);
49      printf("Total Turn Around Time = %.2f\n", atat);
50      printf("Average Turn Around Time = %.2f\n", atat/n);
51      return 0;
52  }
```

Line 47 Prints the total waiting time with two decimals (`%.2f`).

Line 48 `awt/n` is the average waiting time. Because `awt` is a `double`, `n` is converted to `double` and the division is exact. If both were `int`, then $19/4$ would be 4 (the decimals would be cut off).

Line 49 Prints the total turn-around time.

Line 50 Prints the average turn-around time.

Lines 51–52 `return 0;` and the closing brace.

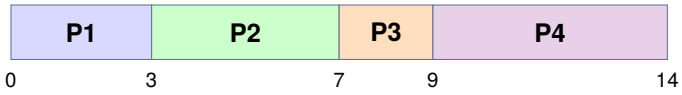
6.5 Dry run

Input from your manual: $n = 4$ and burst times $3,\ 4,\ 2,\ 5$.

i	pid	bt	wt (lines 29, 33)	tat (lines 30, 34)	
0	1	3	0	$3 + 0 = 3$	first process
1	2	4	$0 + 3 = 3$	$4 + 3 = 7$	
2	3	2	$3 + 4 = 7$	$2 + 7 = 9$	
3	4	5	$7 + 2 = 9$	$5 + 9 = 14$	

Total waiting time $= 0 + 3 + 7 + 9 = 19$, average $= 19/4 = 4.75$.
Total turn-around time $= 3 + 7 + 9 + 14 = 33$, average $= 33/4 = 8.25$.

Gantt chart (the picture of who uses the CPU when):



6.6 The output, explained

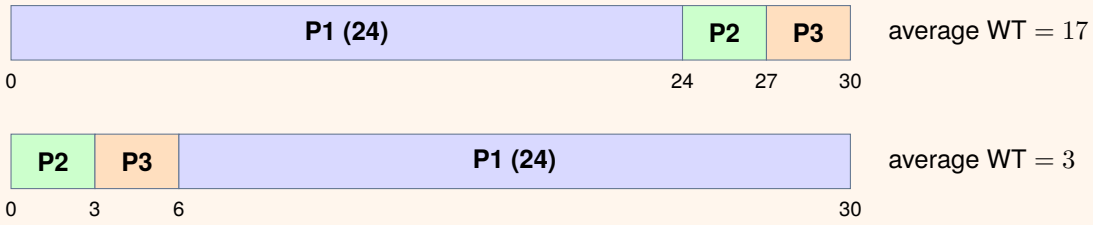
```
 Process BT   WT   TAT
 1       3    0    3
 2       4    3    7
 3       2    7    9
 4       5    9    14
Total Waiting Time = 19.00
Average Waiting Time = 4.75
Total Turn Around Time = 33.00
Average Turn Around Time = 8.25
```

Each row is one pass of the printing loop (lines 37–41). The totals come from lines 42–46, and the averages are the totals divided by 4. All of these numbers match the dry run above, and my own run of your program printed exactly these values. I also ran your program against an independent simulator on 400 random inputs, and every result matched.

6.7 Gotchas and the best method

1. The convoy effect — tested with your program

Run the textbook example with bursts $24, 3, 3$ and then with the same jobs in the order $3, 3, 24$.



Your program printed **17.00** and **27.00** (average waiting and turn-around) for the first order, and **3.00** and **13.00** for the second. Same work, very different waiting. A long job at the front makes everyone wait: that is the convoy effect.

2. Other points

- The variable names `awt` and `atat` suggest "average" but hold *totals* until line 48 and 50 divide them.
- The program assumes every process arrives at time 0. For other arrival times you need the formulas below.
- `scanf` results are not checked: typing a letter instead of a number leaves the variable unchanged.

Twist: FCFS with arrival times (tested)

If your examiner gives arrival times, three things change: sort the processes by arrival time, let the CPU *idle* when no process has arrived, and use

$$\mathrm{CT}_i = \max(\mathrm{CT}_{i-1},\ \mathrm{AT}_i) + \mathrm{BT}_i \qquad \mathrm{TAT}_i = \mathrm{CT}_i - \mathrm{AT}_i \qquad \mathrm{WT}_i = \mathrm{TAT}_i - \mathrm{BT}_i$$

```c
#include <stdio.h>

struct Process { int pid, at, bt, ct, tat, wt; };

int main(void)
{
    int n, i, j, time = 0;
    double twt = 0, ttat = 0;
    printf("Enter number of processes: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 1;
    struct Process p[n], temp;
    for (i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Arrival time and burst time of P%d: ", i + 1);
        if (scanf("%d %d", &p[i].at, &p[i].bt) != 2) return 1;
    }
    for (i = 0; i < n - 1; i++)                 /* bubble sort by arrival time */
        for (j = 0; j < n - 1 - i; j++)         /* (equal times keep pid order) */
            if (p[j].at > p[j + 1].at) {
                temp = p[j]; p[j] = p[j + 1]; p[j + 1] = temp;
            }
    for (i = 0; i < n; i++) {
        if (time < p[i].at) time = p[i].at;     /* CPU idle until it arrives */
        time += p[i].bt;                        /* run the whole burst       */
        p[i].ct  = time;                        /* completion time           */
        p[i].tat = p[i].ct - p[i].at;           /* turnaround = CT - AT      */
        p[i].wt  = p[i].tat - p[i].bt;          /* waiting    = TAT - BT     */
        twt += p[i].wt;  ttat += p[i].tat;
    }
    printf("\nPID AT BT CT TAT WT\n");
    for (i = 0; i < n; i++)
        printf("P%d %d %d %d %d %d\n", p[i].pid, p[i].at, p[i].bt,
               p[i].ct, p[i].tat, p[i].wt);
    printf("Average WT = %.2f   Average TAT = %.2f\n", twt / n, ttat / n);
    return 0;
}
```

Check: for (AT, BT) = (0,5), (1,3), (2,8), (3,6) it prints

```
P1 0 5 5  5  0
P2 1 3 8  7  4
P3 2 8 16 14 6
P4 3 6 22 19 13        Average WT = 5.75   Average TAT = 11.25
```

With a gap, for example P1 (0,2) and P2 (5,3), the CPU idles from time 2 to 5 and P2 starts at 5, so its waiting time is 0. The `if` at the start of the second loop (the one that sets `time = p[i].at`) is what handles the gap. This program matched an independent simulator on 400 random inputs.

Write it from memory

1. `struct Process { int pid, bt, wt, tat; };` and read `n`, then `P[n]`.
2. Read each `bt`. Then `P[0].wt = 0;`
3. `for (i=1; i<n; i++) P[i].wt = P[i-1].wt + P[i-1].bt;` and `tat = bt + wt` for every process.
4. Add the totals in two `double` variables; averages are total divided by `n`.

Remember: **wait = previous wait + previous burst; turn-around = wait + burst**.

Viva questions

Q. Define burst time, waiting time and turn-around time.
A. Burst time: CPU time the process needs. Turn-around time: completion minus arrival. Waiting time: turn-around minus burst, the time spent waiting in the ready queue.

Q. Is FCFS pre-emptive?
A. No. A running process keeps the CPU until it finishes (or blocks for I/O).

Q. What is the convoy effect?
A. Short processes wait behind a long one, which raises the average waiting time and lowers CPU and device utilisation.

Q. Can FCFS cause starvation?
A. No: every process eventually reaches the front of the queue.

Q. Why is the waiting time of the first process 0?
A. It arrives at 0 and starts at 0, so it never waits.

Q. What is a Gantt chart?
A. A bar chart of the schedule, showing which process runs during which time interval.

Q. Why are awt and atat declared double?
A. So that the division by `n` gives an exact decimal result instead of an integer.

Q. What is the time complexity?
A. $O(n)$ for the calculation (one pass); sorting by arrival time, if needed, adds $O(n \log n)$ or $O(n^2)$ for a simple sort.

Q. What are the advantages and disadvantages of FCFS?
A. Simple, fair in arrival order, no starvation. But a long job delays all others, so the average waiting time can be large; it is poor for interactive systems.

Q. Compute: bursts 24, 3, 3 in this order. Average waiting time?
A. $(0 + 24 + 27)/3 = 17$.

EXPERIMENT 7

SJF scheduling

Aim from the lab manual

Write a C program to implement the Shortest Job First (SJF) scheduling algorithm to determine the total waiting time, average waiting time, total turn-around time and average turn-around time.

7.1 Big picture

SJF is FCFS with one change: instead of serving processes in the order they were typed, the CPU always goes to the process with the **smallest burst time**. The program reads the burst times, **sorts** the processes by burst time, and then applies exactly the same waiting-time formulas as Experiment 6 to the sorted list.

As in Experiment 6, all processes arrive at time 0 and a process, once started, runs to completion (*non-preemptive* SJF).

7.2 Concepts you need first

- **Rule:** run the ready process with the shortest next CPU burst. If two bursts are equal, serve the one that came first (FCFS).
- **SJF is optimal:** for a set of processes that are all available at the same time, it gives the *smallest possible average waiting time*. The reason: moving a short job in front of a long one shortens the short job's wait by more than it lengthens the long job's wait. Example with bursts $6, 8, 7, 3$: FCFS gives an average wait of 10.25, SJF gives 7 (I ran both).
- **The catch:** the operating system cannot really know how long the next burst will be. Real systems *predict* it from the past with an exponential average, $\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\, \tau_n$, where t_n is the last measured burst and τ_n the previous prediction.
- **Starvation:** a long process can wait for ever if short processes keep arriving.
- **Preemptive SJF** is called *Shortest Remaining Time First* (SRTF): if a new process arrives with a burst shorter than what is left of the running process, the CPU is taken away from it. A version is given at the end of the chapter.

7.2.1 Selection sort in brief

The program sorts with **selection sort**: for each position i from the left, find the smallest element among positions $i, i+1, \ldots$ and swap it into position i. After the first pass the smallest is in place; after the second the next smallest, and so on.

7.3 The program at a glance

Lines	Job
1–9	headers and `struct Process` (same as Experiment 6)
13–22	variables, read `n`, create the array `p` and a spare record `temp`
23–28	read the burst times
29–44	**sort** the array by burst time (selection sort, ties by process number)
45–51	waiting and turn-around times, in sorted order (the FCFS formulas)
52–57	print the table
58–62	add up totals
63–66	print totals and averages

7.4 Line-by-line explanation

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  struct Process
7  {
8      int pid, bt, wt, tat;
9  };
10
11 int main()
12 {
13     int n, min;
14     double awt=0, atat=0;
15     printf("Enter no of process: ");
16     scanf("%d", &n);
17     if(n <= 0)
18     {
19         printf("Number of processes must be positive\n");
20         return 1;
21     }
22     struct Process p[n], temp;
```

Lines 1–4 Headers. Only `stdio.h` is needed; the other three are unused leftovers.

Lines 6–9 The same `struct Process` as before: `pid`, `bt` (burst), `wt` (waiting), `tat` (turn-around).

Line 13 `int n, min;` — `n` is the number of processes. `min` will hold the *position* (index) of the smallest burst found so far during sorting.

Line 14 `double awt=0, atat=0;` — running totals of waiting and turn-around time.

Lines 15–21 Read `n`; if it is not positive, print a message and stop.

Line 22 `struct Process p[n], temp;` creates the array of `n` processes *and* one extra record, `temp`, used as a parking place when two records are swapped.

```
23     for(int i=0; i<n; i++)
24     {
25         p[i].pid = i+1;
```

```
26          printf("Enter burst time for p%d : ", i+1);
27          scanf("%d", &p[i].bt);
28      }
```

Line 23 `for(int i=0; i<n; i++)` — the loop variable is declared right inside the `for` (allowed since C99).

Lines 25–27 Store the process number `i+1` and read the burst time into `p[i].bt` (note the `&`).

```
29      for(int i=0; i<n; i++)
30      {
31          min = i;
32          for(int j=i+1; j<n; j++)
33          {
34              /* equal burst times: keep arrival (pid) order */
35              if(p[j].bt < p[min].bt ||
36                 (p[j].bt == p[min].bt && p[j].pid < p[min].pid))
37              {
38                  min = j;
39              }
40          }
41          temp = p[i];
42          p[i] = p[min];
43          p[min] = temp;
44      }
```

Line 29 Outer loop: `i` is the position that is about to receive the next-smallest process.

Line 31 `min = i;` — assume, for now, that the smallest burst is the one already at position `i`.

Line 32 Inner loop: `j` walks over all positions after `i`.

Line 34 A comment: for equal burst times, keep arrival (pid) order.

Lines 35–36 The test:

- `p[j].bt < p[min].bt` — process `j` has a *shorter* burst than the best so far; **or**
- `(p[j].bt == p[min].bt && p[j].pid < p[min].pid)` — the bursts are equal but `j` has the smaller process number (it came first).

Lines 37–39 If the test is true, `min = j;` remembers `j` as the new best.

Line 40 End of the inner loop. Now `min` is the position of the smallest burst among positions `i` …`n-1`.

Lines 41–43 **The swap.** `temp = p[i];` copies the whole record at `i` into `temp`; `p[i] = p[min];` copies the smallest record into position `i`; `p[min] = temp;` puts the old record where the smallest one was. Because *whole records* move, each `pid` travels with its burst time, so the program can still print the original process number later. If `min == i` the swap does nothing.

Line 44 End of the outer loop. The array is now sorted by burst time.

```
45      p[0].wt = 0;
46      p[0].tat = p[0].bt + p[0].wt;
47      for(int i=1; i<n; i++)
48      {
49          p[i].wt = p[i-1].wt + p[i-1].bt;
50          p[i].tat = p[i].wt + p[i].bt;
51      }
```

Lines 45–46 The first (shortest) process waits 0, and its turn-around time is burst + wait.

Lines 47–51 Exactly the FCFS rule, but on the *sorted* array: `p[i].wt = p[i-1].wt + p[i-1].bt;` and `p[i].tat = p[i].wt + p[i].bt;`. So SJF = "sort by burst time, then FCFS".

```
52      printf("\nProcess BT\t WT\t TAT");
53      for(int i=0; i<n; i++)
54      {
55          printf("\n%d\t %d\t %d\t %d", p[i].pid, p[i].bt,
56                 p[i].wt, p[i].tat);
57      }
58      for(int i=0; i<n; i++)
59      {
60          awt += p[i].wt;
61          atat += p[i].tat;
62      }
63      printf("\n\nTotal Waiting Time = %.2f\n", awt);
64      printf("Total Turn Around Time = %.2f\n", atat);
65      printf("Average Waiting Time = %.2f\n", awt/n);
66      printf("Average Turn Around Time = %.2f\n", atat/n);
67      return 0;
68  }
```

Line 52 Prints the table heading.

Lines 53–57 One row per process, *in sorted order*: the first column shows the original process number so you can see who ran when.

Lines 58–62 Adds up all waiting times and turn-around times.

Lines 63–66 Prints the totals and the averages (total divided by `n`; `awt` is a `double`, so the division is exact).

Lines 67–68 `return 0;` and the closing brace.

7.5 Dry run

Input from your manual: $n = 3$, burst times $P1 = 6,\ P2 = 7,\ P3 = 2$.

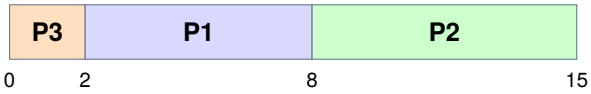
Sorting (lines 29–44):

i	Search for the smallest in positions i...2	Array after the swap
start	—	[P1(6), P2(7), P3(2)]
0	`min`=0; j=1: $7 < 6$? no; j=2: $2 < 6$ yes, `min`=2	swap positions 0 and 2: [P3(2), P2(7), P1(6)]
1	`min`=1; j=2: $6 < 7$ yes, `min`=2	swap positions 1 and 2: [P3(2), P1(6), P2(7)]
2	nothing to compare; `min`=2	swaps with itself: [P3(2), P1(6), P2(7)]

Times (lines 45–51):

Order	Process	BT	WT	TAT
1st	P3	2	0	$2+0=2$
2nd	P1	6	$0+2=2$	$2+6=8$
3rd	P2	7	$2+6=8$	$8+7=15$

Totals: waiting $0+2+8=10$, average $10/3=3.33$; turn-around $2+8+15=25$, average $25/3=8.33$.



Why the tie-break in lines 35–36 is needed — tested

Selection sort can reorder equal elements. Try bursts $P1=5,\ P2=5,\ P3=1$:

i	What happens
0	min $=0$; P2: $5<5$ no, tie but pid $2<1$ no; P3: $1<5$ yes. Swap positions 0 and 2 $\Rightarrow$ [P3, P2, P1]. **P1 and P2 are now in the wrong order.**
1	min $=1$ (P2); P1: $5<5$ no, but *tie and pid* $1<2$: yes. Swap $\Rightarrow$ [P3, P1, P2]. The tie-break repairs the order.

Without the second half of the condition the result would be P3, P2, P1: the process that arrived first would run after its equal. I ran `5 5 1` and the program printed P3, P1, P2.

7.6 The output, explained

```
Process BT   WT   TAT
3       2    0    2          <- P3 first: shortest burst
1       6    2    8
2       7    8    15
Total Waiting Time = 10.00
Total Turn Around Time = 25.00
Average Waiting Time = 3.33
Average Turn Around Time = 8.33
```

The first column is the *original* process number, which is why it reads 3, 1, 2: that is the order of execution. Compare with FCFS on the same data (bursts 6, 7, 2): waiting times $0, 6, 13$ give an average of 6.33, against 3.33 for SJF. I also checked your program against an independent simulator on 400 random inputs (all matched), and with the textbook example $6, 8, 7, 3$, for which it gave the textbook answer of 7 for the average waiting time.

7.7 Gotchas and the best method

Limits of this program

- It assumes **all arrival times are 0** and is **non-preemptive**.
- The OS does not know burst times in advance; in a real system they are only estimated.
- Long processes can starve if short ones keep coming.

- Selection sort takes $O(n^2)$ steps, fine for a lab but slow for thousands of processes (the library function `qsort` would be faster).

Twist 1: SJF with arrival times, non-preemptive (tested)

Now the ready queue changes over time. Keep a clock `time`. Whenever the CPU is free, look at the processes that have *already arrived* (`p[i].at <= time`) and are not done, pick the one with the shortest burst, and run it to the end. If nobody has arrived, the clock simply moves on.

```c
#include <stdio.h>

struct Process { int pid, at, bt, ct, tat, wt, done; };

int main(void)
{
    int n, i, finished = 0, time = 0;
    double twt = 0, ttat = 0;
    printf("Enter number of processes: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 1;
    struct Process p[n];
    for (i = 0; i < n; i++) {
        p[i].pid = i + 1;  p[i].done = 0;
        printf("Arrival time and burst time of P%d: ", i + 1);
        if (scanf("%d %d", &p[i].at, &p[i].bt) != 2) return 1;
    }
    while (finished < n) {
        int k = -1;                              /* index of the chosen process */
        for (i = 0; i < n; i++)
            if (!p[i].done && p[i].at <= time &&             /* has arrived     */
                (k == -1 || p[i].bt < p[k].bt ||             /* shorter burst   */
                 (p[i].bt == p[k].bt && p[i].at < p[k].at))) /* tie: earlier AT */
                k = i;
        if (k == -1) { time++; continue; }       /* nobody has arrived: CPU idle */
        time += p[k].bt;                         /* run it to completion         */
        p[k].ct  = time;
        p[k].tat = p[k].ct - p[k].at;
        p[k].wt  = p[k].tat - p[k].bt;
        p[k].done = 1;  finished++;
        twt += p[k].wt;  ttat += p[k].tat;
    }
    printf("\nPID AT BT CT TAT WT\n");
    for (i = 0; i < n; i++)
        printf("P%d %d %d %d %d %d\n", p[i].pid, p[i].at, p[i].bt,
               p[i].ct, p[i].tat, p[i].wt);
    printf("Average WT = %.2f   Average TAT = %.2f\n", twt / n, ttat / n);
    return 0;
}
```

Check: for (AT, BT) = (0,8), (1,4), (2,9), (3,5) the order is P1, P2, P4, P3 and the average waiting time is **7.75**; for (0,7), (2,4), (4,1), (5,4) it is **4.00**. Both are the textbook answers.

Twist 2: preemptive SJF = SRTF (tested)

At *every time unit* choose, among the arrived processes, the one with the least *remaining* time and run it for one unit.

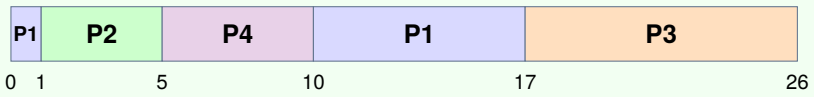
```c
#include <stdio.h>

struct Process { int pid, at, bt, rem, ct, tat, wt; };

```

```c
int main(void)
{
    int n, i, finished = 0, time = 0;
    double twt = 0, ttat = 0;
    printf("Enter number of processes: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 1;
    struct Process p[n];
    for (i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Arrival time and burst time of P%d: ", i + 1);
        if (scanf("%d %d", &p[i].at, &p[i].bt) != 2) return 1;
        p[i].rem = p[i].bt;                     /* remaining burst time */
    }
    while (finished < n) {
        int k = -1;
        for (i = 0; i < n; i++)                 /* arrived, unfinished, least remaining */
            if (p[i].rem > 0 && p[i].at <= time &&
                (k == -1 || p[i].rem < p[k].rem))
                k = i;
        if (k == -1) { time++; continue; }      /* CPU idle */
        p[k].rem--;                             /* run for ONE time unit */
        time++;
        if (p[k].rem == 0) {                    /* just finished */
            p[k].ct  = time;
            p[k].tat = p[k].ct - p[k].at;
            p[k].wt  = p[k].tat - p[k].bt;
            finished++;
            twt += p[k].wt;  ttat += p[k].tat;
        }
    }
    printf("\nPID AT BT CT TAT WT\n");
    for (i = 0; i < n; i++)
        printf("P%d %d %d %d %d %d\n", p[i].pid, p[i].at, p[i].bt,
               p[i].ct, p[i].tat, p[i].wt);
    printf("Average WT = %.2f   Average TAT = %.2f\n", twt / n, ttat / n);
    return 0;
}
```

For (0,8), (1,4), (2,9), (3,5):



The waiting times are P1 = 9, P2 = 0, P3 = 15, P4 = 2, so the average is $26/4 = \mathbf{6.5}$, which is better than the 7.75 of the non-preemptive version. For (0,7), (2,4), (4,1), (5,4) it gives 3.00 against 4.00. Both programs matched an independent simulator on 400 random inputs.

Write it from memory

1. Same struct, same input loop as FCFS.
2. **Sort by burst time**: for each `i`, find `min` among `j > i` (tie: smaller `pid`), then swap whole records with a `temp` record.
3. Apply the FCFS formulas on the sorted array: `wt[0]=0; wt[i]=wt[i-1]+bt[i-1]; tat=wt+bt`.
4. Print, total, divide.

Remember: **SJF = sort by burst, then FCFS**.

Viva questions

Q. What is SJF?
A. A scheduling algorithm that always runs the ready process with the shortest CPU burst next.

Q. Why is SJF optimal?
A. For processes that are all ready at once, it gives the minimum average waiting time, because putting a shorter job first reduces total waiting more than it increases it.

Q. What is the main difficulty of SJF?
A. The length of the next CPU burst is not known in advance; it has to be predicted (exponential averaging).

Q. Can SJF cause starvation?
A. Yes. A long process may never run if shorter processes keep arriving.

Q. Difference between SJF and SRTF?
A. SJF (non-preemptive) lets a running process finish. SRTF (preemptive SJF) interrupts it when a process with a shorter remaining time arrives.

Q. Why do you swap whole structs?
A. So that the process number moves together with its burst time and can be printed after sorting.

Q. Why the extra pid comparison when burst times are equal?
A. Selection sort is not stable; the tie-break keeps equal processes in arrival order.

Q. Compute: bursts 6, 8, 7, 3. Average waiting time with SJF?
A. Order 3, 6, 7, 8 gives waits $0, 3, 9, 16$; average $28/4 = 7$.

Q. What is the time complexity of your program?
A. $O(n^2)$ because of the selection sort; the rest is $O(n)$.

EXPERIMENT 8

Round Robin scheduling

Aim from the lab manual

Write a C program to implement the Round Robin scheduling algorithm to determine the total waiting time, total turn-around time, average waiting time and average turn-around time.

8.1 Big picture

Round Robin (RR) is made for sharing the CPU fairly. Every process gets the CPU for *at most one time quantum* (a small fixed time slice, for example 2 ms). If it is not finished when the quantum ends, it is moved to the back of the queue and the next process gets its turn. This repeats, round and round, until every process has finished.

As in the other scheduling experiments, your program assumes that all processes arrive at time 0.

8.2 Concepts you need first

- **Time quantum** (q): the longest time a process may run before it is pre-empted. RR is therefore **pre-emptive**.
- **Circular ready queue:** processes take turns in a fixed cycle P1, P2, P3, ..., P1, P2, ...Finished processes drop out.
- A process runs for $\min(q,\ \text{remaining burst})$ in each turn. If it needs less than q, it simply gives the CPU back early.
- **Choosing q:** if q is very large, RR behaves exactly like FCFS. If q is very small, the CPU wastes much time on *context switches* (saving one process and loading the next). A common rule of thumb is that about 80% of the CPU bursts should be shorter than q.
- **No starvation** and good response time: with n processes in the queue, no process waits more than $(n-1)\,q$ before its next turn. The price is that the average turn-around time is usually worse than for SJF.
- With all arrival times 0: $\text{TAT} = \text{completion time}$ and $\text{WT} = \text{TAT} - \text{BT}$.

8.3 The program at a glance

Lines	Job
3–6	`struct Process` (id, burst, waiting, turn-around `tt`)
10–11	variables, including the clock `time` and the loop flag
12–27	read `n` and the burst times; copy each burst into `remBT[]` (remaining time)
28–34	read the time quantum `tq`
35–55	**the scheduler**: repeated passes over the processes until all are finished
56–63	print the table and add up the totals
64–69	compute and print totals and averages

8.4 Line-by-line explanation

```
1  #include <stdio.h>
2
3  struct Process
4  {
5      int pid, bt, wt, tt;
6  };
7
8  int main()
9  {
10     int i, n, tq, time=0, totwt=0, tottt=0, flag=0;
11     float avgwt, avgtt;
```

Line 1 `stdio.h` is the only header; this program needs nothing else.

Lines 3–6 `struct Process { int pid, bt, wt, tt; };` — process number, burst time, waiting time and turn-around time (`tt`).

Lines 8–9 `int main()` and the opening brace.

Line 10 Seven integers in one declaration:

- `i` — loop counter; `n` — number of processes; `tq` — time quantum.
- `time=0` — the **simulated clock**: how many time units have passed. It starts at 0 and only moves forward.
- `totwt=0, tottt=0` — totals of waiting and turn-around time.
- `flag=0` — controls the main loop (explained on line 35).

Line 11 `float avgwt, avgtt;` — the two averages, which can have decimals.

```
12     printf("Enter the number of processes: ");
13     scanf("%d", &n);
14     if(n <= 0)
15     {
16         printf("Number of processes must be positive\n");
17         return 1;
18     }
19     struct Process p[n];
20     int remBT[n];
21     for(i=0; i<n; i++)
22     {
23         p[i].pid = i+1;
```

```
24          printf("Enter burst time for p%d : ", i+1);
25          scanf("%d", &p[i].bt);
26          remBT[i] = p[i].bt;
27      }
```

Lines 12–13 Prompt and `scanf("%d", &n);`

Lines 14–18 If `n` is zero or negative: message and stop.

Line 19 `struct Process p[n];` — the array of processes.

Line 20 `int remBT[n];` — **remaining burst time** of each process. It is a separate array because the original burst `p[i].bt` must be kept: it is needed at the end to compute WT = TAT − BT.

Lines 21–27 Loop over the processes: store the id, read the burst time into `p[i].bt`, and copy it into `remBT[i]` (line 26). At the start the whole burst is still remaining.

```
28      printf("Enter the time quantum: ");
29      scanf("%d", &tq);
30      if(tq <= 0)
31      {
32          printf("Time quantum must be positive\n");
33          return 1;
34      }
```

Lines 28–29 Prompt and read the time quantum `tq`.

Lines 30–34 A quantum of zero or less would give no progress, so the loop below would never finish. The program rejects it and stops.

```
35      while(flag==0)
36      {
37          flag=1;
38          for(i=0; i<n; i++)
39          {
40              if(remBT[i] > 0)
41              {
42                  flag=0;
43                  if(remBT[i] > tq)
44                  {
45                      time = time + tq;
46                      remBT[i] = remBT[i]-tq;
47                  } else {
48                      time = time + remBT[i];
49                      remBT[i] = 0;
50                      p[i].tt = time;
51                      p[i].wt = p[i].tt - p[i].bt;
52                  }
53              }
54          }
55      }
```

Line 35 `while(flag==0)` — repeat *passes* over the processes. A pass is one trip round the circular queue. The loop continues while `flag` is 0.

Line 37 `flag=1;` — at the start of each pass, *assume* that nothing is left to do.

Line 38 `for(i=0; i<n; i++)` — visit P1, P2, ...in order: the turns of one round.

Line 40 `if(remBT[i] > 0)` — does process `i` still need the CPU? If not (already finished), skip it.

Line 42 `flag=0;` — we found unfinished work, so the assumption of line 37 was wrong: at least one more pass will be needed.

Line 43 `if(remBT[i] > tq)` — needs more than one quantum.

Lines 45–46 It runs for a *full quantum*: the clock advances by `tq` (`time = time + tq;`) and its remaining time shrinks by `tq`. It is not finished, so it simply waits for its next turn in the next pass.

Line 47 `else` — needs a quantum or less, so it will finish now.

Line 48 `time = time + remBT[i];` — it runs only for what it still needs; the clock advances by that amount.

Line 49 `remBT[i] = 0;` — nothing remains.

Line 50 `p[i].tt = time;` — the process finishes *now*, so its turn-around time equals the current clock (arrival time is 0).

Line 51 `p[i].wt = p[i].tt - p[i].bt;` — waiting time = turn-around − burst.

Lines 52–55 Closing braces: end of `else`, of `if`, of `for` (one pass done) and of `while`. The `while` now tests `flag`: still 0 means another pass; 1 means every process was already finished during the last pass, so stop.

```
56      printf("\nProcess BT\t WT\t TAT ");
57      for(i=0; i<n; i++)
58      {
59          printf("\n %d\t %d\t %d\t %d ", p[i].pid, p[i].bt,
60                 p[i].wt, p[i].tt);
61          totwt = totwt + p[i].wt;
62          tottt = tottt + p[i].tt;
63      }
64      avgwt = (float) totwt/n;
65      avgtt = (float) tottt/n;
66      printf("\nTotal Waiting Time: %d ms\nAverage Waiting Time: %.2f ms\n",
67             totwt, avgwt);
68      printf("Total Turn Around Time: %d ms\nAverage Turn Around Time: %.2f ms\n",
69             tottt, avgtt);
70      return 0;
71  }
```

Line 56 Prints the table heading.

Lines 57–63 The loop prints one row per process (id, burst, waiting, turn-around) and, in the same loop, adds `p[i].wt` to `totwt` and `p[i].tt` to `tottt`.

Line 64 `avgwt = (float) totwt/n;` — the **cast** `(float)` turns `totwt` into a float *before* the division, so the division keeps its decimals. Without it, $26/4$ would be the integer 6.

Line 65 The same for turn-around time.

Lines 66–69 Print the totals (`%d`) and averages (`%.2f`) followed by "ms".

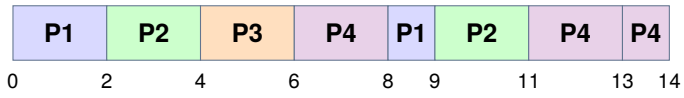
Lines 70–71 `return 0;` and the closing brace.

8.5 Dry run

Input from your manual: burst times $P1 = 3,\ P2 = 4,\ P3 = 2,\ P4 = 5$ and quantum $q = 2$. The clock `time` starts at 0.

Pass	Process	remBT before	runs for	time after	remBT after / result
1	P1	3	2 (full quantum)	2	1
1	P2	4	2	4	2
1	P3	2	2 (finishes)	6	0: TT$=6$, WT$=6-2=4$
1	P4	5	2	8	3
2	P1	1	1 (finishes)	9	0: TT$=9$, WT$=9-3=6$
2	P2	2	2 (finishes)	11	0: TT$=11$, WT$=11-4=7$
2	P3	0	skipped	—	—
2	P4	3	2	13	1
3	P4	1	1 (finishes)	14	0: TT$=14$, WT$=14-5=9$
4	all	0	—	—	`flag` stays 1: loop ends

Totals: waiting $6+7+4+9=26$, average 6.50; turn-around $9+11+6+14=40$, average 10.00. (Pass 4 is an empty pass that only proves that everyone has finished.)



The last two blocks are both P4, because P4 is the only process left; they can be drawn as one block from 11 to 14. There are 8 time slices, but the last two both belong to P4, so the CPU switches from one process to a different one only 6 times (P1→P2, P2→P3, P3→P4, P4→P1, P1→P2, P2→P4).

8.6 The output, explained

```
Process BT   WT   TAT
 1      3    6    9
 2      4    7    11
 3      2    4    6
 4      5    9    14
Total Waiting Time: 26 ms
Average Waiting Time: 6.50 ms
Total Turn Around Time: 40 ms
Average Turn Around Time: 10.00 ms
```

Every number matches the dry run, and my run of your program gave exactly this output. I also compared it with an independent simulator on 400 random inputs, and all results matched. Note that the rows are in process order (1, 2, 3, 4), unlike the SJF output, because RR never reorders the array.

8.7 Gotchas and the best method

1. The quantum changes the result — tested with your program

For the textbook jobs with bursts $24, 3, 3$:

Quantum	Avg waiting	Avg turn-around	Comment
1	5.67	15.67	many switches
2	6.33	16.33	
4	5.67	15.67	the textbook example
6	7.00	17.00	
100	17.00	27.00	larger than every burst: **identical to FCFS**

The average does not rise or fall steadily with q, and a huge q turns RR into FCFS. Your program ignores the time lost in context switches; in a real system a very small q is expensive.

2. Other points

- The array scan is a correct "circular queue" only because all processes arrive at time 0 and none is added later. With arrival times you need a real queue (below).
- The variable `time` has the same name as the library function `time()`. It is fine here, because `time.h` is not included, but avoid the clash in larger programs.
- The totals are `int` and the averages `float`. The cast on lines 64–65 is what prevents integer division.
- If the examiner asks for the Gantt chart, record each slice (process, start, end) while the loop runs.

Twist: Round Robin with arrival times and a real queue (tested)

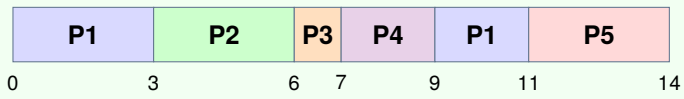
New processes must join the queue *when they arrive*. The program keeps an array `q[]` used as a circular queue (`head` = front, `tail` = back, `count` = how many are waiting). Steps: admit everything that has arrived, take the front process, run it for $\min(q, \text{remaining})$, admit the processes that arrived *during* that slice, and only then put the unfinished process at the back. If the queue is empty, the clock jumps to the next arrival.

```
1  #include <stdio.h>
2
3  struct Process { int pid, at, bt, rem, ct, tat, wt; };
4
5  int main(void)
6  {
7      int n, i, j, tq, time = 0, next = 0, finished = 0;
8      int head = 0, tail = 0, count = 0;          /* circular ready queue */
9      double twt = 0, ttat = 0;
10     printf("Enter number of processes: ");
11     if (scanf("%d", &n) != 1 || n <= 0) return 1;
12     struct Process p[n], temp;
13     int q[n];                                   /* holds indexes of waiting processes */
14     for (i = 0; i < n; i++) {
15         p[i].pid = i + 1;
```

```
        printf("Arrival time and burst time of P%d: ", i + 1);
        if (scanf("%d %d", &p[i].at, &p[i].bt) != 2) return 1;
        p[i].rem = p[i].bt;
    }
    printf("Enter the time quantum: ");
    if (scanf("%d", &tq) != 1 || tq <= 0) return 1;
    for (i = 0; i < n - 1; i++)                  /* sort by arrival time */
        for (j = 0; j < n - 1 - i; j++)
            if (p[j].at > p[j + 1].at) {
                temp = p[j]; p[j] = p[j + 1]; p[j + 1] = temp;
            }
    while (finished < n) {
        while (next < n && p[next].at <= time) { /* admit everyone who has arrived */
            q[tail] = next++;  tail = (tail + 1) % n;  count++;
        }
        if (count == 0) { time = p[next].at; continue; }   /* CPU idle: jump ahead */
        i = q[head];  head = (head + 1) % n;  count--;     /* take the front process */
        int slice = (p[i].rem < tq) ? p[i].rem : tq;
        time += slice;
        p[i].rem -= slice;
        while (next < n && p[next].at <= time) { /* arrivals during the slice go FIRST */
            q[tail] = next++;  tail = (tail + 1) % n;  count++;
        }
        if (p[i].rem > 0) {                      /* not finished: back of the queue */
            q[tail] = i;  tail = (tail + 1) % n;  count++;
        } else {                                 /* finished */
            p[i].ct  = time;
            p[i].tat = p[i].ct - p[i].at;
            p[i].wt  = p[i].tat - p[i].bt;
            finished++;
            twt += p[i].wt;  ttat += p[i].tat;
        }
    }
    printf("\nPID AT BT CT TAT WT\n");
    for (i = 0; i < n; i++)
        printf("P%d %d %d %d %d %d\n", p[i].pid, p[i].at, p[i].bt,
               p[i].ct, p[i].tat, p[i].wt);
    printf("Average WT = %.2f   Average TAT = %.2f\n", twt / n, ttat / n);
    return 0;
}
```

Convention: when a new process arrives at the same moment as a quantum ends, it is queued *before* the pre-empted process. This is the usual textbook convention, but some books do the opposite, so follow your teacher's.

Check: (AT, BT) = (0,5), (1,3), (2,1), (3,2), (4,3) with $q = 3$ gives



with waiting times $6, 2, 4, 4, 7$, so the average waiting time is $23/5 = \mathbf{4.60}$ and the average turnaround time is $37/5 = \mathbf{7.40}$. I verified this by hand and the program agrees. It matched an independent simulator on 400 random inputs, and it reproduces the textbook example (bursts $24, 3, 3$, $q = 4$, average wait 5.67).

Write it from memory

1. Struct, read `n`, burst times, copy them to `remBT[]`; read `tq`.
2. `while (flag == 0) { flag = 1; for (i = 0; i < n; i++) if (remBT[i] > 0) { flag = 0; ... } }`
3. Inside: `if (remBT[i] > tq) { time += tq; remBT[i] -= tq; } else { time += remBT[i]; remBT[i] = 0; tt = time; wt = tt - bt; }`
4. Total, average with a `(float)` cast, print.

Remember: **run for min(quantum, remaining); when done, TT = clock and WT = TT − BT.**

Viva questions

Q. What is a time quantum?
A. The maximum time a process may run before the scheduler pre-empts it and gives the CPU to the next process.

Q. Is Round Robin pre-emptive?
A. Yes. A process is pre-empted when its quantum expires.

Q. What happens if the quantum is very large? Very small?
A. Very large: RR behaves like FCFS. Very small: many context switches, so much of the CPU time is overhead.

Q. What is a context switch?
A. Saving the state of the running process and loading the state of the next one so it can run.

Q. Can RR cause starvation?
A. No. Every process gets a turn at least once every $(n-1)q$ time units.

Q. Why do you keep a separate remBT array?
A. To track remaining time without destroying the original burst, which is needed to compute waiting time.

Q. What is the job of the flag variable?
A. It ends the `while` loop when a full pass finds no process with remaining time.

Q. Why the cast (float) before totwt/n?
A. Both are integers, so plain division would drop the decimals. The cast makes it a floating-point division.

Q. How is waiting time obtained here?
A. Turn-around time (completion time, since arrival is 0) minus burst time.

Q. Which scheduling algorithm gives the best response time for interactive systems?
A. Round Robin, because every process gets the CPU regularly.

EXPERIMENT 9

Producer–Consumer using semaphores

Aim from the lab manual

Write a C program to simulate the producer–consumer problem using semaphores.

Read this first: the manual's program contains no semaphore

The program in your manual includes `pthread.h` but it creates no thread, and it never calls `sem_init`, `sem_wait` or `sem_post`. It is a **menu-driven simulation** of the bounded buffer: you choose "Produce" or "Consume" by hand, and an integer `item` counts how full the buffer is. It works and its output is correct, but the aim says "using semaphores". An examiner who reads the aim literally may expect `sem_t`.

So this chapter has **three programs**:

- **Program A** — your manual's program, explained line by line.
- **Program B** — the *same menu program* with real POSIX semaphores. It differs from A in only a few lines, so it is the easiest one to write in an exam.
- **Program C** — the textbook solution: two threads, three semaphores.

9.1 Big picture

A **producer** makes items and puts them into a buffer of fixed size. A **consumer** takes items out, oldest first. Three rules must hold:

1. If the buffer is **full**, the producer must wait.
2. If the buffer is **empty**, the consumer must wait.
3. The two must never change the buffer at the same moment, or it can be corrupted.

This is the classic *bounded-buffer problem*. Semaphores are the tool that enforces all three rules.

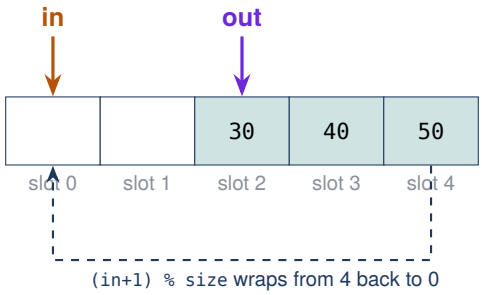
9.2 Concepts you need first

9.2.1 Race condition and critical section

If two threads change shared data without coordination, the result depends on timing: a **race condition**. The part of the code that touches the shared data is the **critical section**. Only one thread at a time may be inside it (*mutual exclusion*).

9.2.2 The circular buffer

The buffer is an array used as a ring. `in` is the slot where the next item will be put; `out` is the slot from which the next item will be taken. After each use the index moves forward with `(index + 1) % size`, which returns to 0 after the last slot (the *remainder* operator `%` makes the wrap-around).



Here a buffer of size 5 holds three items (30, 40, 50). The producer has written slots 0 to 4 and `in` has wrapped round to 0; the consumer has already taken slots 0 and 1, so `out` is 2.

9.2.3 Semaphores

A **semaphore** (Dijkstra, 1965) is an integer that can be changed only by two *atomic* (indivisible) operations:

Operation	Meaning
`wait` (also P, `sem_wait`)	if the value is greater than 0, subtract 1 and go on; if it is 0, **sleep** until somebody posts
`signal` (also V, `sem_post`)	add 1, and wake up one sleeping thread if there is one

A *counting* semaphore counts resources (free slots, filled slots). A *binary* semaphore has only the values 0 and 1 and works as a lock (*mutex*).

9.2.4 The three semaphores of the solution

Name	Starts at	Meaning
`empty`	N (buffer size)	number of **free** slots. The producer waits on it.
`full`	0	number of **filled** slots. The consumer waits on it.
`mutex`	1	lock around the buffer and the indices `in`, `out`.

Producer	Consumer
`wait(empty)` wait for a free slot	`wait(full)` wait for an item
`wait(mutex)` enter critical section	`wait(mutex)` enter critical section
put the item at `buffer[in]`	take the item from `buffer[out]`
`signal(mutex)` leave	`signal(mutex)` leave
`signal(full)` one more item	`signal(empty)` one more free slot

9.2.5 The POSIX functions

Function (header `semaphore.h`)	Meaning
`sem_init(&s, 0, value)`	set up semaphore `s` with a start value. The `0` means "shared between the threads of this process" (non-zero would mean between processes).
`sem_wait(&s)`	wait (P): subtract 1, or sleep while the value is 0
`sem_trywait(&s)`	like `sem_wait`, but if it would have to sleep it returns -1 at once (`errno` = `EAGAIN`)
`sem_post(&s)`	signal (V): add 1, wake a sleeper
`sem_destroy(&s)`	free the semaphore
`pthread_create(&t, NULL, f, arg)`	start a new thread that runs `f(arg)`
`pthread_join(t, NULL)`	wait until thread `t` ends (like `wait` for processes)

Compile with `gcc prog.c -o prog -pthread`. (The manual page of `sem_init` says "link with -pthread". Newer glibc versions merged the thread library into the main C library, but the flag is always correct.)

9.3 Program A: your manual's program

Lines	Job
1–4	headers
8	variables (capacity, menu choice, `in`, `out`, `item` counter, values)
9–11	read the buffer size and create the array
12–52	the menu loop (`do ... while`)
19–32	choice 1: produce
33–45	choice 2: consume
46–50	choice 3 (exit) and invalid choices

9.3.1 Line-by-line explanation

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <pthread.h>
5
6  int main()
7  {
8      int buffersize, choice=0, in=0, out=0, item=0, produce, consume;
9      printf("Enter the buffer size: ");
10     scanf("%d", &buffersize);
11     int buffer[buffersize];
```

Line 1 `stdio.h`: `printf`, `scanf`. This is the only header actually used.

Lines 2–4 `stdlib.h`, `unistd.h` and `pthread.h` are included but nothing from them is used. In particular no thread is ever created.

Line 8 One declaration, seven variables:

- `buffersize` — capacity N of the buffer.
- `choice=0` — the user's menu choice.
- `in=0` — index where the next produced item goes.
- `out=0` — index from which the next item is consumed.
- `item=0` — **how many items are in the buffer right now** (between 0 and `buffersize`). This counter does the job of the semaphores.
- `produce`, `consume` — the value typed in, and the value taken out.

Lines 9–10 Prompt and `scanf("%d", &buffersize);`

Line 11 `int buffer[buffersize];` creates the buffer: an array of `buffersize` integers (size chosen at run time).

```
12      do
13      {
14          printf("\n1. Produce\n2. Consume\n3. Exit\n");
15          printf("Enter the choice: ");
16          scanf("%d", &choice);
17          switch(choice)
18          {
19              case 1:
20                  if (item == buffersize)
21                  {
22                      printf("Buffer is full\n");
23                  }
24                  else
25                  {
26                      printf("Enter the value: ");
27                      scanf("%d", &produce);
28                      buffer[in] = produce;
29                      in = (in+1) % buffersize;
30                      item++;
31                  }
32                  break;
```

Lines 12–13 `do {` starts the menu loop. A `do ... while` loop always runs at least once, which suits a menu.

Lines 14–15 Print the three menu lines and the prompt.

Line 16 `scanf("%d", &choice);` reads the choice.

Lines 17–18 `switch(choice) {` jumps to the matching `case`.

Line 19 `case 1:` — produce.

Line 20 `if (item == buffersize)` — the buffer is full. In semaphore language: `empty` is 0, so `sem_wait(&empty)` would have to wait.

Lines 21–23 Print "Buffer is full". Nothing is produced.

Lines 24–25 `else {` — there is room.

Lines 26–27 Ask for a value and read it into `produce`.

Line 28 `buffer[in] = produce;` puts the item into the slot `in`.

Line 29 `in = (in+1) % buffersize;` moves `in` forward and wraps to 0 after the last slot.

Line 30 `item++;` one more item in the buffer (this is what `signal(full)` and `wait(empty)` do together).

Lines 31–32 Close the `else`; `break;` leaves the `switch`. Without `break` the program would fall through into `case 2`.

```
33              case 2:
34                  if (item == 0)
35                  {
36                      printf("Buffer is empty\n");
37                  }
38                  else
39                  {
40                      consume = buffer[out];
41                      printf("Consumed item is: %d\n", consume);
42                      out = (out+1) % buffersize;
43                      item--;
44                  }
45                  break;
46              case 3:
47                  printf("Exiting...\n");
48                  break;
49              default:
50                  printf("Invalid choice\n");
51          }
52      } while (choice != 3);
53      return 0;
54  }
```

Line 33 `case 2:` — consume.

Line 34 `if (item == 0)` — the buffer is empty. In semaphore language `full` is 0, so `sem_wait (&full)` would have to wait.

Lines 35–37 Print "Buffer is empty".

Lines 38–39 `else {` — there is at least one item.

Line 40 `consume = buffer[out];` takes the *oldest* item (first in, first out).

Line 41 Prints "Consumed item is" and the value.

Line 42 `out = (out+1) % buffersize;` moves `out` forward with wrap-around.

Line 43 `item--;` one item fewer.

Lines 44–45 Close the `else`; `break;`.

Lines 46–48 `case 3:` prints "Exiting..." and breaks.

Lines 49–50 `default:` any other number: "Invalid choice". It is the last label, so it needs no `break`.

Line 51 Closes the `switch`.

Line 52 `} while (choice != 3);` repeat the menu until the user chooses 3.

Lines 53–54 `return 0;` and the closing brace.

9.3.2 How the counter relates to semaphores

In Program A	Semaphore idea
`item == buffersize` ("Buffer is full")	`sem_wait(&empty)` would block: `empty` = 0
`item == 0` ("Buffer is empty")	`sem_wait(&full)` would block: `full` = 0

In Program A	Semaphore idea
`item`	the value of `full`
`buffersize - item`	the value of `empty`
`item++` after producing	`sem_wait(&empty)` then `sem_post(&full)`
`item--` after consuming	`sem_wait(&full)` then `sem_post(&empty)`
(nothing)	`mutex`: not needed, because only one thing happens at a time

A real semaphore would make the producer *sleep* when the buffer is full; the menu program cannot do that (the only thread would freeze), so it prints a message instead. That is why it is a *simulation*.

9.3.3 Dry run

Sample input from your manual: buffer size 2; produce 2; produce 3; produce (full); consume; consume; consume (empty); exit.

#	Action	`in`	`out`	`item`	buffer	Output
0	start	0	0	0	[,]	
1	choice 1, value 2	1	0	1	[2,]	
2	choice 1, value 3	0	0	2	[2, 3]	$(in = (1+1)\%2 = 0)$
3	choice 1	0	0	2	[2, 3]	Buffer is full
4	choice 2	0	1	1	[2, 3]	Consumed item is: 2
5	choice 2	0	0	0	[2, 3]	Consumed item is: 3
6	choice 2	0	0	0	[2, 3]	Buffer is empty
7	choice 3					Exiting...

Consumed values stay in memory but are ignored; `item` says what is valid. Items come out in the order they went in (FIFO), and `in` and `out` both wrapped round from 1 to 0.

9.3.4 The output, explained

Your recorded output shows exactly this sequence: `Buffer is full`, `Consumed item is: 2`, `Consumed item is: 3`, `Buffer is empty`, `Exiting`.... I ran your program with the same inputs and got the same lines.

Weak points of Program A — tested

- **No semaphores, no threads** (see the box at the start of the chapter).
- **Non-numeric input loops for ever.** When I typed `abc` at the menu, `scanf` failed and left `choice` unchanged and the letters unread, so the loop printed "Invalid choice" millions of times in two seconds. Program B checks `scanf`'s return value.
- The buffer size is not checked. A size of zero or less is not valid for an array.

9.4 Program B: the menu program with real semaphores

Only a few lines change. `item` disappears (the semaphores count for us), and `sem_trywait` replaces the full/empty tests. I use `sem_trywait` because a blocking `sem_wait` would freeze this single-

threaded program for ever.

```c
#include <stdio.h>
#include <semaphore.h>

int main(void)
{
    int size, choice = 0, in = 0, out = 0, value;
    sem_t empty, full;                      /* two counting semaphores */

    printf("Enter the buffer size: ");
    if (scanf("%d", &size) != 1 || size <= 0) {
        printf("Buffer size must be a positive number\n");
        return 1;
    }
    int buffer[size];
    sem_init(&empty, 0, size);              /* every slot is empty at the start */
    sem_init(&full, 0, 0);                  /* no slot is filled at the start   */

    do {
        printf("\n1. Produce\n2. Consume\n3. Exit\nEnter the choice: ");
        if (scanf("%d", &choice) != 1) break;
        switch (choice) {
        case 1:
            if (sem_trywait(&empty) == -1) {          /* no free slot: would block */
                printf("Buffer is full\n");
            } else {
                printf("Enter the value: ");
                if (scanf("%d", &value) != 1) value = 0;
                buffer[in] = value;
                in = (in + 1) % size;
                sem_post(&full);                      /* one more filled slot */
            }
            break;
        case 2:
            if (sem_trywait(&full) == -1) {           /* nothing to take: would block */
                printf("Buffer is empty\n");
            } else {
                value = buffer[out];
                out = (out + 1) % size;
                printf("Consumed item is: %d\n", value);
                sem_post(&empty);                     /* one more free slot */
            }
            break;
        case 3:
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice\n");
        }
    } while (choice != 3);

    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

Line 2 `semaphore.h` replaces `pthread.h`.

Line 7 `sem_t empty, full;` two counting semaphores. There is no `mutex` because there is only one thread.

Lines 10–13 The size is checked: it must be a number greater than 0.

Line 15 `sem_init(&empty, 0, size);` every slot is free at the start, so `empty` = size.

Line 16 `sem_init(&full, 0, 0);` nothing is stored yet, so `full` = 0.

Line 20 If `scanf` does not read a number, `break` leaves the loop (this prevents the endless loop of Program A).

Line 23 `sem_trywait(&empty) == -1` tries to take one free slot. If none is left it fails at once: "Buffer is full". If it succeeds, `empty` has gone down by 1.

Lines 26–29 Read the value, store it, advance `in` (same as Program A).

Line 30 `sem_post(&full);` announces one more filled slot.

Line 34 `sem_trywait(&full) == -1` tries to take one item. If there is none: "Buffer is empty".

Lines 37–39 Take the item, advance `out`, print it (same as Program A).

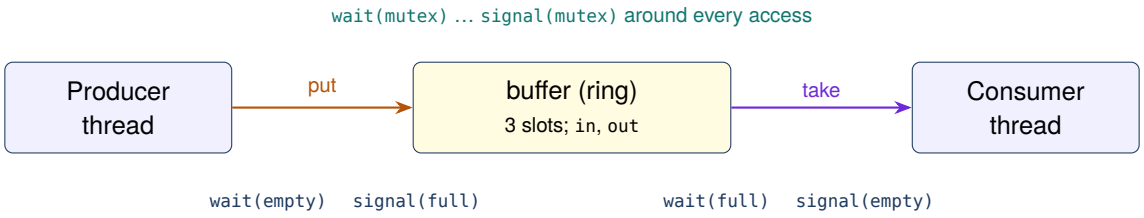
Line 40 `sem_post(&empty);` announces one more free slot.

Lines 51–52 `sem_destroy` releases the two semaphores.

I ran Program B with your sample input (2, then 1-2, 1-3, 1, 2, 2, 2, 3). It printed `Buffer is full`, `Consumed item is: 2`, `Consumed item is: 3`, `Buffer is empty`, `Exiting…`: the same behaviour as your manual, now driven by real semaphores.

9.5 Program C: two threads and three semaphores

This is the standard textbook solution, with a producer thread and a consumer thread running at the same time. The consumer sleeps 0.1 s after each item so that you can see the producer being forced to wait.




```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFSIZE 3
#define ITEMS   8

int buffer[BUFSIZE];
int in = 0, out = 0;

sem_t empty;      /* counts free slots,   starts at BUFSIZE */
sem_t full;       /* counts filled slots, starts at 0       */
sem_t mutex;      /* binary semaphore guarding buffer,in,out, starts at 1 */
```

Lines 1–4 `stdio.h` (`printf`), `pthread.h` (threads), `semaphore.h` (`sem_t` and its functions), `unistd.h` (`usleep`).

Lines 6–7 `#define` makes a named constant: the preprocessor replaces `BUFSIZE` by 3 and `ITEMS` by 8 everywhere. The buffer has 3 slots and 8 items will be produced.

Lines 9–10 The shared data: the ring `buffer` and the indices `in` and `out`. They are *global*, so both threads see the same variables (threads of one process share globals).

Lines 12–14 The three semaphores, also global: `empty` (free slots, will start at `BUFSIZE`), `full` (filled slots, starts at 0), `mutex` (lock, starts at 1).

```
16 void *producer(void *arg)
17 {
18     (void)arg;
19     for (int i = 1; i <= ITEMS; i++) {
20         sem_wait(&empty);                /* wait for a free slot      */
21         sem_wait(&mutex);                /* enter critical section    */
22         buffer[in] = i;
23         printf("Produced %d in slot %d\n", i, in);
24         in = (in + 1) % BUFSIZE;
25         sem_post(&mutex);                /* leave critical section    */
26         sem_post(&full);                 /* one more filled slot      */
27     }
28     return NULL;
29 }
```

Lines 16–17 `void *producer(void *arg)` — a thread function must take one `void *` parameter and return a `void *`. This is the code the producer thread runs.

Line 18 `(void)arg;` tells the compiler we deliberately ignore the argument.

Line 19 The loop produces the items $1, 2, \ldots, 8$.

Line 20 `sem_wait(&empty);` — take a free slot. If `empty` is above 0 it is decreased and the thread goes on; if it is 0 (buffer full) the thread **sleeps** until the consumer posts.

Line 21 `sem_wait(&mutex);` — enter the critical section; no other thread may touch the buffer now.

Line 22 `buffer[in] = i;` stores the item.

Line 23 Prints what was produced and where. It is inside the critical section, so the messages appear in the order the buffer really changed.

Line 24 `in = (in + 1) % BUFSIZE;` advances `in` with wrap-around.

Line 25 `sem_post(&mutex);` — leave the critical section.

Line 26 `sem_post(&full);` — one more filled slot; this wakes a consumer that is sleeping on `full`.

Lines 27–29 End of the loop; `return NULL;` ends the thread.

```
31 void *consumer(void *arg)
32 {
33     (void)arg;
34     for (int i = 1; i <= ITEMS; i++) {
35         sem_wait(&full);                 /* wait for a filled slot    */
36         sem_wait(&mutex);
37         int item = buffer[out];
38         printf("    Consumed %d from slot %d\n", item, out);
39         out = (out + 1) % BUFSIZE;
40         sem_post(&mutex);
41         sem_post(&empty);                /* one more free slot        */
```

```
42          usleep(100000);                 /* slow consumer: 0.1 second */
43      }
44      return NULL;
45  }
```

Lines 31–34 The consumer thread mirrors the producer: it takes `ITEMS` items.

Line 35 `sem_wait(&full);` — wait for a filled slot (sleep while the buffer is empty).

Line 36 `sem_wait(&mutex);` — enter the critical section.

Line 37 `int item = buffer[out];` take the oldest item.

Line 38 Print it (indented, so you can tell producer and consumer lines apart).

Line 39 `out = (out + 1) % BUFSIZE;` advance `out`.

Line 40 `sem_post(&mutex);` leave the critical section.

Line 41 `sem_post(&empty);` one more free slot; this wakes a producer that is sleeping on `empty`.

Line 42 `usleep(100000);` pause for 100 000 microseconds (0.1 s): a deliberately slow consumer, only for the demonstration.

Lines 43–45 End of loop, `return NULL;`, closing brace.

```
47  int main(void)
48  {
49      pthread_t p, c;
50      sem_init(&empty, 0, BUFSIZE);
51      sem_init(&full, 0, 0);
52      sem_init(&mutex, 0, 1);
53
54      pthread_create(&p, NULL, producer, NULL);
55      pthread_create(&c, NULL, consumer, NULL);
56      pthread_join(p, NULL);
57      pthread_join(c, NULL);
58
59      sem_destroy(&empty);
60      sem_destroy(&full);
61      sem_destroy(&mutex);
62      return 0;
63  }
```

Line 49 `pthread_t p, c;` variables that will hold the two thread identifiers.

Lines 50–52 `sem_init(&empty, 0, BUFSIZE);` `sem_init(&full, 0, 0);` `sem_init(&mutex, 0, 1);` Arguments: the semaphore, `0` = shared by threads of this process, and the start value.

Lines 54–55 `pthread_create(&p, NULL, producer, NULL);` starts a new thread. Arguments: where to store its id, default attributes (`NULL`), the function to run, and its argument (none). From here both threads run at the same time.

Lines 56–57 `pthread_join` makes `main` wait for each thread to finish. Without it, `main` would return at once and the whole program would end, killing the threads.

Lines 59–61 Destroy the semaphores.

Lines 62–63 `return 0;` and the closing brace.

9.5.1 How the semaphore values change

Buffer size 3, one produce followed by one consume, starting from the beginning:

Step	`empty`	`full`	`mutex`
start	3	0	1
producer: `sem_wait(&empty)`	2	0	1
producer: `sem_wait(&mutex)`	2	0	0
producer puts the item, then `sem_post(&mutex)`	2	0	1
producer: `sem_post(&full)`	2	1	1
consumer: `sem_wait(&full)`	2	0	1
consumer: `sem_wait(&mutex)`	2	0	0
consumer takes the item, then `sem_post(&mutex)`	2	0	1
consumer: `sem_post(&empty)`	3	0	1

9.5.2 Output of Program C (a real run)

```
Produced 1 in slot 0
Produced 2 in slot 1
Produced 3 in slot 2            <- the buffer is now full: empty = 0
    Consumed 1 from slot 0      <- consumer frees slot 0 ...
Produced 4 in slot 0            <- ... and the sleeping producer wakes up and reuses it
    Consumed 2 from slot 1
Produced 5 in slot 1
    Consumed 3 from slot 2
Produced 6 in slot 2
...
Produced 8 in slot 1
    Consumed 6 from slot 2
    Consumed 7 from slot 0
    Consumed 8 from slot 1
```

The producer filled all three slots and then slept at `sem_wait(&empty)` until the (slower) consumer freed a slot. The exact interleaving can differ from run to run, but two things are always true, and I checked both with a script: the items are consumed in the order they were produced $(1, \ldots, 8)$, and there are never more than 3 items in the buffer.

The order of the two waits matters — tested

In the producer the order is `sem_wait(&empty)` *then* `sem_wait(&mutex)`. Reverse them and the program can **deadlock**: with a full buffer the producer takes the mutex and then sleeps on `empty`; the consumer needs the mutex to remove an item, so it sleeps too; nobody can ever post. I swapped lines 20 and 21 and ran the program: it printed the first few lines and then froze; the `timeout` command had to kill it after 4 seconds. The `post` operations may be in either order, but the two `wait`s must be: resource semaphore first, mutex second.

Other points

- With exactly one producer and one consumer each index is touched by only one thread, so `mutex` is not strictly necessary here; it becomes essential with several producers or consumers.
- A *mutex* (`pthread_mutex_t`) must be unlocked by the thread that locked it. A binary semaphore has no owner: any thread may post it.
- `sem_wait` can return early with `EINTR` if a signal arrives. Production code retries.

- Do not forget `-pthread` when compiling.

Write it from memory: choose the version that fits the question

Menu program with semaphores (Program B): declare `sem_t empty, full;  sem_init (&empty,0,N); sem_init(&full,0,0);` produce: `if (sem_trywait(&empty)==-1) printf(" full"); else {put; in=(in+1)%N; sem_post(&full);}` consume: the mirror image with `full` and `empty` exchanged.
Threads (Program C): three globals `empty(N)`, `full(0)`, `mutex(1)`; producer: `wait(empty) wait (mutex) put post(mutex) post(full)`; consumer: `wait(full) wait(mutex) take post(mutex ) post(empty)`; `pthread_create` twice, `pthread_join` twice.
Memory aid: **the producer waits on EMPTY first and posts FULL last; the consumer does the opposite.**

Viva questions

Q. What is the producer–consumer (bounded-buffer) problem?
A. A producer puts items into a fixed-size buffer and a consumer takes them out. The producer must wait if the buffer is full, the consumer must wait if it is empty, and they must not access the buffer simultaneously.

Q. What is a semaphore?
A. An integer variable changed only by the atomic operations wait (P) and signal (V). `wait` decrements it or sleeps if it is 0; `signal` increments it and wakes a waiter.

Q. Counting versus binary semaphore?
A. A counting semaphore can hold any non-negative value and counts resources. A binary semaphore holds only 0 or 1 and is used as a lock.

Q. Why three semaphores?
A. `empty` blocks the producer when the buffer is full, `full` blocks the consumer when it is empty, and `mutex` gives mutual exclusion on the buffer.

Q. What are the starting values?
A. `empty` $= N$, `full` $= 0$, `mutex` $= 1$.

Q. What happens if the producer calls wait(mutex) before wait(empty)?
A. Possible deadlock: it can hold the mutex while sleeping on `empty`, and the consumer cannot get the mutex to free a slot.

Q. What is a race condition? What is a critical section?
A. A race condition is a result that depends on the timing of threads. A critical section is the code that accesses shared data and must run by one thread at a time.

Q. What do in and out mean, and why % buffersize?
A. `in` is the next slot to write, `out` the next slot to read. The remainder operator makes the index wrap to 0 after the last slot, so the array behaves as a ring.

Q. Your manual's program does not call sem_wait. How is it a simulation?
A. The counter `item` plays the role of the semaphore values: `item == buffersize` is the case where `empty` would block, and `item == 0` where `full` would block. Since there is only one thread, it prints a message instead of sleeping.

Q. What is sem_trywait?
A. A non-blocking `sem_wait`: it returns -1 immediately instead of sleeping when the value is 0.

Q. What does pthread_join do?

A. It waits for a thread to finish, like `wait` does for a child process.

Q. **Difference between a mutex and a binary semaphore?**

A. A mutex has an owner: only the thread that locked it may unlock it. A binary semaphore can be posted by any thread.

EXPERIMENT 10

Banker's algorithm (deadlock avoidance)

Aim from the lab manual

Banker's algorithm for deadlock avoidance. Write a C program to demonstrate Banker's deadlock avoidance.

Read this first: two things about your notebook program

1. It can loop for ever. The variable `found` is set to 0 only once, at the start of the program. If some processes can finish but the others never can (an *unsafe* state), `found` stays 1 and the test `if(found==0)` never fires, so the `while` loop runs for ever. I ran your program on 600 random inputs: in 52 of them it hung, and every one of those was an unsafe state reached after some progress. One extra line fixes it (line 41 below). With your notebook data, which is a safe state, the problem never shows, so your recorded output is correct.
2. It tests safety only. A complete Banker's algorithm has two parts: the *safety algorithm* (is the state safe?) and the *resource-request algorithm* (may this request be granted?). Your program does the first part. The second part is a typical exam extension, so it is explained and tested at the end of this chapter.

10.1 Big picture

Several processes share some resources. Before the system hands out anything, it asks one question: *if I say yes, can every process still finish?* The **Banker's algorithm** (Dijkstra) answers that question. It is called "Banker's" because a banker never lends so much money that he could not satisfy all his customers' agreed credit limits.

Your program reads the **Max** matrix (the most each process may ever ask for), the **Allocation** matrix (what each process holds now) and the **Available** vector (what is free). It computes the **Need** matrix and then searches for an order in which all processes can run to completion. If such an order exists, the state is **safe** and the order is printed.

10.2 Concepts you need first

10.2.1 Deadlock and the four conditions

A **deadlock** is a situation in which every process in a group waits for a resource that another process in the group holds, so nobody can ever continue. Deadlock needs all four of these conditions at the same time (the Coffman conditions):

Condition	Meaning
Mutual exclusion	a resource can be used by only one process at a time
Hold and wait	a process keeps the resources it has while it waits for more
No pre-emption	a resource is released only voluntarily by the process that holds it

Condition	Meaning
Circular wait	there is a cycle of processes, each waiting for a resource held by the next

Ways to deal with deadlock: *prevention* (make one of the four conditions impossible), *avoidance* (let all four conditions exist, but never enter a dangerous state; **this is Banker's algorithm**), *detection and recovery* (let it happen, find it, repair it), or *ignore it*.

10.2.2 Safe and unsafe states

- A state is **safe** if there is some order of the processes, a **safe sequence**, such that each process in turn can get everything it still needs from what is free plus what the earlier processes give back when they finish.
- Otherwise the state is **unsafe**.
- **A safe state can never lead to deadlock. An unsafe state *may* lead to deadlock.** So unsafe does *not* mean "deadlocked"; it means "the system can no longer promise to avoid it".

10.2.3 The data structures

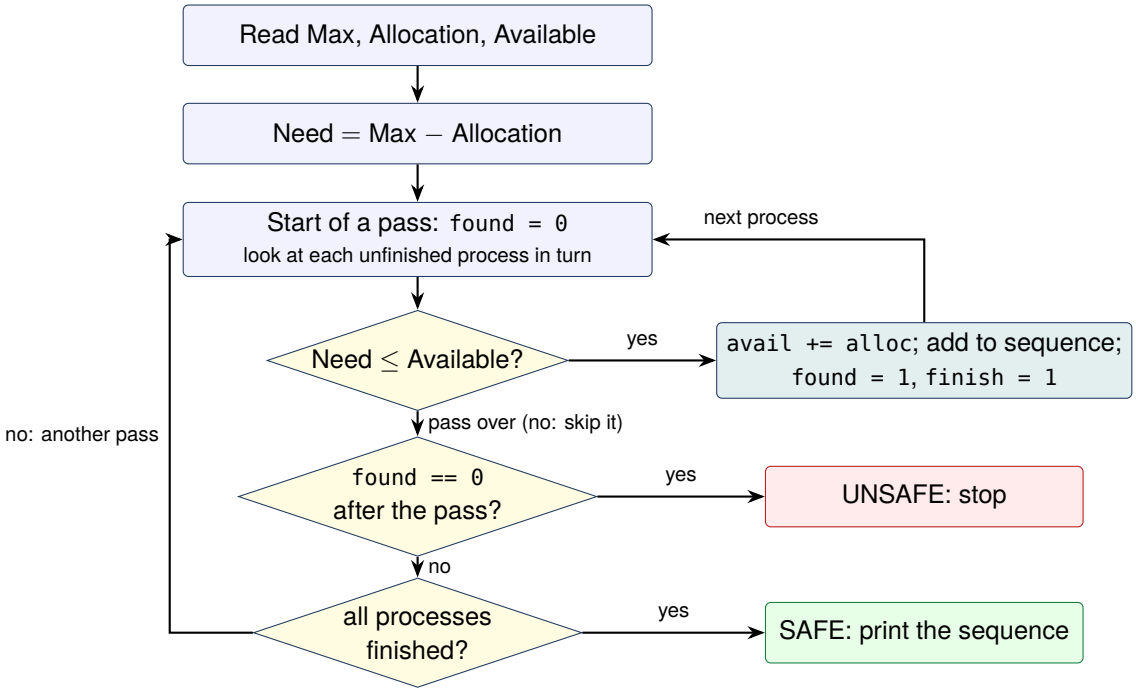
Let n be the number of processes and m the number of resource types. In your program n is `p` and m is `r`.

Name	Size	Meaning
`avail` (Available)	vector of m	`avail[j]` = number of free units of resource type j (a vector: one number per resource type; the prompt says "matrix")
`max` (Max)	$n \times m$	`max[i][j]` = the most that process i may ever request of type j
`alloc` (Allocation)	$n \times m$	`alloc[i][j]` = units of type j process i holds now
`need` (Need)	$n \times m$	`need[i][j]` = units it may still ask for: Need = Max − Allocation

10.2.4 The safety algorithm

1. Let *Work* = Available, and *Finish*$[i]$ = false for every process.
2. Find a process i with Finish$[i]$ = false and Need$_i \le$ Work (every resource type). If there is none, go to step 4.
3. Pretend process i runs to completion and returns its resources: Work = Work + Allocation$_i$, Finish$[i]$ = true. Go to step 2.
4. If Finish$[i]$ is true for *all* i, the state is **safe**; otherwise it is unsafe.

In your program `avail` plays the role of *Work* (it is changed in place), `finish[]` is *Finish*, `safeseq[]` records the order in which processes are found, and `count` says how many were found. The algorithm needs about $m \times n^2$ steps. The vector inequality $X \le Y$ means "$X[j] \le Y[j]$ for every j".



10.3 The program at a glance

Lines	Job
6–10	variables; read the number of processes `p` and resource types `r`
11–15	stop if either number is not positive (added check)
16–17	the arrays `max`, `alloc`, `avail`, `need`, `finish`, `safeseq`
18–30	read the Max matrix, the Allocation matrix and the Available vector
31–39	compute and print the Need matrix; clear the `finish` flags
40–61	**the safety algorithm**: passes over the processes until all are found or no progress is made
62–67	print "safe" and the safe sequence

10.4 Line-by-line explanation

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int p, r, count=0, found=0, i, j;
7      printf("Enter the number of processes:");
8      scanf("%d", &p);
9      printf("Enter the number of resources:");
10     scanf("%d", &r);
```

Line 1 `stdio.h` provides `printf` and `scanf`.

Line 2 `stdlib.h` is not used here; it does no harm.

Lines 4–5 `int main()` and the opening brace.

Line 6 Six integers:

- `p` — number of processes; `r` — number of resource *types*.
- `count=0` — how many processes have been placed in the safe sequence so far.
- `found=0` — a flag: "did the current pass find a process that can finish?"
- `i`, `j` — loop counters (`i` for processes, `j` for resource types).

Lines 7–8 Prompt (no new line) and `scanf("%d", &p);`. The `&` gives `scanf` the address of `p`.

Lines 9–10 The same for the number of resource types `r`.

```
11      if(p <= 0 || r <= 0)
12      {
13          printf("Number of processes and resources must be positive\n");
14          return 1;
15      }
16      int max[p][r], alloc[p][r], avail[r], need[p][r], finish[p],
17      safeseq[p];
```

Lines 11–15 `if(p <= 0 || r <= 0)` is an added safety check: the arrays below have a size that depends on `p` and `r`, and a size must be positive. If not, a message is printed and the program stops with status 1 (`||` means OR).

Lines 16–17 One declaration continued on a second line creates six arrays. Their sizes are chosen at run time (variable-length arrays, C99):

- `max[p][r]` — maximum demand of each process for each resource type.
- `alloc[p][r]` — what each process currently holds.
- `avail[r]` — free units of each resource type (one number per type).
- `need[p][r]` — what each process may still request.
- `finish[p]` — a 0/1 flag per process: 1 means "can finish; already counted".
- `safeseq[p]` — process numbers (starting from 0) in the order in which they can finish.

```
18      printf("Enter the MAX matrix:\n");
19      for(i=0; i<p; i++){
20          for(j=0; j<r; j++){
21              scanf("%d", &max[i][j]);
22          }
23      }
24      printf("Enter the Allocation matrix:\n");
25      for(i=0; i<p; i++)
26          for(j=0; j<r; j++)
27              scanf("%d", &alloc[i][j]);
28      printf("Enter the Available matrix:\n");
29      for(i=0; i<r; i++)
30          scanf("%d", &avail[i]);
```

Line 18 Prints the heading for the Max input.

Lines 19–23 Two nested loops read the Max matrix *row by row*: for process `i` the inner loop reads `r` numbers into `max[i][0]`...`max[i][r-1]`. The braces on lines 20, 22 and 23 close the inner and outer loop. You type p rows of r numbers.

Lines 24–27 The same for the Allocation matrix. The loops have no braces, because each `for` controls exactly one statement.

Lines 28–30 The Available vector: only `r` numbers, so only one loop (with `i` as the index).

```
31      printf("The Need matrix:\n");
32      for(i=0; i<p; i++){
33          for(j=0; j<r; j++){
34              need[i][j] = max[i][j] - alloc[i][j];
35              printf("%d\t", need[i][j]);
36          }
37          finish[i]=0;
38          printf("\n");
39      }
```

Line 31 Prints the heading "The Need matrix".

Line 32 Outer loop over the processes.

Lines 33–36 Inner loop over the resource types. Line 34 computes the Need: `need[i][j] = max[i][j] - alloc[i][j];` — what the process may still ask for equals its maximum minus what it already holds. Line 35 prints it followed by a tab (`\t`), so the numbers line up in columns.

Line 37 `finish[i]=0;` — at the start no process is finished.

Line 38 `printf("\n");` ends the printed row, so there is one row per process.

Line 39 Closes the outer loop. The Need matrix has now been computed and printed in the same loop.

```
40      while(count<p){
41          found = 0;   /* new pass: nobody has been found yet */
42          for(i=0; i<p; i++){
43              if(finish[i]==0){
44                  for(j=0; j<r; j++){
45                      if(need[i][j] > avail[j])
46                          break;
47                  }
48                  if(j==r){
49                      for(int k=0; k<r; k++)
50                          avail[k] += alloc[i][k];
51                      safeseq[count++] = i;
52                      found = 1;
53                      finish[i] = 1;
54                  }
55              }
56          }
57          if(found==0){
58              printf("System is in unsafe state\n");
59              return 0;
60          }
61      }
```

Line 40 `while(count<p){` — repeat *passes* until all `p` processes are in the safe sequence. A pass is one trip over all processes.

Line 41 `found = 0;` — **the fix.** At the start of every pass, assume that no process can finish. Without this line the flag keeps the value 1 from an earlier pass (see the gotchas).

Line 42 `for(i=0; i<p; i++){` tries the processes in order P1, P2, ...

Line 43 `if(finish[i]==0){` — skip processes that are already finished.

Lines 44–47 The test "is Need ≤ Available?" for one process: the loop compares `need[i][j]` with `avail[j]` for each resource type `j`. As soon as one type needs more than is available, `break` leaves the loop.

Line 48 `if(j==r){` — the clever part. If the loop above ended *without* `break`, then `j` has counted all the way up to `r`. That means *every* resource type was available, so process `i` can run to completion.

Lines 49–50 `avail[k] += alloc[i][k];` — pretend process `i` runs, finishes and *returns everything it holds*: the free resources grow by its allocation. (`k` is a new loop variable declared inside the `for`.)

Line 51 `safeseq[count++] = i;` — store `i` as the next process of the safe sequence, *then* increase `count` (post-increment: the old value of `count` is used as the index).

Line 52 `found = 1;` — this pass made progress.

Line 53 `finish[i] = 1;` — mark the process as finished.

Lines 54–56 Close the braces of `if(j==r)`, `if(finish[i]==0)` and the `for` over the processes. One pass is over.

Line 57 `if(found==0){` — after a whole pass: if *no* process could finish, then `avail` did not change during the pass, so no process will ever be able to finish.

Lines 58–59 Print "System is in unsafe state" and stop with `return 0;`.

Lines 60–61 Close the `if` and the `while`. The `while` tests `count<p` again: if it is still true, another pass starts.

```
62      printf("System is in safe state\n");
63      printf("Safe Sequence is:");
64      for(int k=0; k<p; k++)
65          printf(" P%d\t", safeseq[k]+1);
66      return 0;
67  }
```

Line 62 The `while` ended, so `count` reached `p`: every process could finish. Print "System is in safe state".

Line 63 Prints "Safe Sequence is:".

Lines 64–65 A loop prints each stored process number plus 1: `safeseq` holds numbers from 0, but people say P1, P2, ..., so `safeseq[k]+1` is printed with the format `" P%d\t"`.

Lines 66–67 `return 0;` and the closing brace of `main`.

10.5 Dry run

Input from your notebook: 5 processes, 4 resource types. Available $= (1, 5, 2, 0)$. The Need matrix printed by the program is

	Need			
P1	0	0	0	0
P2	0	7	5	0
P3	1	0	0	2
P4	0	0	2	0
P5	0	6	4	2

Pass	Process	Need	avail before	Need $\leq$ avail?	avail after
1	P1	0000	1520	yes	1532
1	P2	0750	1532	no $(7 > 5)$	1532
1	P3	1002	1532	yes	2886
1	P4	0020	2886	yes	214118
1	P5	0642	214118	yes	2141212
2	P1, P3, P4, P5	—	—	skipped (`finish` is 1)	—
2	P2	0750	2141212	yes	3141212

After pass 2, `count` is 5, so the `while` ends. The safe sequence is **P1, P3, P4, P5, P2**. Pass 1 found four processes; P2 had to wait until the others had returned their resources. *Check:* the final `avail` $= (3, 14, 12, 12)$ is the total number of units of each type (the free units plus everything that was allocated), as it must be when every process has given its resources back.

10.6 The output, explained

```
Enter the number of processes:5
Enter the number of resources:4
Enter the MAX matrix:            <- you type 5 rows of 4 numbers
Enter the Allocation matrix:     <- 5 more rows
Enter the Available matrix:      <- one row of 4 numbers
The Need matrix:                 <- computed by line 34, printed by line 35
0       0       0       0
0       7       5       0
1       0       0       2
0       0       2       0
0       6       4       2
System is in safe state          <- line 62
Safe Sequence is: P1     P3      P4      P5      P2      <- lines 63-65
```

I ran your program with exactly the data in your notebook: the Need matrix and the safe sequence P1 P3 P4 P5 P2 are as above. The published answer key for this textbook problem gives the same Need matrix and the safe sequence $\langle P_0, P_2, P_3, P_4, P_1 \rangle$, which is P1, P3, P4, P5, P2 when counted from 1 (Appendix D lists the source).

Check one number in your notebook

In the scanned output page, the third row of the Need matrix looks like `1 0 0 0`. For your data it is Max $-$ Allocation $= (2, 3, 5, 6) - (1, 3, 5, 4) = (1, 0, 0, \mathbf{2})$, and that is what the program prints (and what the published answer key shows). If your notebook really says `1 0 0 0`, it is a copying slip and worth correcting before you submit it.

10.7 Gotchas and the best method

1. The endless loop — tested

Input: 3 processes, 1 resource type. Max $= 2, 9, 9$; Allocation $= 1, 4, 4$; Available $= 1$. The Need values are $1, 5, 5$.

- Pass 1: P1 needs 1 and 1 is free, so it finishes; `avail` becomes 2 and `found` becomes 1. P2 and P3 each need 5, which is more than 2.
- Pass 2: nobody can finish. But `found` is still 1 from pass 1, so `if(found==0)` is false, `count` stays 1, and the `while` starts pass 3, 4, 5, ... for ever.

Your notebook version had to be killed by the `timeout` command (exit status 124). The corrected version prints `System is in unsafe state` and ends. If nobody can finish in the very first pass, both versions work (`found` is still 0). The three differences between your notebook program and `exp10.c` (the version in the lab manual) are exactly these:

```
@@ -10,2 +10,7 @@
     scanf("%d", &r);
+    if(p <= 0 || r <= 0)
+    {
+        printf("Number of processes and resources must be positive\n");
+        return 1;
+    }
     int max[p][r], alloc[p][r], avail[r], need[p][r], finish[p],
@@ -35,2 +40,3 @@
     while(count<p){
+        found = 0;   /* new pass: nobody has been found yet */
         for(i=0; i<p; i++){
@@ -51,3 +57,3 @@
         if(found==0){
-            printf("System is in unsafe state");
+            printf("System is in unsafe state\n");
             return 0;
```

The first block is the positive-size check, the second is the fix for the endless loop, and the third only adds a new line after the "unsafe" message.

2. Other points

- **Unsafe does not mean deadlocked.** The program says only that no safe order exists; a deadlock *may* follow.
- **Safety only.** A request from a process is not handled (see the twist below).
- `avail` is *changed* by the algorithm (it becomes "Work"). After the run it no longer holds the original Available vector.
- The data must be consistent: Allocation $\leq$ Max. If a process holds more than its Max, the Need is negative and the algorithm treats it as "can always run".
- The results of `scanf` are not checked, as in the other experiments.
- The prompt says "Available matrix", but Available is a vector (one number per resource type).

Twist: "Can this request be granted?" The resource-request algorithm (tested)

This is the second half of the Banker's algorithm. When process P_i requests the vector *Request*:

1. If Request $>$ Need$_i$, it is an error: the process has exceeded its maximum claim.
2. If Request $>$ Available, the process must *wait*: the resources are not free.

3. Otherwise *pretend* to grant it: Available$-=$ Request, Allocation$_i+=$ Request, Need$_i-=$ Request.
4. Run the safety algorithm. If the new state is safe, **grant** the request; if not, **restore the old state** and make the process wait.

The safety algorithm is moved into a function `is_safe` that works on a copy (`work`) of `avail`, so the real `avail` is not damaged and the pretend grant can be undone.

```c
#include <stdio.h>

/* Safety algorithm: returns 1 and fills seq[] (0-based) if the state is safe */
int is_safe(int p, int r, int alloc[p][r], int need[p][r], int avail[r], int seq[p])
{
    int work[r], finish[p], count = 0, found, i, j;
    for(j=0; j<r; j++)
        work[j] = avail[j];              /* work starts as a copy of avail */
    for(i=0; i<p; i++)
        finish[i] = 0;
    while(count < p){
        found = 0;
        for(i=0; i<p; i++){
            if(finish[i]==0){
                for(j=0; j<r; j++)
                    if(need[i][j] > work[j])
                        break;
                if(j==r){                /* process i can finish */
                    for(j=0; j<r; j++)
                        work[j] += alloc[i][j];
                    seq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if(found==0)
            return 0;                    /* nobody can finish: unsafe */
    }
    return 1;
}

int main()
{
    int p, r, i, j, q;
    printf("Enter the number of processes:");
    scanf("%d", &p);
    printf("Enter the number of resources:");
    scanf("%d", &r);
    if(p <= 0 || r <= 0)
    {
        printf("Number of processes and resources must be positive\n");
        return 1;
    }
    int max[p][r], alloc[p][r], need[p][r], avail[r], req[r], seq[p];
    printf("Enter the MAX matrix:\n");
    for(i=0; i<p; i++)
        for(j=0; j<r; j++)
            scanf("%d", &max[i][j]);
    printf("Enter the Allocation matrix:\n");
    for(i=0; i<p; i++)
        for(j=0; j<r; j++)
            scanf("%d", &alloc[i][j]);
```

```
    printf("Enter the Available matrix:\n");
    for(j=0; j<r; j++)
        scanf("%d", &avail[j]);
    for(i=0; i<p; i++)
        for(j=0; j<r; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    if(!is_safe(p, r, alloc, need, avail, seq))
    {
        printf("System is in unsafe state\n");
        return 0;
    }
    printf("System is in safe state\nSafe Sequence is:");
    for(i=0; i<p; i++)
        printf(" P%d", seq[i]+1);

    printf("\nEnter the process number (1 to %d) that makes a request:", p);
    scanf("%d", &q);
    q--;
    if(q < 0 || q >= p)
    {
        printf("Invalid process number\n");
        return 1;
    }
    printf("Enter its request:\n");
    for(j=0; j<r; j++)
        scanf("%d", &req[j]);

    for(j=0; j<r; j++)                  /* step 1: request <= need ? */
        if(req[j] > need[q][j])
        {
            printf("Error: the process has exceeded its maximum claim\n");
            return 1;
        }
    for(j=0; j<r; j++)                  /* step 2: request <= available ? */
        if(req[j] > avail[j])
        {
            printf("P%d must wait: the resources are not available\n", q+1);
            return 0;
        }
    for(j=0; j<r; j++)                  /* step 3: pretend to grant it */
    {
        avail[j]    -= req[j];
        alloc[q][j] += req[j];
        need[q][j]  -= req[j];
    }
    if(is_safe(p, r, alloc, need, avail, seq))
    {
        printf("Request can be granted. Safe Sequence is:");
        for(i=0; i<p; i++)
            printf(" P%d", seq[i]+1);
        printf("\n");
    }
    else
    {
        for(j=0; j<r; j++)              /* undo the pretend grant */
        {
            avail[j]    += req[j];
            alloc[q][j] -= req[j];
            need[q][j]  += req[j];
        }
```

```
115         printf("Request cannot be granted: the system would become unsafe\n");
116     }
117     return 0;
118 }
```

Results I obtained (processes numbered from 1):

State	Request	Result
your notebook data (Available 1 5 2 0)	P2 asks $(0, 4, 2, 0)$	can be granted; safe sequence P1 P3 P4 P5 P2
your notebook data	P3 asks $(0, 0, 2, 0)$	error: it exceeds its maximum claim (its Need is 1 0 0 2)
your notebook data	P5 asks $(0, 3, 3, 0)$	must wait: only 2 units of the third type are free
textbook snapshot with 3 resource types (Available 3 3 2)	P2 asks $(1, 0, 2)$	can be granted; safe sequence P2 P4 P5 P1 P3
the same, after that grant (Available 2 3 0)	P5 asks $(3, 3, 0)$	must wait: not enough free
the same state	P1 asks $(0, 2, 0)$	refused: the resulting state would be unsafe

The first row is the classic exam question for your data ("can the request of P1 for $(0, 4, 2, 0)$ be granted immediately?") and the textbook answer is yes. The last three rows match the textbook discussion of the second snapshot. I also compared the program with an independent reference on 800 random cases (every decision matched), and I compared the corrected `exp10.c` on 1 000 random inputs: no hang and no mismatch in the Need matrix, the verdict or the safe sequence.

Write it from memory

1. Read `p`, `r`; declare `max[p][r], alloc[p][r], avail[r], need[p][r], finish[p], safeseq[p]`; read Max, Allocation, Available.
2. `need[i][j] = max[i][j] - alloc[i][j];` and `finish[i] = 0;`
3. `while (count < p) { found = 0; for each unfinished i: if need[i][j] <= avail[j] for all j then { avail += alloc[i]; safeseq[count++] = i; finish[i] = 1; found = 1; } if (!found) { unsafe; return; } }`
4. Print "safe" and `safeseq[k]+1`.

Remember: **Need = Max − Allocation; a process can run if Need ≤ Available; when it finishes it gives its Allocation back; reset `found` at the start of every pass.**

Viva questions

Q. What is a deadlock? Name the four necessary conditions.
A. Processes waiting for each other for ever. Mutual exclusion, hold and wait, no pre-emption, circular wait.

Q. Difference between deadlock prevention, avoidance and detection?
A. Prevention makes one of the four conditions impossible. Avoidance (Banker's algorithm) allows all conditions but never enters an unsafe state. Detection lets deadlock happen, finds it and recovers.

Q. What is a safe state? What is a safe sequence?
A. A state in which all processes can finish in some order. That order is the safe sequence: each

process can get its remaining need from the free resources plus what earlier processes return.

Q. Is an unsafe state the same as a deadlock?
A. No. A deadlock is an unsafe state, but an unsafe state only *may* lead to deadlock. A safe state never does.

Q. Why is it called the Banker's algorithm?
A. A banker lends money only if he can still satisfy all customers' credit limits; the OS grants resources only if all processes can still finish.

Q. Name the data structures.
A. Available (vector of m), Max, Allocation and Need (each $n \times m$). Need $=$ Max $-$ Allocation.

Q. Describe the safety algorithm.
A. Work $=$ Available; find an unfinished process with Need $\leq$ Work; add its Allocation to Work and mark it finished; repeat. If all can be finished the state is safe.

Q. What does if(j==r) mean in the program?
A. The inner loop ended without `break`, so the process's Need was $\leq$ Available for every resource type: the process can run to completion.

Q. Why must found be reset in every pass?
A. Otherwise it stays 1 after the first progress, an unsafe state is never detected, and the `while` loop runs for ever.

Q. What is the time complexity?
A. About $m \times n^2$ operations (n processes, m resource types): up to n passes, each looking at up to n processes and m resource types.

Q. What is the resource-request algorithm?
A. Check Request $\leq$ Need and Request $\leq$ Available; pretend to allocate; run the safety algorithm; grant only if the state stays safe, otherwise restore the old state and let the process wait.

Q. What are the limitations of the Banker's algorithm?
A. Every process must declare its maximum demand in advance, the number of processes and resources must be fixed, and processes must return resources in finite time. It also costs time on every request.

APPENDIX A

Quick reference: functions and system calls

Name	Header	A call looks like	Returns
`open`	`fcntl.h`	`open(path, flags, mode)`	descriptor ≥ 0; -1 on error. `mode` only with `O_CREAT`
`read`	`unistd.h`	`read(fd, buf, count)`	bytes read; **0 at end of file**; -1 on error
`write`	`unistd.h`	`write(fd, buf, count)`	bytes written; -1 on error
`close`	`unistd.h`	`close(fd)`	0; -1 on error
`access`	`unistd.h`	`access(path, R_OK)`	0 if allowed; -1 otherwise. Modes: `F_OK R_OK W_OK X_OK`
`fork`	`unistd.h`	`fork()`	child's PID in the parent; **0 in the child**; -1 on failure
`execl`	`unistd.h`	`execl(path, arg0, ..., (char *)NULL)`	**never returns on success**; -1 on error
`wait`	`sys/wait.h`	`wait(&status)`	PID of the child that ended; -1 if no child is left
`waitpid`	`sys/wait.h`	`waitpid(pid, &status, 0)`	PID of the child; -1 on error
`getpid` `getppid`	`unistd.h`	`getpid()`	own PID / parent's PID
`exit`	`stdlib.h`	`exit(status)`	never returns; flushes stdio buffers
`_exit`	`unistd.h`	`_exit(status)`	never returns; does *not* flush stdio
`signal`	`signal.h`	`signal(SIGINT, handler)`	the previous handler; `SIG_ERR` on error
`sigaction`	`signal.h`	`sigaction(sig, &act, &old)`	0; -1 on error
`kill`	`signal.h`	`kill(pid, SIGTERM)`	0; -1 on error
`alarm`	`unistd.h`	`alarm(20)`	seconds left on the previous alarm (0 if none)
`sleep`	`unistd.h`	`sleep(5)`	0; or the seconds left if a signal handler interrupted it
`perror`	`stdio.h`	`perror("text")`	nothing; prints `text: reason` on standard error
`fflush`	`stdio.h`	`fflush(stdout)`	0; `EOF` on error
`scanf`	`stdio.h`	`scanf("%d", &n)`	number of items read; `EOF` at end of input
`sem_init`	`semaphore.h`	`sem_init(&s, 0, value)`	0; -1 on error
`sem_wait`	`semaphore.h`	`sem_wait(&s)`	0 after decrementing (sleeps while the value is 0)
`sem_trywait`	`semaphore.h`	`sem_trywait(&s)`	0, or -1 at once (`EAGAIN`) if it would sleep
`sem_post`	`semaphore.h`	`sem_post(&s)`	0; increments, wakes one sleeper
`sem_destroy`	`semaphore.h`	`sem_destroy(&s)`	0; -1 on error

Name	Header	A call looks like	Returns
`pthread_create`	`pthread.h`	`pthread_create(&t, NULL, f, arg)`	0 on success, an error number otherwise
`pthread_join`	`pthread.h`	`pthread_join(t, NULL)`	0 on success

APPENDIX B

Scheduling cheat sheet

B.1 Formulas

Learn these five lines

- $\text{TAT} = \text{CT} - \text{AT}$ $\quad$ $\text{WT} = \text{TAT} - \text{BT}$ $\quad$ (all arrival times 0: TAT = CT, WT = start time)
- **FCFS:** $\text{WT}_1 = 0$, $\quad \text{WT}_i = \text{WT}_{i-1} + \text{BT}_{i-1}$
- **FCFS with arrival times:** $\text{CT}_i = \max(\text{CT}_{i-1}, \text{AT}_i) + \text{BT}_i$
- **SJF:** sort by burst time (ties: earlier arrival), then use the FCFS formulas
- **Round Robin:** each turn runs $\min(q, \text{remaining})$; when a process finishes, CT = clock
- **Average** = total / n (use `double` or a `(float)` cast)

B.2 Comparison

	FCFS	**SJF**	**SRTF**	**Round Robin**
Pre-emptive?	no	no	yes	yes
Starvation?	no	possible	possible	no
Average waiting time	can be long (convoy effect)	smallest for jobs ready together	smallest in the examples here	in between; depends on q
Needs burst times in advance?	no	yes (estimated)	yes (estimated)	no
Response time	poor	poor for long jobs	good for short jobs	good
Context switches	few	few	many	many; more as q shrinks
Ready queue	FIFO	ordered by burst	ordered by remaining time	circular

B.3 Worked examples (all produced by the programs in this guide)

Jobs (arrival, burst)	Algorithm	Avg WT	Avg TAT
bursts 24, 3, 3 (all arrive at 0)	FCFS	17.00	27.00
bursts 3, 3, 24	FCFS	3.00	13.00
bursts 6, 8, 7, 3	FCFS	10.25	16.25
bursts 6, 8, 7, 3	SJF	7.00	13.00
bursts 24, 3, 3, quantum 4	Round Robin	5.67	15.67
bursts 3, 4, 2, 5, quantum 2 (your manual)	Round Robin	6.50	10.00
(0,5) (1,3) (2,8) (3,6)	FCFS	5.75	11.25
(0,8) (1,4) (2,9) (3,5)	SJF non-pre-emptive	7.75	14.25
(0,8) (1,4) (2,9) (3,5)	SRTF	6.50	13.00
(0,7) (2,4) (4,1) (5,4)	SJF non-pre-emptive	4.00	8.00
(0,7) (2,4) (4,1) (5,4)	SRTF	3.00	7.00
(0,5) (1,3) (2,1) (3,2) (4,3), quantum 3	Round Robin	4.60	7.40

APPENDIX C

Commands and common errors

C.1 Commands you will use

Command	What it does
`gcc -Wall -Wextra -g main.c -o main`	compile with warnings and debug information
`gcc main.c -o main -pthread`	compile a program that uses threads or semaphores
`./main`	run the program
`echo $?`	show the exit status of the last command
`./main < input.txt`	take the program's input from a file instead of the keyboard
`printf '4\n3\n4\n2\n5\n' | ./main`	feed several input lines from the shell (fine for Experiments 6–9; not for Experiment 4)
`ls -l /bin/sh`	shows which shell `sh` really is (dash or bash)
`ps -f`	list your processes with PID and PPID
`kill -SIGUSR1 PID`	send a signal to a process (see Experiment 5)
`man 2 fork`, `man 3 printf`, `man 7 signal`	read the manual page: section 2 system calls, 3 library functions, 7 overviews
`strace ./main`	if installed: list the system calls a program makes

C.2 Common errors

Message or symptom	Meaning and fix
`implicit declaration of function 'fork'`	a header is missing; for `fork`, `read`, `write`, `sleep`, `alarm` add `#include <unistd.h>`
`undefined reference to 'sem_init'` or `'pthread_create'`	add `-pthread` to the `gcc` command
`Segmentation fault`	illegal memory access: a missing `&` in `scanf`, an array that is too small, or an uninitialised pointer
`a.out: command not found`	type `./a.out` (the current folder is not searched)
`No such file or directory`	wrong name or wrong folder: check with `pwd` and `ls`
`Permission denied`	you lack the permission: look at `ls -l` and `chmod`
the program seems frozen	it may be waiting for input (Experiment 4 needs Ctrl+D), stuck in an endless loop (Ctrl+C), or in a deadlock (Experiment 9)
`warning: ignoring return value of 'scanf'`	harmless here; serious programs check the return value
output order differs from the manual (Experiments 3 and 9)	normal: with several processes or threads the scheduler decides the order

APPENDIX D

How this guide was checked, and sources

D.1 What was tested

- **Environment:** Ubuntu 24.04 (Linux x86-64), gcc 13.3, `/bin/sh` = dash 0.5.12. Every program was compiled with `-Wall -Wextra` and compiled without errors.
- **Your ten programs** were re-typed with the same line numbers as your manual (only indentation was added) and run with the inputs from your manual. Experiment 10 was typed from your handwritten notebook pages. The outputs match. The interactive ones (Experiments 1, 2, 4) were driven through a pseudo-terminal, so that the keyboard behaviour, including Ctrl+D, was real.
- **Signals** (Experiment 5): the program was run in the background and sent `SIGINT`, `SIGHUP`, `SIGUSR1`, `SIGALRM` and `SIGTERM`; it printed the lines in your manual and ended after `SIGTERM`. It was also run for 22 seconds with no signal to watch `alarm(20)` fire.
- **Scheduling** (Experiments 6–8 and the four "twist" programs): each program was compared with an independent simulator written separately in Python on 400 random inputs per program (2 800 cases, no mismatch), and against textbook examples (Appendix B).
- **Other experiments in the text:** the 32-process explosion without `exit(0)`, the duplicated lines without `fflush`, Ctrl+D in the middle of a line, the empty file created when `scanf` and `read` are mixed on a pipe, the deadlock caused by swapped `sem_wait` calls, and the endless menu loop on bad input were each reproduced.
- **Banker's algorithm** (Experiment 10): the notebook program was run on your data (safe sequence P1 P3 P4 P5 P2, as in your notebook) and on 600 random inputs, where it hung in 52 unsafe cases. The corrected program was compared with an independent Python reference on 1 000 random inputs (Need matrix, verdict and safe sequence: no mismatch, no hang). The request-algorithm program was checked on two textbook snapshots and on 800 random cases (no mismatch).
- **Likely exam variations** (Appendix E) were each compiled and run; the multi-producer program was run five times with a script that checks the results.
- **Improved versions** (`exp1_better`, `exp1_fork`, `exp2_better`, `exp3_status`, `exp4_better`, `exp5_better`, Programs B and C of Experiment 9, and the four scheduling twists) were all compiled and run.

D.2 Limits

- Everything was run on one machine. Your lab PC may have a different shell, compiler or kernel. If something differs, check `ls -l /bin/sh` and `gcc --version` first.
- The order of output lines in Experiments 3 and 9C depends on the scheduler and will differ from run to run.
- The tool `strace` was not available on the test machine, so no `strace` output is shown.
- I could not see your college's own lab environment or marking scheme. Follow your teacher's conventions where they differ (for example the Round Robin tie-break rule).

D.3 Sources checked on the internet

- `exec(3)` — https://man7.org/linux/man-pages/man3/execle.3@@man-pages.html
- `fork(2)` — https://man7.org/linux/man-pages/man2/fork.2.html
- `wait(2)` — https://linux.die.net/man/2/wait
- `open(2)` — https://man7.org/linux/man-pages/man2/openat.2.html
- `signal(2)` — https://man7.org/linux/man-pages/man2/signal.2.html
- `signal-safety(7)` — https://www.man7.org/linux/man-pages/man7/signal-safety.7.html
- `alarm(2)` — https://www.man7.org/linux/man-pages/man2/alarm.2.html
- `sleep(3)` — https://linux.die.net/man/3/sleep
- terminal driver and the end-of-file character (`termios`) — https://linux.die.net/man/3/tcgetattr
- `sem_init(3)`, semaphores — https://linux.die.net/man/3/sem_init
- `scanf` field width and the string terminator — https://en.cppreference.com/c/io/scanf
- `read -p` is not specified by POSIX — https://lists.llvm.org/pipermail/llvm-dev/2008-May/014316.html
- writing `sh` scripts without bashisms (`printf` before `read`) — https://dev.to/bowmanjd/writing-bash-scripts-that-are-not-only-bash-checking-for-bashisms-and-testing-with-dash-1bli
- ShellCheck wiki, portability of `#!/bin/sh` scripts — https://shellcheck.net/wiki/SC2039
- CPU scheduling: FCFS, SJF, SRTF, Round Robin, exponential averaging — https://www.cs.colostate.edu/~cs370/Fall23/lectures/CS370-L15-CPU-Scheduling-PartB.pdf, https://www.cse.iitm.ac.in/~vplab/courses/OS/Ch-5.pdf, https://www.inf.ed.ac.uk/teaching/courses/os/slides/08-sched16.pdf
- bounded buffer with semaphores (`mutex`, `empty`, `full`) — https://cs.gordon.edu/courses/cs322/lectures/transparencies/bounded_buffer.html, https://nob.cs.ucdavis.edu/classes/ecs150-2022-02/handouts/sync-pcsem.html
- deadlock, its four conditions, safe and unsafe states — https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/7_Deadlocks, https://www.cs.sjtu.edu.cn/~wuct/os/slides/lec8-OS.pdf
- Banker's algorithm: data structures, safety algorithm and its cost ($m \times n^2$ steps) — https://www.cs.umd.edu/~hollings/cs412/s16/lectures/lect10.pdf, https://uomosul.edu.iq/en/computerscience/wp-content/uploads/sites/11/2025/05/lec10.pdf
- the data set of your notebook (a standard textbook exercise) with the published answer: Need matrix, safe sequence $\langle P_0, P_2, P_3, P_4, P_1 \rangle$, and the request of P1 for $(0, 4, 2, 0)$ can be granted — https://old.mu.ac.in/wp-content/uploads/2017/11/QP-code-40533-OS-SEM-IV-CBCGS-answer-model-key.pdf
- the second textbook snapshot and its request examples ($(1, 0, 2)$ granted, $(3, 3, 0)$ must wait, $(0, 2, 0)$ unsafe) — https://www.cs.csustan.edu/~john/Classes/CS3750/Examples/bankersAlgEg.html

APPENDIX E

Likely exam variations (tested)

An examiner may change a program a little. The changes below are **my guesses** of typical ones; I have not seen your examiner's questions. Each variation was compiled and run before I added it. The programs are in the folder `code/variations` of the source zip, and the file `CHANGES.txt` there contains the diffs shown in this appendix. In a diff, a line that starts with `-` was removed, a line that starts with `+` was added, and the other lines are unchanged context.

Exp.	Possible change	What changes	How I checked it
1	run the script with bash, or find `sh` through the PATH	one line: `execl("/bin/bash", "bash", ...)` or `execlp("sh", "sh", ...)`	`largest.sh` gave the right answer with both
2	count the lines of the file	a counter, a loop over the bytes just read that looks for `'\n'`, and one `printf`	a 3-line file printed "Number of lines: 3"
3	the user types the number of children	a new variable `n` used in both loops	input 3 created 3 children
4	append instead of overwrite	`O_TRUNC` becomes `O_APPEND`	two runs: the file held both lines
5	handle one more signal	one more `if` in the handler and one more `signal()` line	`kill -USR2` printed the new message
6	print a Gantt chart	six lines after the table loop	same chart as in your manual
7	priority scheduling	a `pr` field, read it, and sort by `pr` instead of `bt`	textbook example: order P2 P5 P1 P3 P4, average wait 8.2
7	longest job first	one comparison: `<` becomes `>`	bursts 6, 7, 2 gave the order P2, P1, P3
8	print every time slice	remember the start time, print after each slice	same slices as your manual's chart
9	several producers and consumers	each thread gets an id; same three semaphores	2+2 threads, 5 runs: every item consumed once, never more than 3 in the buffer
10	can this request be granted?	the resource-request algorithm: check the request, pretend to grant it, run the safety test, undo it if unsafe	six requests on two textbook snapshots and 800 random cases (Chapter 10)

Experiment 1: run the script with bash

Only the `execl` line changes. (For `execlp`, write `execlp("sh", "sh", sname, (char*)NULL);` it searches the PATH for `sh`.)

```
@@ -13,3 +13,3 @@
     }
-    if(execl("/bin/sh", "sh", sname, (char*)NULL) == -1)
+    if(execl("/bin/bash", "bash", sname, (char*)NULL) == -1)
         perror("Error: cannot execute script");
```

Experiment 2: count the lines

A counter `lines`, a loop over the `n` bytes that were just read, and a final `printf`.

```
@@ -9,3 +9,4 @@
     int fd;
-    ssize_t n;
+    ssize_t n, k;
+    long lines = 0;
     printf("Enter filename: ");
@@ -24,2 +25,5 @@
     {
+        for (k = 0; k < n; k++)
+            if (buffer[k] == '\n')
+                lines++;
         if (write(STDOUT_FILENO, buffer, n) != n)
@@ -33,2 +37,3 @@
     close(fd);
+    printf("Number of lines: %ld\n", lines);
     return 0;
```

Experiment 3: the number of children is typed by the user

Replace the two 5s by `n`. The `wait` loop must use `n` too.

```
@@ -7,5 +7,7 @@
 {
-    int i;
+    int i, n;
     pid_t pid;
-    for(i=1; i<=5; i++){
+    printf("How many children? ");
+    scanf("%d", &n);
+    for(i=1; i<=n; i++){
         /* flush so the child does not copy unflushed output */
@@ -26,3 +28,3 @@
     }
-    for(i=1; i<=5; i++)
+    for(i=1; i<=n; i++)
     {
```

Experiment 4: append instead of overwrite

One flag changes. The mode `0644` is still needed because of `O_CREAT`.

```
@@ -11,3 +11,3 @@
     scanf("%99s", fname);
-    fd = open(fname, O_CREAT | O_WRONLY | O_TRUNC, 0644);
+    fd = open(fname, O_CREAT | O_WRONLY | O_APPEND, 0644);
     if (fd == -1)
```

Experiment 5: one more signal

Two additions: a branch in the handler and a registration in `main`. Test with `kill -SIGUSR2 PID`.

```
@@ -23,2 +23,6 @@
     }
+    if(sig == SIGUSR2)
+    {
+        printf("Caught a Second User Signal\n");
+    }
     if(sig == SIGALRM)
```

```
@@ -36,2 +40,3 @@
     signal(SIGUSR1, signalhandler);
+    signal(SIGUSR2, signalhandler);
     alarm(20);
```

Experiment 6: print a Gantt chart

The chart is printed after the table. Each process is a 5-character box; the times are printed right-aligned under the box borders (they line up for one-digit times).

```
@@ -41,2 +41,8 @@
     }
+    printf("\n\nGantt chart:\n");
+    for(i=0; i<n; i++)
+        printf("| P%d ", P[i].pid);
+    printf("|\n0");
+    for(i=0; i<n; i++)
+        printf("%5d", P[i].tat);
     for(i=0; i<n; i++)
```

Experiment 7: longest job first

One character changes in the sort test.

```
@@ -34,3 +34,3 @@
             /* equal burst times: keep arrival (pid) order */
-            if(p[j].bt < p[min].bt ||
+            if(p[j].bt > p[min].bt ||
                (p[j].bt == p[min].bt && p[j].pid < p[min].pid))
```

Experiment 7: priority scheduling (lower number = higher priority)

Add the field `pr`, read burst time and priority together, and sort by `pr`. Equal priorities keep arrival order, as before. Input for the textbook example: burst and priority pairs `10 3, 1 1, 2 4, 1 5, 5 2`.

```
@@ -7,3 +7,3 @@
 {
-    int pid, bt, wt, tat;
+    int pid, bt, pr, wt, tat;
 };
@@ -25,4 +25,4 @@
         p[i].pid = i+1;
-        printf("Enter burst time for p%d : ", i+1);
-        scanf("%d", &p[i].bt);
+        printf("Enter burst time and priority for p%d : ", i+1);
+        scanf("%d %d", &p[i].bt, &p[i].pr);
     }
@@ -34,4 +34,4 @@
             /* equal burst times: keep arrival (pid) order */
-            if(p[j].bt < p[min].bt ||
-               (p[j].bt == p[min].bt && p[j].pid < p[min].pid))
+            if(p[j].pr < p[min].pr ||
+               (p[j].pr == p[min].pr && p[j].pid < p[min].pid))
             {
@@ -51,7 +51,7 @@
     }
-    printf("\nProcess BT\t WT\t TAT");
+    printf("\nProcess BT\t PR\t WT\t TAT");
     for(int i=0; i<n; i++)
     {
-        printf("\n%d\t %d\t %d\t %d", p[i].pid, p[i].bt,
```

```
-               p[i].wt, p[i].tat);
+       printf("\n%d\t %d\t %d\t %d\t %d", p[i].pid, p[i].bt,
+               p[i].pr, p[i].wt, p[i].tat);
     }
```

Experiment 8: print every time slice

Remember the clock before the slice, print after it. The output matches the Gantt chart of your manual.

```
@@ -42,2 +42,3 @@
                 flag=0;
+                int start = time;
                 if(remBT[i] > tq)
@@ -46,2 +47,3 @@
                     remBT[i] = remBT[i]-tq;
+                    printf("P%d: %d-%d\n", p[i].pid, start, time);
                 } else {
@@ -51,2 +53,3 @@
                     p[i].wt = p[i].tt - p[i].bt;
+                    printf("P%d: %d-%d\n", p[i].pid, start, time);
                 }
```

Experiment 9: several producers and consumers

The file `code/variations/v9_multi.c` starts two producers and two consumers. Each thread receives an id (`int id = *(int *)arg;`); the producers make the numbers 101–104 and 201–204, and every consumer takes four items. The three semaphores and the order of the waits stay exactly as in Program C of Chapter 9. With more than one producer or consumer, the `mutex` semaphore is *essential*: without it two threads could write the same slot. I ran it five times and checked that every item was consumed exactly once and that the buffer never held more than three items.

Experiment 10: can this request be granted?

The complete program is printed in Chapter 10 (it is also the file `code/exp10_request.c`). The safety test of your program is moved into a function `is_safe` that works on a copy of Available; the new code in `main` reads the process number and its request, checks $Request \leq Need$ and $Request \leq Available$, pretends to grant the request, calls `is_safe`, and undoes the pretend grant if the result is unsafe.

More practice changes: one for every experiment

Here are nine more changes of the kind an examiner might ask for. Each one was compiled and run; the files are `code/variations/drill_exp*.c` and the diffs are listed below (also in `code/variations/DRILLS.txt`).

Exp.	The change	What to add	Result I got
2	also print the total number of bytes read	a counter `total`; `total += n;` after each `read`; one `printf` at the end	a 12-byte file printed 12
3	each child sleeps `i` seconds before printing	`sleep(i);` at the start of the child branch	the Child lines came out in the order 1 to 5 (the run took 5 s)
4	report how many bytes were written	a counter `total`; `total += n;` after `write`; change the final message	`hello world` and Enter gave 12 bytes

Exp.	The change	What to add	Result I got
5	end the program after the third `SIGUSR1`	`static int count = 0;` in the `SIGUSR1` branch and `if(++count == 3) exit(0);`	the process ended after the third `kill`
6	print the process with the longest waiting time	a variable `worst`; one `if` in the totals loop; one `printf`	bursts 3 4 2 5 gave P4 (9)
7	print the Gantt chart of SJF	six lines before the totals loop (the turn-around time is the completion time)	bursts 6 7 2 gave `| P3	P1	P2 |` and times 0 2 8 15
8	print how many turns each process got	an array `turns[]` and `turns[i]++;` for every turn	bursts 3 4 2 5, quantum 2: 2, 2, 1, 3 turns
9	show the buffer after every operation	a loop from `out` over `item` entries, after `item++` and `item--`	the sample session showed 2, then 2 3, then 3, then an empty buffer
10	print the total units of every resource type	after the run `avail` holds the totals, so print it	notebook data: 3 14 12 12

Two remarks. In Experiment 3 the Child lines come out in order only because of the `sleep`; without it the order is up to the scheduler. In Experiment 10 the totals are correct only because the safety algorithm adds every Allocation back into `avail`: after a *safe* run, `avail` equals the total number of units of each type.

```
More practice changes, one for every experiment. Each was compiled and run.
(diff -U1: '-' = removed, '+' = added, other lines = unchanged context)

== variations/drill_exp2_bytes.c  (changed from exp2.c)
@@ -10,2 +10,3 @@
     ssize_t n;
+    long total = 0;
     printf("Enter filename: ");
@@ -24,2 +25,3 @@
     {
+        total += n;
         if (write(STDOUT_FILENO, buffer, n) != n)
@@ -33,2 +35,3 @@
     close(fd);
+    printf("Total bytes read: %ld\n", total);
     return 0;

== variations/drill_exp3_sleep.c  (changed from exp3.c)
@@ -18,2 +18,3 @@
         } else if(pid == 0) {
+            sleep(i);
             printf("Child %d with id %d and Parent Process id %d\n",

== variations/drill_exp4_bytes.c  (changed from exp4.c)
@@ -8,2 +8,3 @@
     int fd;
+    long total = 0;
     char buffer[1000], fname[100];
@@ -23,2 +24,3 @@
             write(fd, buffer, n);
+            total += n;
         }
@@ -34,3 +36,3 @@
     close(fd);
-    printf("Contents written to %s\n", fname);
+    printf("Contents written to %s (%ld bytes)\n", fname, total);
     return 0;
```

```
== variations/drill_exp5_usr1x3.c  (changed from exp5.c)
@@ -21,3 +21,6 @@
     {
+        static int count = 0;
         printf("Caught a User Signal\n");
+        if(++count == 3)
+            exit(0);
     }

== variations/drill_exp6_longest.c  (changed from exp6.c)
@@ -12,3 +12,3 @@
 {
-    int n, i;
+    int n, i, worst = 0;
     double awt = 0, atat = 0;
@@ -45,2 +45,4 @@
         atat += P[i].tat;
+        if(P[i].wt > P[worst].wt)
+            worst = i;
     }
@@ -50,2 +52,3 @@
     printf("Average Turn Around Time = %.2f\n", atat/n);
+    printf("Longest waiting time: P%d (%d)\n", P[worst].pid, P[worst].wt);
     return 0;

== variations/drill_exp7_gantt.c  (changed from exp7.c)
@@ -57,2 +57,8 @@
     }
+    printf("\n\nGantt chart:\n");
+    for(int i=0; i<n; i++)
+        printf("| P%d ", p[i].pid);
+    printf("|\n0");
+    for(int i=0; i<n; i++)
+        printf("%5d", p[i].tat);
     for(int i=0; i<n; i++)

== variations/drill_exp8_turns.c  (changed from exp8.c)
@@ -19,3 +19,3 @@
     struct Process p[n];
-    int remBT[n];
+    int remBT[n], turns[n];
     for(i=0; i<n; i++)
@@ -26,2 +26,3 @@
         remBT[i] = p[i].bt;
+        turns[i] = 0;
     }
@@ -42,2 +43,3 @@
                 flag=0;
+                turns[i]++;
                 if(remBT[i] > tq)
@@ -63,2 +65,5 @@
     }
+    printf("\n");
+    for(i=0; i<n; i++)
+        printf("P%d got %d turn(s)\n", p[i].pid, turns[i]);
     avgwt = (float) totwt/n;

== variations/drill_exp9_show.c  (changed from exp9.c)
@@ -30,2 +30,6 @@
                     item++;
+                    printf("Buffer now:");
+                    for(int k=0; k<item; k++)
+                        printf(" %d", buffer[(out+k) % buffersize]);
+                    printf("\n");
                 }
```

```
@@ -43,2 +47,6 @@
                     item--;
+                    printf("Buffer now:");
+                    for(int k=0; k<item; k++)
+                        printf(" %d", buffer[(out+k) % buffersize]);
+                    printf("\n");
                 }

== variations/drill_exp10_total.c  (changed from exp10.c)
@@ -65,2 +65,5 @@
         printf(" P%d\t", safeseq[k]+1);
+    printf("\nTotal units of each resource type:");
+    for(j=0; j<r; j++)
+        printf(" %d", avail[j]);
     return 0;
```

APPENDIX F

Two mock papers (with answers)

Each paper is 40 marks and about 45 minutes. Do it on paper first, then compare with the answers and mark yourself. Every program in the answers was compiled and run, and the scheduling answers were also checked with an independent simulator. Together the two papers touch all ten experiments.

F.1 Mock paper A (Experiments 3, 6–10)

1. **Experiment 3 (10 marks).** Write a C program that creates 3 child processes. Each child prints its own PID and its parent's PID and exits with a status equal to its number (1, 2, 3). The parent waits for every child and prints each child's PID and exit status.
2. **Experiments 6–8 (15 marks).** Processes (arrival, burst): P1 (0, 6), P2 (1, 8), P3 (2, 7), P4 (3, 2). Draw the Gantt chart and find the average waiting time for FCFS, non-pre-emptive SJF, SRTF and Round Robin with quantum 4. Which algorithm is best here, and why?
3. **Experiment 10 (10 marks).** 5 processes, 3 resource types. Max = (7 5 3), (3 2 2), (9 0 2), (2 2 2), (4 3 3). Allocation = (0 1 0), (2 0 0), (3 0 2), (2 1 1), (0 0 2). Available = (3 3 2). (a) Compute the Need matrix. (b) Is the state safe? Give a safe sequence. (c) Can a request from P2 for (1, 0, 2) be granted at once? (d) After that, can P1's request for (0, 2, 0) be granted?
4. **Experiment 9 (5 marks).** A bounded buffer has 3 slots. Give the starting values of `empty`, `full` and `mutex`. What happens if the consumer runs first? Why must the producer call `wait(empty)` before `wait(mutex)`?

Answers to paper A

Q1. Loop three times: `fflush(stdout)`, `fork()`; the child prints `getpid()` and `getppid()` and calls `exit(i)`; the parent loops on `wait(&status)`, printing the PID and `WEXITSTATUS(status)` when `WIFEXITED(status)` is true, and stops when `wait` returns -1. This is `exp3_status.c` with 5 changed to 3 (file `code/mock/mock_q1.c`):

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int i, status;
    for (i = 1; i <= 3; i++) {
        fflush(stdout);
        pid_t pid = fork();
        if (pid < 0) { perror("fork"); return 1; }
        if (pid == 0) {                       /* child */
            printf("Child %d: pid=%d parent=%d\n", i, getpid(), getppid());
            exit(i);                          /* exit status = child number */
        }
        printf("Parent created child %d with pid %d\n", i, pid);
    }
```

```c
    while (1) {                             /* reap children until none are left */
        pid_t done = wait(&status);
        if (done == -1) break;              /* ECHILD: no more children */
        if (WIFEXITED(status))
            printf("Parent: child %d ended with status %d\n", done, WEXITSTATUS(status));
    }
    printf("Parent %d: all children finished\n", getpid());
    return 0;
}
```

A real run (the PIDs change every time; the statuses 1, 2, 3 are what matters):

```
Parent: child 84 ended with status 1
Parent: child 86 ended with status 2
Parent: child 87 ended with status 3
Parent 83: all children finished
```

Q2.

Algorithm	Gantt chart	Waits P1–P4	Average
FCFS	P1 0–6, P2 6–14, P3 14–21, P4 21–23	0, 5, 12, 18	8.75
SJF	P1 0–6, P4 6–8, P3 8–15, P2 15–23	0, 14, 6, 3	5.75
SRTF	P1 0–3, P4 3–5, P1 5–8, P3 8–15, P2 15–23	2, 14, 6, 0	5.50
RR, $q = 4$	P1 0–4, P2 4–8, P3 8–12, P4 12–14, P1 14–16, P2 16–20, P3 20–23	10, 11, 14, 9	11.00

SRTF is best: at time 3, P4 needs only 2 units while P1 still needs 3, so P4 takes the CPU and the short job never waits behind long ones. Round Robin is worst here because it keeps cutting the long jobs into slices. For Round Robin I used the usual rule that a process arriving during a slice joins the queue before the process that was just pre-empted; some books do the opposite, so follow your teacher's rule.

Q3.

- (a) Need = Max − Allocation: P1 (7,4,3), P2 (1,2,2), P3 (6,0,0), P4 (0,1,1), P5 (4,3,1).
- (b) Safe. One safe sequence: **P2, P4, P5, P1, P3**.
- (c) Yes. The request is within P2's Need (1,2,2) and within Available (3,3,2). After the pretend grant the state is still safe (sequence P2, P4, P5, P1, P3).
- (d) No. Available is now (2,3,0). The request fits P1's Need and Available, but granting it would leave the system unsafe, so P1 must wait.

Q4. `empty` = 3 (all slots free), `full` = 0, `mutex` = 1. If the consumer runs first, `wait(full)` finds 0 and it sleeps until the producer posts `full`. With the waits the other way round, a producer that holds `mutex` and then sleeps on `empty` (buffer full) stops the consumer from getting `mutex`, so nobody can free a slot: a deadlock. I reproduced this and the program froze.

F.2 Mock paper B (Experiments 1, 2, 4, 5)

1. **Experiment 1 (8 marks).** (a) Change the shell script of Experiment 1 so that it prints the *smallest* of three numbers. (b) Explain the four arguments of `execl("/bin/sh", "sh", sname, (char*) NULL)`. What happens to the lines after it when it succeeds?
2. **Experiments 2 and 4 (14 marks).** Write a program that asks for a source file name and a destination file name and copies the source to the destination using only `open`, `read`, `write` and `close`.

Print the reason with `perror` if a file cannot be opened. The destination must be created if it is missing and emptied if it exists.

3. **Experiment 5 (10 marks).** Write a program that installs a handler for `SIGINT`. Each Ctrl+C prints how many times it has been pressed, and at the third press the program ends. `main` waits with `pause()` in a loop.
4. **Concepts (8 marks).** (a) Difference between `fork` and `exec`? (b) What is a zombie process? (c) A handler interrupts `sleep(5)` after 2 seconds. What does `sleep` return? (d) Why does `read` return 0 when you press Ctrl+D on an empty line?

Answers to paper B

Q1. (a) Replace `-ge` by `-le` and "greatest" by "smallest" (file `code/mock/smallest.sh`); `-le` rather than `-lt` is what makes ties work. I ran it through the launcher: `7 3 9` gives 3 and `5 5 8` gives 5.

```
#!/bin/sh
printf "Enter 3 numbers: "
read a b c
if [ "$a" -le "$b" ] && [ "$a" -le "$c" ]
then
    echo "$a is the smallest"
elif [ "$b" -le "$a" ] && [ "$b" -le "$c" ]
then
    echo "$b is the smallest"
else
    echo "$c is the smallest"
fi
```

(b) `"/bin/sh"` is the program to start; `"sh"` becomes `argv[0]`, the program's own name; `sname` becomes `argv[1]`, the script to run; `(char*)NULL` ends the argument list. When `execl` succeeds it never returns, because the process now runs the shell; the lines after it are reached only if `execl` failed.

Q2. Marks: open the source 2, open the destination with the right flags and mode 3, the loop that writes n bytes 4, error handling 3, closing both files 2. File `code/mock/copy.c`:

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main()
{
    char src[100], dst[100], buffer[100];
    int in, out;
    ssize_t n;
    printf("Enter source file: ");
    scanf("%99s", src);
    printf("Enter destination file: ");
    scanf("%99s", dst);
    in = open(src, O_RDONLY);
    if (in == -1)
    {
        perror(src);
        return 1;
    }
    out = open(dst, O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (out == -1)
    {
        perror(dst);
        close(in);
```

```
        return 1;
    }
    while ((n = read(in, buffer, sizeof(buffer))) > 0)
    {
        if (write(out, buffer, n) != n)
        {
            perror("write");
            break;
        }
    }
    if (n == -1)
        perror("read");
    close(in);
    close(out);
    printf("Copy finished\n");
    return 0;
}
```

My run copied a 7 000-byte file and `cmp` found no difference. A missing source printed `nofile.txt: No such file or directory`. The new file had the permissions `-rw-r--r--`.

Q3. File `code/mock/sigcount.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

int count = 0;

void handler(int sig)
{
    count++;
    printf("Ctrl+C pressed %d time(s)\n", count);
    if(count == 3)
    {
        printf("Third time: exiting\n");
        exit(0);
    }
}

int main()
{
    signal(SIGINT, handler);
    printf("PID = %d\n", getpid());
    while(1)
        pause();
    return 0;
}
```

`pause()` suspends the process until a signal is caught, so there is no busy loop. `count` is global so it keeps its value between calls. I sent three `kill -INT` signals: it printed 1, 2, 3 and ended itself.

Q4. (a) `fork` makes a new process, a copy of the caller; `exec` replaces the program inside the same process and keeps the PID. (b) A child that has ended but whose parent has not yet called `wait`; only its small record (PID and exit status) remains. (c) About 3: `sleep` returns the number of seconds left when a handler interrupts it. (d) Ctrl+D is the terminal's end-of-file key; on an empty line it delivers zero bytes, so `read` returns 0.